

QASH Protocol Specification v1.0

Normative Reference — Pre-Genesis Lock

QASH Protocol Authors

2026-05-29

Contents

1	Preamble	8
2	QASH Execution Model	8
2.1	Purpose	8
2.2	§E0 — Execution Domain Partition (Protocol Law)	9
2.2.1	Domain A — Deterministic Consensus Domain	9
2.2.2	Domain B — PAL / Operational Domain	9
2.2.3	Boundary rules	10
2.3	§E1 — Integer Arithmetic Law	10
2.3.1	Primitive types	10
2.3.2	Arithmetic semantics	11
2.3.3	Canonical Σ_i, t update rule	11
2.3.4	Shift operations	11
2.3.5	Fixed-point arithmetic	11
2.4	§E2 — Forbidden Operations	12
2.4.1	Floating point	12
2.4.2	Nondeterministic ordering	12
2.4.3	Allocator-dependent behavior	13
2.4.4	Undefined behavior	13
2.4.5	Clock and entropy	13
2.4.6	Panics and exits	13
2.5	§E3 — Memory Model	13
2.5.1	Static bounds	13
2.5.2	Endianness	13
2.5.3	Alignment	14
2.6	§E4 — Hash Primitive Law	14
2.6.1	Consensus hash suite	14
2.6.2	Domain separation	14
2.6.3	Hash cascade	15
2.7	§E5 — Absorbing Halt Semantics	15
2.7.1	Trigger conditions	15
2.7.2	Halt behavior	16
2.7.3	Succession	16

2.8	§E6 — ISA Support Policy	16
2.8.1	Authorized platforms	16
2.8.2	Verification requirement	17
2.8.3	ISA-specific restrictions	17
2.9	§E7 — Compilation Requirements	17
3	QASH Consensus Specification	17
3.1	Notation	17
3.2	§0 — Deterministic Execution Constraints	18
3.2.1	Forbidden operations	18
3.2.2	Required properties	19
3.2.3	Fixed-point arithmetic rules	19
3.3	§0b — Adversarial Scope and Claim Separation	19
3.4	§1 — State Space	19
3.4.1	Admissibility constraints	21
3.5	§2 — Canonical Encoding	21
3.5.1	Encoding vs state root computation	21
3.5.2	Prior root as a total function	21
3.5.3	substitute_root	22
3.5.4	Encoding ontology (normative)	22
3.5.5	Wire format	23
3.5.6	Deterministic rejection	24
3.5.7	Genesis hash procedure (normative)	24
3.6	§3 — Transition Function	25
3.6.1	Input set	25
3.6.2	Transition steps	25
3.7	§3.11 — Causal Fingerprint	26
3.7.1	Definition	26
3.7.2	Properties	26
3.7.3	Implementation	27
3.8	§4 — Stability Functions	27
3.8.1	§4a — Operational Lyapunov Candidate $V_{convergence}$	27
3.8.2	§4c — Cascade Health Factor CH_t	28
3.8.3	§4b — Safety Accumulator Φ_{safety}	29
3.9	§5 — Stability Criterion	30
3.9.1	Condition 1 — Convergence gate	30
3.9.2	Condition 2 — Safety admissibility gate	31
3.9.3	Combined criterion	31
3.9.4	Epoch gating invariant	31
3.10	§6 — Replay Invariance Theorem Statement	31
3.10.1	Theorem RT-1 (Replay Invariance)	32
3.10.2	Proof obligations	32
3.10.3	Corollary RT-2 (Succession Soundness)	32
3.11	§7 — Theorem Dependency Graph	32
3.11.1	Axiom layer (class: ASSUMED — trusted, not proved within the system)	33
3.11.2	Theorem and verification claim layer	33
3.11.3	Dependency graph (ASCII)	35

3.11.4	Genesis lock gate	35
3.12	Appendix A — Binding of Genesis Constants	36
3.13	Appendix B — Relationship to 00_execution_model.md	37
4	QASH Transition Axioms	37
4.1	Purpose and Position in the Spec Stack	37
4.2	Notation	37
4.3	§A0 — Transition Admissibility	38
4.4	§A1 — Determinism Axiom	38
4.5	§A2 — Termination Axiom	39
4.6	§A3 — State Locality Axiom	39
4.7	§A4 — Encoding Preservation Axiom	40
4.8	§A5 — Replay Preservation Axiom	40
4.9	§A6 — Halt Monotonicity Axiom	40
4.10	§A7 — Domain Separation Axiom	41
4.11	§A8 — δ _window Compatibility Axiom	41
4.11.1	Form A — Non-increase proof (strong form)	41
4.11.2	Form B — Bounded perturbation proof (acceptable form)	41
4.11.3	Form C — Φ _safety-only effect (special case)	42
4.11.4	No other forms are admissible	42
4.11.5	Connection to TH-3	43
4.12	§A9 — Halt Propagation Rule	43
4.13	§A10 — Transition Composability	43
4.13.1	Canonical ordering law	44
4.14	§A11 — Minimal Admissible Transaction Set (Initial)	44
4.15	Appendix — Transaction Admission Template	45
5	QASH Transaction Semantics	45
5.1	Purpose and Position in the Spec Stack	45
5.2	Notation (aligned with 02_transition_axioms.md)	46
5.3	§0 — Transaction Model	46
5.3.1	Transaction universe	46
5.3.2	Ontology invariant	47
5.3.3	Transaction lifecycle	47
5.4	§1 — Canonical Transaction Encoding	47
5.4.1	Envelope structure (frozen)	47
5.4.2	Encoding rules	47
5.4.3	Envelope wire format	47
5.4.4	TxID computation	48
5.4.5	Decode rejection rules	48
5.5	§2 — Transaction Admissibility	48
5.5.1	General admissibility predicate	48
5.5.2	Admissibility class	49
5.5.3	Failure ontology recap	49
5.5.4	Nonce semantics	49
5.6	§3 — Transaction Application Semantics	50
5.6.1	Transition function contract	50
5.6.2	Touch-set declaration	50

5.6.3	Effect declarations	50
5.7	§4 — Deterministic Ordering	50
5.7.1	Sort key definition	50
5.7.2	Ordering relation	51
5.7.3	Uniqueness guarantee	51
5.7.4	Admission ordering invariant	51
5.8	§5 — Stability Budget System (§A8 Interface)	51
5.8.1	Perturbation budget declaration	51
5.8.2	Epoch budget constraint	51
5.8.3	Budget accounting function	52
5.8.4	Proof obligation	52
5.9	§6 — Slash Semantics	52
5.9.1	Slash increment bound	52
5.9.2	Transactions with $\sigma_{\tau} = 0$	52
5.9.3	Slash-producing transactions	52
5.9.4	Post-genesis transaction types (economic)	53
5.10	§7 — Replay and Idempotence	53
5.10.1	Idempotence law	53
5.10.2	Replay safety	53
5.10.3	Deduplication in input sets	53
5.11	§TX-0 — No-Op Transaction	53
5.11.1	Purpose	53
5.11.2	Payload schema	54
5.11.3	Type-specific decode failures	54
5.11.4	Admissibility predicate	54
5.11.5	Transition function	54
5.11.6	Touch-set	54
5.11.7	Effect declarations	55
5.11.8	Declared bounds	55
5.11.9	§A8 form: Form A (Non-increase)	55
5.11.10	Proof obligation	55
5.11.11	Idempotence	55
5.11.12	Rejection taxonomy specific to TX-0	56
5.11.13	Test vectors	56
5.12	§TX-1 — BoundedValidatorScoreDecrement	57
5.12.1	Purpose	57
5.12.2	Wire format	57
5.12.3	Type-specific decode failures	58
5.12.4	Admissibility predicate	58
5.12.5	Transition function	58
5.12.6	Invariant impact	59
5.12.7	§A8 properties	59
5.12.8	§A1 determinism argument	59
5.12.9	§A2 termination bound B_{τ}	59
5.12.10	§A4 StateWF preservation	59
5.12.11	Rejection taxonomy	60
5.12.12	Test vectors	60
5.13	Appendix A — Transaction Type Registry	62

5.14	Appendix B — Cross-Document Dependency	62
6	QASH Adversarial Model (Scoped)	62
6.1	§A0 — Scope and Partition Alignment	63
6.2	§A1 — Scenario Classes	63
6.2.1	A1.1 Network partitions (Domain B)	63
6.2.2	A1.2 Message reorderings / duplication (Domain B)	63
6.2.3	A1.3 Byzantine input construction (boundary adversary)	63
6.3	§A2 — Safety vs Liveness Separation	63
6.3.1	Safety claims (must hold under all A1 scenarios)	63
6.3.2	Liveness claims (conditional)	64
6.4	§A3 — Theorem Obligations by Scenario Class	64
6.5	§A4 — Simulation Hooks and CI Progression	64
7	QASH Astronomical Hash Cascade	64
7.1	Purpose	65
7.2	Formal Definition	65
7.2.1	Layer 1 — Parallel primitive application	65
7.2.2	Layer 2 — Binding	65
7.2.3	Layers 3-6 — Recursive expansion	66
7.2.4	Layer 7 — Finalization	66
7.3	Domain Separators	66
7.4	Cascade Proof Format	66
7.5	Cascade Health Factor Derivation	67
7.6	Implementation Notes	67
7.6.1	Cross-ISA determinism	67
7.6.2	Parallelism	67
7.6.3	No streaming	67
7.7	Relationship to DD-3 (SHA3-256 as canonical consensus hash)	67
7.8	§4 — Cascade Blinding (Context-Keyed Variant)	68
7.8.1	Formal definition	68
7.8.2	Blinding in cascade proofs	68
7.8.3	Genesis hash blinding nonce	69
7.9	§5 — Hierarchical Deterministic Cascade Derivation	69
7.9.1	Epoch cascade root chain	69
7.9.2	Security properties	69
7.10	§6 — Implementation Requirements	70
8	QASH Canonical Test Vectors	70
8.1	Purpose	70
8.2	Notation	70
8.3	Test Vectors	71
8.3.1	TV-0 — Genesis state root	71
8.3.2	TV-0a — Genesis epoch-1 state-root commitment KAT	71
8.3.3	TV-1 — 3-epoch idle root (TH-7 anchor)	71
8.3.4	TV-2 — TX-0 nonce advance	72
8.3.5	TV-3 — TX-0 replay rejection	72
8.3.6	TV-4 — TX-0 unknown author rejection	72

8.3.7	TV-5 — Halt trigger (H1: Lyapunov violation)	72
8.3.8	TV-6 — Halt is absorbing	73
8.3.9	TV-7 — Entropy chain determinism	73
8.3.10	TV-8 — Hash cascade determinism	73
8.4	Gate Rule (TH-7)	73
8.5	Regeneration Procedure	74
9	QASH Protocol Migration: v1.0 → v1.1	74
9.1	Summary of Changes	74
9.2	Compatibility Window	74
9.2.1	Window semantics	75
9.2.2	State conversion	75
9.3	Weight Change Impact on $\Phi_{\text{max_safe}}$	75
9.4	Cascade Health (CH) — New Term in $V_{\text{convergence}}$	75
9.5	Clone Protocol v1.2 Changes	76
9.6	Formal Obligations	76
9.6.1	Genesis Lock Gate — Current Status (v1.0)	77
10	QASH Privacy Model	77
10.1	§P0 — Privacy posture	77
10.2	§P0a — What QASH does NOT protect against (normative)	77
10.3	§P1 — Dual-layer network architecture	78
10.4	§P2 — Transaction lifecycle visibility boundaries	78
10.5	§P3 — Public observation transcript	79
10.5.1	§P3a — PublicTranscript change-control process (normative)	79
10.6	§P4 — Observer classes	80
10.6.1	§P4a — Normative class taxonomy (Phase 4-A)	80
10.6.2	§P4b — Observer class summary table	81
10.7	§P5 — Validator identity vs user identity	82
10.8	§P6 — Leaf obfuscation	83
10.9	§P7 — Receipt privacy	83
10.10	§P8 — OEM/blinding trust boundary	83
10.11	§P9 — Offline clone privacy	84
10.12	§P10 — Theorem targets	84
11	QASH Clone Protocol — §10	85
11.1	§10.0 — Purpose and scope	85
11.2	§10.1 — Transport layer abstraction	85
11.3	§10.2 — Ultrasonic transport (§10.2 normative; optional/experimental)	86
11.4	§10.3 — Transport selection and priority	87
11.5	§10.4 — Dual-mode BLE operation	87
11.6	§10.5 — Store-and-forward relay	88
11.7	§10.6 — Cover traffic and timing obfuscation	88
11.8	§10.7 — Binary packet format and compression	88
11.9	§10.8 — Bloom filter deduplication	89
11.10	§10.9 — Emergency wipe	89
11.11	§10.10 — Offline operation limits	89
11.12	§10.11 — Security properties	90

11.1§10.12 — Out of scope (deferred)	90
12 QASH ABCR Handshake Protocol	90
12.1§H0 — Transport Security	91
12.2§H1 — ABCR Protocol (normative)	91
12.2.1Step 1 — Challenge (Sender → Recipient)	91
12.2.2Step 2 — Attested Response (Recipient → Sender)	91
12.2.3Step 3 — Verification (Sender)	92
12.3§H2 — Security Properties	92
12.4§H3 — Threat Model	93
12.5§H4 — Integration Layer	93
12.6§H5 — Implementation Notes	93
13 QASH Security Model	94
13.1§11.1 Threat Surface	94
13.2§11.2 Domain Partition Security (Domain A / Domain B)	94
13.3§11.3 Absorbing Halt Containment	95
13.4§11.4 Clone Protocol Bounded Reconciliation	95
13.5§11.5 Tolerance-Based Divergence Containment (Bat Immunology Model)	96
13.5.1Formal Mapping	96
13.5.2Tolerance Detection Equation	96
13.5.3Formal Guarantee: TH-GC (Grace Convergence)	97
13.5.4Why Cross-ISA Divergence Does Not Split the Network	97
13.6§11.6 Formal Guarantee Summary	97
14 QASH Sharded Protocol	98
14.1§12.1 — Scope	98
14.2§12.2 — Shard Assignment	98
14.3§12.3 — Shard Commitments	99
14.4§12.4 — Cross-Shard Receipts	99
14.5§12.5 — Epoch Finality Beacon	100
14.6§12.6 — ZK-STARK Verification Boundary	101
14.7§12.7 — Public Transcript	102
14.8§12.8 — Implementation Mapping	102
15 QASH Zero-Persistence Execution Pipeline	103
15.1§14.1 — Purpose	103
15.2§14.2 — Definitions	103
15.3§14.3 — Normative Invariants	104
15.3.1ZP-1: No durable raw graph material	104
15.3.2ZP-2: No owned payload copies in the admission path	104
15.3.3ZP-3: Commitment-only Domain A crossing	104
15.3.4ZP-4: Blind audit only	105
15.3.5ZP-5: Feature bifurcation	105
15.4§14.4 — Admission Lifecycle	105
15.5§14.5 — WAL and Replay Boundary	106
15.6§14.6 — Hardware Profiles	106
15.6.1Global Standard profile	106

15.6.2	Sovereign Hardened profile	107
15.7§14.7	— CI and Verification Gates	107
15.8§14.8	— Claims Boundary	107
16	QASH Trustless Genesis and Vendor-Agnostic Local Hardware OpSec	108
16.1§15.1	— Purpose	108
16.2§15.2	— Trustless Genesis Axiom	108
16.3§15.3	— Anti-Ceremony Rules	109
16.3.1	AC-1: No trusted setup	109
16.3.2	AC-2: No genesis cabal	109
16.3.3	AC-3: No protocol-critical vendor hardware	109
16.3.4	AC-4: No human liveness dependency	109
16.3.5	AC-5: No hardware-derived consensus semantics	109
16.3.6	AC-6: Standards before vendors	109
16.4§15.4	— Permitted Local Hardware OpSec	110
16.5§15.5	— Vendor-Agnostic Hardware Boundary	110
16.6§15.6	— Public Transcript Privacy Rule	111
16.7§15.7	— Failure Semantics	111
16.8§15.8	— Review Gate	112

1 Preamble

This document is the normative specification for the QASH protocol, version 1.0. It is the authoritative source of truth for protocol intent, behaviour, and compliance requirements.

Genesis lock status: Pre-lock. Until all evidence-gate conditions in docs/traceability.md are satisfied and genesis_status = "authoritative" is set in GENESIS_CONSTANTS.toml, this document is provisional. All page references in docs/traceability.md, docs/errata/, and docs/adr/ must be verified against the committed PDF before the genesis lock tag (v1.0-reference) may be applied.

Domain partition: The protocol enforces a hard boundary between Domain A (deterministic consensus — crates/consensus/) and Domain B (PAL/operational — crates/pal/). Domain B values must never flow into Domain A computations. See §2 (Execution Model) for the full partition rules.

2 QASH Execution Model

2.1 Purpose

This document defines the **deterministic execution substrate** on which all consensus logic runs. It is not about what the protocol computes — that is 01_consensus.md — but about the computational laws under which it computes.

Every implementation must satisfy this document before it can claim to implement QASH.

2.2 §E0 — Execution Domain Partition (Protocol Law)

All protocol execution is permanently partitioned into two domains. This partition is not an implementation guideline — it is protocol law. Any code, data flow, or value that violates the partition boundary makes the implementation non-conforming regardless of test results.

2.2.1 Domain A — Deterministic Consensus Domain

Everything subject to replay invariance and formal proof.

Scope:

- State transition function $T(S_t, I_t)$
- $V_{convergence}(S_t)$ and $\Phi_{safety}(S_t)$
- Canonical encoding $Encode(S_t)$
- State root computation R_t
- Input admissibility verification
- Epoch advancement and halt evaluation
- All cryptographic operations over consensus data

Properties (enforceable, not aspirational):

- Replay invariant across all Tier A authorized ISAs
- Proof-eligible: formal theorems may quantify over Domain A execution
- No unsafe blocks
- No entropy ingress from outside the protocol
- No allocator-order-dependent behavior
- No nondeterministic iteration order
- All arithmetic checked; overflow triggers absorbing halt

Unsafe blocks: FORBIDDEN

2.2.2 Domain B — PAL / Operational Domain

Everything excluded from replay invariance scope.

Scope:

- Wall-clock time and epoch timing
- Network I/O (send/recv)
- Hardware attestation
- OS scheduler interaction
- Logging, instrumentation, metrics

- Human operator interfaces
- Cryptographic acceleration (SIMD, hardware AES, etc.)
- Zero-copy serialization with platform-specific alignment

Properties:

- Nondeterministic behavior permitted
- Hardware-specific optimization permitted
- unsafe blocks permitted under formal audit
- Excluded from all replay theorem scopes (TH-1 through TH-8)
- Must NOT pass nondeterministic values into Domain A

Unsafe blocks: PERMITTED under PAL audit

2.2.3 Boundary rules

The following are **unconditional boundary violations**:

VIOLATION: Any value originating in Domain B that influences a Domain A computation

VIOLATION: Any Domain A function that calls a Domain B function

VIOLATION: Clock, entropy, or network state used as input to $T(S_t, I_t)$

VIOLATION: Attestation results used to alter state transition semantics

The following are **permitted boundary crossings**:

PERMITTED: Domain A outputs (state roots, halt signals) passed to Domain B

PERMITTED: Domain B providing externally-signed inputs I_t that are then verified by Domain A admissibility checks before use

PERMITTED: Domain B invoking Domain A replay verification (read-only)

Cross-domain contamination is a protocol violation. An implementation may pass all tests and still be non-conforming if a Domain B value reaches Domain A computation.

2.3 §E1 — Integer Arithmetic Law

All arithmetic in the deterministic domain obeys the following laws without exception.

2.3.1 Primitive types

Only the following integer types are permitted:

u8, u16, u32, u64, u128

i8, i16, i32, i64, i128

bool (encoded as u8: 0x00 = false, 0x01 = true)

Types `usize` and `isize` are **forbidden** in the deterministic domain because their width is platform-dependent.

2.3.2 Arithmetic semantics

The deterministic domain uses **checked arithmetic with absorbing halt** throughout. There is no saturating arithmetic in Domain A. The $\Sigma_{i,t}$ slash accumulator uses an explicit pre-checked conditional cap — this is state transition semantics, not arithmetic semantics. See `01_consensus.md §4b`.

Context	Semantics
All Domain A arithmetic	Checked: overflow triggers absorbing halt
Lyapunov intermediate products	Checked i128
Epoch counter	Checked u64
$\Sigma_{i,t}$ slash accumulator	Pre-checked cap at <code>INT_MAX</code> via conditional, not saturation
Encoding byte offsets	Checked u32 or u64

No arithmetic in Domain A may be unchecked, wrapping, or saturating.

2.3.3 Canonical $\Sigma_{i,t}$ update rule

The slash accumulator cap is a **control-flow decision**, not arithmetic saturation. The canonical form is:

```
// Both operands checked before comparison – overflow here triggers absorbing halt
let sum: i128 = (current as i128) + (increment as i128);
if sum > INT_MAX {
    current = INT_MAX;           // control-flow assignment, not saturation
} else {
    current = sum as i64;        // safe: sum ≤ INT_MAX = 2^63 - 1
}
```

The intermediate sum is computed as i128 so the comparison `sum > INT_MAX` is itself overflow-free. If increment is somehow out of the i64 range (which admissibility constraints must prevent), the i128 addition will still not overflow, but the transition that produced such an increment is inadmissible and must be rejected before this point.

2.3.4 Shift operations

All bit shifts must be by constant or explicitly range-checked values. Shift by $\geq$ bit-width is a protocol halt condition.

2.3.5 Fixed-point arithmetic

All protocol values that represent fractional quantities use the $\square_p$ representation:

```

scale:      p = 1_000_000
width:      i64 for storage, i128 for intermediate computation
rounding:    floor toward negative infinity
              (for positive quotients: integer division; for negative: subtract 1 if remainder ≠ 0)

```

The rounding rule is canonically defined as:

```

floor_div(a: i128, b: i128) -> i128:
    q = a / b                // Rust truncating division
    r = a % b
    if r != 0 && (a ^ b) < 0: // signs differ and there is a remainder
        q - 1
    else:
        q

```

Implementation note: Rust’s default / operator for signed integers truncates toward zero, which differs from `floor_div` when the result is negative with a non-zero remainder. QASH requires Euclidean (floor) division throughout Domain A. Implementations must use `i128::div_euclid()` and `i128::rem_euclid()`, or explicitly apply the correction above. Using Rust’s default / for signed fixed-point division is a protocol violation that will cause replay divergence on negative intermediate values.

This function is part of the protocol and must produce identical results on all platforms.

2.4 §E2 — Forbidden Operations

The following operations are **unconditionally forbidden** in the deterministic domain. This list is exhaustive. If an operation is not explicitly permitted, it is forbidden.

2.4.1 Floating point

```

FORBIDDEN: f32, f64, f128 (if ever stabilized)
FORBIDDEN: any call to a function that internally uses floating-
point arithmetic
FORBIDDEN: SIMD intrinsics with floating-point lanes in the D domain
              (integer SIMD lanes are permitted subject to cross-
ISA determinism verification)

```

Rationale: IEEE 754 arithmetic may produce platform-divergent NaN representations, rounding differences under FMA fusion, and is not reproducible across all authorized ISAs.

2.4.2 Nondeterministic ordering

```

FORBIDDEN: HashMap, HashSet with random seeds (use BTreeMap, BTreeSet)
FORBIDDEN: Any collection whose iteration order depends on allocation address
FORBIDDEN: Parallel iterators over protocol state (rayon, etc.)

```

2.4.3 Allocator-dependent behavior

FORBIDDEN: Heap allocation in the deterministic domain
(all data structures must be statically bounded)
FORBIDDEN: Structure layout that differs across allocator versions
FORBIDDEN: Drop order that has semantic side-effects in the D domain

2.4.4 Undefined behavior

FORBIDDEN: unsafe blocks in Domain A (the deterministic consensus domain)
unsafe is permitted in Domain B (PAL) under formal audit – see §E0
FORBIDDEN: Any operation with defined-but-platform-dependent behavior
(e.g. integer overflow before explicit checked arithmetic)
FORBIDDEN: Transmuting types across endian boundaries

2.4.5 Clock and entropy

FORBIDDEN: `std::time::SystemTime`, `Instant`, or any wall-clock access in D domain
FORBIDDEN: os-provided random bytes as input to state transition
(protocol `entropy_seed` is the only admissible entropy source)

2.4.6 Panics and exits

FORBIDDEN: `panic!()` in the deterministic domain (must use `absorbing_reset()` instead)
FORBIDDEN: `unwrap()`, `expect()` on protocol-path code paths
(use explicit match or `if-let` with `halt` on failure)
FORBIDDEN: `std::process::exit()` outside of `Halt::absorbing_reset()`

2.5 §E3 — Memory Model

2.5.1 Static bounds

No unbounded, allocator-dependent, or nondeterministically-sized allocation is permitted in the deterministic domain. All data structures must have a statically-known maximum size derivable from genesis constants at compile time. Statically-bounded allocators are permitted provided their layout is deterministic and independent of allocation order.

2.5.2 Endianness

All multi-byte integer values in the deterministic domain are **little-endian** in canonical form. This applies to:

- All fields in `Encode(S_t)`
- All hash inputs that include integer values
- All cryptographic domain separation tags that include integers

2.5.3 Alignment

No code in the deterministic domain may rely on or assume any particular memory alignment. All reads and writes must be via properly-typed, alignment-safe operations.

2.6 §E4 — Hash Primitive Law

All cryptographic hash operations in the deterministic domain use the following canonical construction unless explicitly specified otherwise.

2.6.1 Consensus hash suite

State roots are not defined by a single primitive. Each active consensus primitive hashes the same domain-separated preimage, and the primitive outputs are folded into one 32-byte state root. The active v1 suite is:

```
SHA3-256(input) → [u8; 32]
SM3-256(input)  → [u8; 32]
```

SHA3-256 remains the fold/hash used for entropy advancement and `GENESIS_CONSTANTS.toml` lock hashing, but state-root security depends on the entire active digest set. A primitive that is merely logged outside this construction is not consensus-active.

Input must be a single contiguous byte slice. No streaming API is used in the D domain (streaming APIs permit platform-divergent buffering behavior).

2.6.2 Domain separation

All hash inputs in the D domain are domain-separated to prevent cross-context collisions. Domain tags are 4-byte little-endian u32 values prepended to the input.

Domain tag assignments:

0x00000001	STATE_ROOT	– Encode(S _t) → primitive sub-roots
0x00000002	ENTROPY_ADVANCE	– seed _t → seed _{t+1}
0x00000003	VALIDATOR_ID	– validator _{id} generation
0x00000004	LEAF_HASH	– sparse Merkle leaf
0x00000005	INTERNAL_HASH	– sparse Merkle internal node
0x00000006	CONSENSUS_ROOT	– folds active primitive sub-roots

Canonical single-primitive form:

```
H_domain(tag: u32, input: &[u8]) → [u8; 32]:
  SHA3-256( tag.to_le_bytes() □ input )
```

Consensus state-root form:

```
root_SHA3 = SHA3-256(STATE_ROOT.to_le_bytes() □ input)
root_SM3  = SM3-256(STATE_ROOT.to_le_bytes() □ input)
```

```
H_consensus_domain(STATE_ROOT, input) =
```

```

SHA3-256(
  CONSENSUS_ROOT.to_le_bytes()
  □ 2u32.to_le_bytes()
  □ SHA3_PRIMITIVE_ID.to_le_bytes() □ root_SHA3
  □ SM3_PRIMITIVE_ID.to_le_bytes() □ root_SM3
)

```

Validators must compute every active primitive. Divergence in any primitive changes the folded root and is consensus-visible.

2.6.3 Hash cascade

For operations requiring the full cascade (obfuscation leaf hashing, clone-chunk verification, cascade-health commitment):

$H_{\text{cascade}}(\text{input}: \&[u8]) \rightarrow [u8; 64]:$

Layer composition (depth = 7, mode = recursive_binding):

```

L1: h1_sha3  = SHA3-256( "QASH:CASCADE:L1:PARALLEL" □ input ) // [u8; 32]
    h1_blake3 = BLAKE3(  "QASH:CASCADE:L1:PARALLEL" □ input ) // [u8; 32]
    h1_k12    = K12(     "QASH:CASCADE:L1:PARALLEL" □ input ) // [u8; 32]
    h1_sm3    = SM3(     "QASH:CASCADE:L1:PARALLEL" □ input ) // [u8; 32]
    h1_streeb = Streebog( "QASH:CASCADE:L1:PARALLEL" □ input ) // [u8; 32]
    parallel  = h1_sha3 □ h1_blake3 □ h1_k12 □ h1_sm3 □ h1_streeb // [u8; 160]

L2: bound = SHA3-512( "QASH:CASCADE:L2:BIND" □ parallel ) // [u8; 64]

L3..L6: (recursive expansion – each layer feeds the next)
    L_n = SHA3-512( "QASH:CASCADE:L{n}:EXPAND" □ L_{n-1} ) // [u8; 64]

L7: output = SHA3-512( "QASH:CASCADE:L7:FINALIZE" □ L6 ) // [u8; 64]

```

The cascade is used for: obfuscation layer leaf hashing. It is **not** used for state root computation; state roots use $H_{\text{consensus_domain}}$ above.

2.7 §E5 — Absorbing Halt Semantics

An absorbing halt is an **irreversible terminal state** for the current network instance.

2.7.1 Trigger conditions

The following conditions unconditionally trigger absorbing halt:

```

H1: ΔV_t > ε (Lyapunov stability violation, see §5 of 01_consensus.md)
H2: i128 overflow in Lyapunov intermediate computation
H3: u64 overflow of epoch counter
H4: Decode(bytes) returns DecodeResult::Invalid on the local state root

```

H5: Canonical encoding of S_t does not round-trip through $\text{Decode}(\text{Encode}(S_t)) = S_t$
H6: `halt_flag = true` in any admitted S_t

2.7.2 Halt behavior

On halt trigger:

1. Freeze epoch advancement immediately.
No S_{t+1} is computed or committed.
2. Preserve the last valid state root R_{t-1} .
This root is the succession anchor.
3. Reject all incoming transitions.
Return `DecodeResult::Invalid` with reason `0xFF (HALTED)` for all future inputs.
4. Signal the PAL layer via `Halt::absorbing_reset()`.
On embedded targets: trigger hardware watchdog.
On hosted targets: `std::process::exit(1)`.
5. The halt is network-wide if and only if a quorum of validators halt on the same epoch. A single-validator halt does not halt the network; it removes that validator from the active set.

2.7.3 Succession

A successor network G' anchoring to R_{t-1} is the only valid recovery path. There is no in-protocol recovery from a halt. No governance mechanism can override it.

2.8 §E6 — ISA Support Policy

2.8.1 Authorized platforms

The deterministic domain is guaranteed to produce identical results on:

Tier A (primary, CI-verified):

- x86_64 with AVX2
- aarch64 with NEON
- riscv64 with V-extension (vector)

Tier B (authorized, verified on release):

- loongarch64
- riscv64 without V-extension (scalar fallback)
- arm (32-bit, Cortex-M, ITRON targets)

Tier C (authorized, attested via software-hash-Merkle):

- esp32s3
- itron_renesas_rx
- itron_renesas_rl78

2.8.2 Verification requirement

RT-1 (replay invariance) is tested across at minimum all Tier A platforms in CI on every push. Tier B and C platforms are verified on tagged release builds.

2.8.3 ISA-specific restrictions

x86_64: No x87 FPU instructions in D domain (use SSE2 integer ops only)
aarch64: No VFP or ASIMD floating-point lanes in D domain
riscv64: No F/D/Q extensions in D domain; Zbb permitted for bit operations
arm32: Thumb-2 only; no ARM NEON floating-point in D domain

2.9 §E7 — Compilation Requirements

To satisfy RT-1, builds must be reproducible across: - Compiler versions (pinned via `rust-toolchain.toml`) - Build timestamps (suppressed via `SOURCE_DATE_EPOCH=0`) - Link IDs (suppressed via `--build-id=none`) - Incremental artifacts (disabled via `CARGO_INCREMENTAL=0`) - Codegen units (single via `codegen-units=1` in release profile)

Any binary that is not reproducible under these constraints is not a conforming QASH build, regardless of whether its output passes test vectors.

End of *docs/spec/00_execution_model.md*

3 QASH Consensus Specification

3.1 Notation

Symbol	Meaning
S_t	Protocol state vector at epoch t
$S_t[k]$	Component k of state at epoch t
$T(S_t, I_t)$	State transition function
I_t	Canonical input set at epoch t
$V_{\text{convergence}}(S_t)$	Operational Lyapunov candidate — dynamic terms only (D, C, CH)
$\Phi_{\text{safety}}(S_t)$	Monotone safety accumulator — slash evidence only
CH_t	Cascade health factor: cascade verification pass rate, epoch-relative

Symbol	Meaning
δ_{window}	Rolling-window excursion: $V_{\text{convergence}}(S_t) - \min(\text{lyapunov_window})$. Not a temporal derivative — measures deviation from the rolling minimum, not $V_{\{t+1\}} - V_t$
ϵ	Convergence tolerance (<code>epsilon_threshold</code> in genesis)
Φ_{max}	Derived safety bound: $N_{\text{max}} \times \gamma \times \text{INT_MAX}$
$\Phi_{\text{max_safe}}$	Halt threshold: $\Phi_{\text{max}} / 2$
<code>INT_MAX</code>	$2^{63} - 1$ (pure arithmetic; maps to <code>i64::MAX</code> in Rust)
<code>N_max</code>	Maximum validator cardinality = 1024
$\mathbb{F}_p$	Fixed-point integer field, scale $p = 1\,000\,000$
<code>i128</code>	128-bit signed integer (intermediate arithmetic width)
$\perp$	Absorbing halt state (irreversible)
<code>G</code>	Genesis constants block (immutable)
R_n	$S_n.\text{state_root}$ — the stored consensus artifact; admissibility invariant defined in §2
$\parallel$	Concatenation
H_{domain}	Domain-separated hash (defined in <code>00_execution_model.md</code> §E4)

All scalar values are elements of $\mathbb{F}_p$ unless otherwise stated. All arithmetic in the deterministic domain uses **checked arithmetic with absorbing halt** on overflow. There is no saturating arithmetic in the consensus execution path. There is no floating point anywhere in the consensus execution path.

The single exception is $\Sigma_{i,t}$ (slash accumulator), which is explicitly capped at `INT_MAX` via a pre-checked conditional — not via saturation. See §4b.

3.2 §0 — Deterministic Execution Constraints

These constraints are **globally binding** across the entire consensus execution path. Any implementation that violates one is incorrect regardless of test passage.

3.2.1 Forbidden operations

The following are unconditionally forbidden in any code reachable from $T(S_t, I_t)$, $V(S_t)$, $\text{Encode}(S_t)$, or any function they call:

FORBIDDEN:

`f32`, `f64`, or any IEEE 754 floating-point type or operation

- nondeterministic iteration order (HashMap without deterministic seed, BTreeMap is permitted)
- allocator-dependent structure layout or ordering
- clock-derived state mutation (wall time must never enter state)
- host entropy in consensus execution (entropy sources are protocol-derived only)
- signed integer overflow (all arithmetic is checked; overflow triggers absorbing halt)
- architecture-dependent alignment assumptions
- platform-dependent integer widths (use explicit i8/u8/.../i128/u128 only)
- `std::process::exit` or `panic!()` outside of `absorbing_reset()`
- unsafe code blocks

3.2.2 Required properties

REQUIRED:

3.2.3 Fixed-point arithmetic rules

3.3 §0b – Adversarial Scope and Claim Separation

- Safety claims are unconditional with respect to transport behavior.
- Liveness claims are conditional and must state environmental assumptions.
- Consensus-state determinism claims are Domain-A claims and are independent of Domain-B message timing, ordering, and scheduling behavior.

The protocol state at epoch t is a tuple:

```

    ledger_root: [u8; 32], // root of the sparse Merkle accumulator
    entropy_seed: [u8; 32], // forward-secure: seed_{t+1} = SHA3-256(seed_t)
    lyapunov_window: [i64; W], // ring buffer of V_convergence(S_k) for k in [t-
W, t-1]
                                // stores the W PRECEDING values only –
excludes current epoch
                                // initialized to 0 for all k < 0 (pre-genesis padding)
    halt_reason: u8, // 0x00=None (running); 0x01–0x07=halt codes; once non-
zero, all transitions produce ⊥
)

```

Validator cardinality bound:

$N_{\max} = 1024$ (protocol law, not configurable)

N at genesis must satisfy $N \leq N_{\max}$. The serialized validator array always contains exactly N entries; unused slots do not exist. N is immutable after genesis.

Each validator record V_i is a tuple:

```

V_i = (
  id: [u8; 48], // STABLE public-key-derived validator identity
                // = H_domain(VALIDATOR_ID, public_key_bytes)[0..48]
                // Fixed at genesis; never changes across epochs.
                // Anchors slash accounting continuity.
  divergence: i64, //  $D_{i,t} \in [0, \text{SCALE}]$ ; deviation metric, defined in §4a
  conflict: i64, //  $C_{i,t} \in [0, \text{SCALE}]$ ; conflict metric, defined in §4a
  slash_acc: i64, //  $\Sigma_{i,t} \geq 0$  (monotone non-decreasing); defined in §4b
  nonce: u64, // per-author replay counter; advanced on each admitted TX
)

```

Implementation note: In the Rust reference implementation, `id` and `nonce` are stored as parallel arrays (`EpochState.validator_ids` and `EpochState.nonces`) rather than inside `ValidatorMetrics`, for cache efficiency. The spec presents all fields together under V_i for conceptual clarity. The wire encoding interleaves fields per validator slot (`divergence`, `conflict`, `slash_acc`, `nonce`, `id`) as defined in §2. There is no semantic difference between the two representations.

Validator identity vs Merkle leaf index: $V_i.id$ is the validator’s **stable consensus identity** — a 48-byte truncated hash of the validator’s public key, fixed at genesis. It never changes across epochs and anchors slash accounting continuity.

The obfuscation section of `GENESIS_CONSTANTS.toml` defines a separate **Merkle leaf index** construction: `validator_id(8) ⊔ epoch(8) ⊔ seed(32)`. This 48-byte epoch-relative concatenation is used exclusively for sparse Merkle tree leaf addressing — not for validator consensus identity. The two constructions are distinct and must not be conflated. Epoch-relative Merkle leaf indices change each epoch; the consensus identity $V_i.id$ never does.

3.4.1 Admissibility constraints

A state S_t is **admissible** if and only if:

1. $S_t.\text{epoch}$ is strictly greater than $S_{\{t-1\}}.\text{epoch}$
 2. $S_t.\text{halt_reason} = 0x00$ (None) (halt states are terminal; any non-zero halt_reason is inadmissible)
 3. $\forall i: S_t.\text{validators}[i].\text{divergence} \in [0, i64::\text{MAX}]$ (divergence is non-negative)
 4. $\forall i: S_t.\text{validators}[i].\text{slash_acc} \in [0, i64::\text{MAX}]$ (slash accumulator is non-negative)
 5. $S_t.\text{entropy_seed} \neq [0u8; 32]$ (zero seed is forbidden post-genesis)
 6. $S_t.\text{state_root} = H_{\text{consensus_domain}}(\text{STATE_ROOT}=0x00000001, \text{Encode_for_commitment}(S_t, \text{prior_root}(t) = S_{\{t-1\}}.\text{state_root}$ for $t \geq 1$; $[0u8; 32]$ for $t=0$.
This is the canonical commitment invariant defined in §2.
-

3.5 §2 — Canonical Encoding

ENCODING FREEZE DECLARATION

Canonical encoding is consensus identity. $\text{Encode}(S_t)$ as defined in this section is permanently frozen.

Any modification to the wire format — field order, field width, endianness, padding, domain tags, or type representation — constitutes a new protocol universe and requires a new genesis. There are no “compatible upgrades” to encoding. Serialization is ontology for a replay-first protocol: two states that encode identically are the same state; two protocols that encode differently are different protocols.

The SHA3-256 of this document is recorded in `GENESIS_CONSTANTS.toml` at genesis lock time. Any runtime that implements a different encoding is not a QASH implementation regardless of other conformance.

3.5.1 Encoding vs state root computation

$\text{Encode}(S_t)$ encodes the state **as-is**, including the current `state_root` field. This is used for: wire transmission, storage, and replay.

3.5.2 Prior root as a total function

```
prior_root :  $\mathbb{N} \rightarrow [u8; 32]$   
prior_root(0)      =  $[0u8; 32]$   
prior_root(t + 1) =  $S_t.\text{state\_root}$ 
```

This is the authoritative definition. All references to “prior root” use $\text{prior_root}(t)$. The function is total over all epoch indices, supporting structural induction in Coq.

3.5.3 substitute_root

```
substitute_root(S : State, new_root : [u8; 32]) → State
  S' where S'.state_root = new_root
         S'.f = S.f for all field f ≠ state_root
```

substitute_root is a pure record update. Then:

```
Encode_for_commitment(S, prior_root) := Encode(substitute_root(S, prior_root))
```

3.5.4 Encoding ontology (normative)

No other encoding variants are permitted anywhere in this protocol.

Use case	Function
Transmission, storage, replay, equality, decoding, TH-1	Encode
State root commitment (transition step 9 only)	Encode_for_commitment

Two states are consensus-identical iff $\text{Encode}(S_a) = \text{Encode}(S_b)$. Future documents must not introduce alternative serializations or zeroing variants.

$\text{Encode_for_commitment}(S, \text{prior_root})$ is a **commitment preimage constructor**. It is NOT an alternative wire encoding — Encode remains the sole canonical wire format. Encode_for_commitment is a deterministic state transformation used solely before hashing:

```
Encode_for_commitment(S, prior_root) := Encode(substitute_root(S, prior_root))
```

This is the **sole normative definition**. The informal notation $\text{Encode}(S$ with $\text{state_root} := \text{prior_root})$ is non-normative shorthand only and must not be treated as a separate semantic object in proofs or implementations.

The state root is then:

```
state_root_t = H_consensus_domain(STATE_ROOT, Encode_for_commitment(S_t, prior_root(t)))
```

Key invariant: The wire encoding of S_t always contains the commitment to S_{t-1} . The state_root field binds S_t to the prior state, creating a verifiable chain analogous to blockchain header chaining.

Genesis: $S_0.\text{state_root} = [0u8; 32]$ (prior_root at genesis = all-zeros).

Why not zeroing: Zeroing (the Encode_canonical_inputs approach) is equivalent at genesis but loses chain history at $t > 0$. Prior_root substitution makes chaining explicit and preserves the “Encode = identity” invariant.

$\text{Encode}(S_t)$ produces a deterministic byte sequence. This encoding is **protocol law**: any deviation is a consensus failure, not an implementation bug.

3.5.5 Wire format

All fields are serialized in the order listed. No padding. No alignment gaps. All integers are **little-endian**.

Encode(S_t): –
112 fixed bytes, then validators, then window

epoch:	u64	→ 8 bytes, little-endian
state_root:	[u8;32]	→ 32 bytes, verbatim
ledger_root:	[u8;32]	→ 32 bytes, verbatim
entropy_seed:	[u8;32]	→ 32 bytes, verbatim
halt_reason:	u8	→ 1 byte (valid values: 0x00–0x07; any other value is malformed)
	0x00	None (running)
	0x01	LyapunovViolation (H1)
	0x02	ArithOverflow (H2)
	0x03	EpochOverflow (H3)
	0x04	DecodeInvalid (H4)
	0x05	RoundtripFailure (H5)
	0x06	HaltFlagSet (H6, reserved)
	0x07	PhiSafetyViolation (H7)
pad:	[u8; 3]	→ 3 bytes, must be 0x00 0x00 0x00; non-zero is malformed
validator_count:	u32	→ 4 bytes, little-endian, must equal N from genesis
		– subtotal: 8+32+32+32+1+3+4 = 112 bytes
validators:	[Encode(V_i); N]	→ N × 80 bytes, concatenated, no separators

Encode(V_i): – 80 bytes per validator

divergence:	i64	→ 8 bytes, little-endian, two's complement
conflict:	i64	→ 8 bytes, little-endian, two's complement
slash_acc:	i64	→ 8 bytes, little-endian, two's complement
nonce:	u64	→ 8 bytes, little-endian
id:	[u8;48]	→ 48 bytes, verbatim
		– subtotal: 8+8+8+8+48 = 80 bytes

Lyapunov window: – 28 bytes

window_filled:	u8	→ 1 byte (∈ [0, W]; value > W is malformed)
pad:	[u8; 3]	→ 3 bytes, must be 0x00 0x00 0x00; non-zero is malformed
lyapunov_window:	[i64; W]	→ W × 8 bytes, little-endian, two's complement, W=3
		– subtotal: 1+3+24 = 28 bytes

Total encoded size (fixed and computable at genesis from N and W):

$|Encode(S_t)| = 112 + N \times 80 + 28$ bytes

Variable-length encoding is forbidden.

3.5.6 Deterministic rejection

Any input byte sequence that: - has incorrect total length, - contains a `bool` field with value other than `0x00` or `0x01`, - has `validator_count` $\neq N$, - or violates any admissibility constraint after decoding

must produce an identical, deterministic rejection result on all platforms:

`DecodeResult::Invalid(reason: u8)`

where `reason` is a protocol-defined error code. The error code is part of the canonical state.

3.5.7 Genesis hash procedure (normative)

At genesis lock time, the SHA3-256 of the canonical spec document set is computed and recorded in `GENESIS_CONSTANTS.toml` as `genesis_hash`. This commits the genesis network to an immutable document tree. `GENESIS_CONSTANTS.toml` itself is excluded to avoid circularity.

Document set (concatenated in lexicographic file-path order):

```
docs/spec/00_execution_model.md
docs/spec/01_consensus.md
docs/spec/02_transition_axioms.md
docs/spec/03_transactions.md
```

Computation:

```
python3 -c "
import hashlib, pathlib
files = sorted([
    'docs/spec/00_execution_model.md',
    'docs/spec/01_consensus.md',
    'docs/spec/02_transition_axioms.md',
    'docs/spec/03_transactions.md',
])
h = hashlib.sha3_256()
for f in files:
    h.update(pathlib.Path(f).read_bytes())
print('SHA3-256:' + h.hexdigest())
"
```

Format in `GENESIS_CONSTANTS.toml`: `genesis_hash = "SHA3-256:<64 lowercase hex digits>"`

Any subsequent modification to the above four documents constitutes a new genesis and requires recomputing this value.

3.6 §3 — Transition Function

$T : (S_t, I_t) \rightarrow S_{\{t+1\}} \mid \perp$

3.6.1 Input set

I_t is the canonical input at epoch t :

```
 $I_t = ($   
  transactions: [Tx; M],    //  $M \leq \text{max\_queries\_per\_epoch}$  from genesis  
  validator_sigs: [Sig; N], // one Dilithium5 signature per active validator  
  epoch_anchor: [u8; 32], // SLH-DSA-SHA3-256 anchor over  $I_t \setminus \{\text{epoch\_anchor}\}$   
 $)$ 
```

I_t is admissible if and only if: - all signatures verify under their respective validator public keys, - epoch_anchor verifies under the epoch anchor key, - $M \leq \text{max_queries_per_epoch}$, - **I_t .transactions is canonically ordered before admission into Domain A.**

The canonical ordering is defined in 02_transition_axioms.md §A10: transactions are sorted by $H_{\text{domain}}(\text{ENTROPY_ADVANCE}, S_t.\text{entropy_seed} \sqcup \text{Encode}(\tau))$. An I_t whose transactions are not in this order is inadmissible and triggers absorbing halt. This closes the determinism leak where two validators could admit semantically equivalent but differently-ordered transaction batches.

3.6.2 Transition steps

Given admissible (S_t, I_t) :

1. If $S_t.\text{halt_reason} \neq \text{None}$ (0x00): return $\perp$
2. Apply transactions:
 $S' \leftarrow \text{ApplyAll}(S_t, I_t.\text{transactions})$
 If any transaction triggers overflow_policy: return $\perp$
3. Advance entropy:
 $S'.\text{entropy_seed} \leftarrow \text{SHA3-256}(S_t.\text{entropy_seed})$
4. Compute convergence potential:
 $v \leftarrow V_{\text{convergence}}(S')$ // defined in §4a
5. Compute safety accumulator:
 $\phi \leftarrow \Phi_{\text{safety}}(S')$ // defined in §4b
6. Check stability criterion (§5) BEFORE updating the window:
 // Window contains preceding W values; current v is not yet in it
 If $\delta_{\text{window}}(v, S_t.\text{lyapunov_window}) > \varepsilon$ OR $\phi \geq \Phi_{\text{max_safe}}$:
 $S'.\text{halt_reason} \leftarrow \text{LyapunovViolation}$ (0x01)
 return S' with halt_reason set (absorbing)

7. Update Lyapunov window with current value:
`S'.lyapunov_window ← rotate_left(S_t.lyapunov_window, v)`
`// Now window stores [S_{t-W+1}, ..., S_t] for next epoch's check`
8. Increment epoch:
`S'.epoch ← S_t.epoch + 1`
9. Compute new state root using prior-root substitution (LAST step):
`S'.state_root ← H_consensus_domain(STATE_ROOT=0x00000001, Encode_for_commitment(S', pr`
`// prior_root(t+1) = S_t.state_root by definition (§2)`
10. return S'

All steps are pure functions over fixed-size, statically-bounded data structures. No unbounded or nondeterministically-sized allocation is permitted. See 00_execution_model.md §E3 for the full allocation policy.

3.7 §3.11 — Causal Fingerprint

The **Causal Fingerprint** is a rolling hash that accumulates every epoch transition into a single 32-byte value, guaranteeing that two sequences with the same final fingerprint had identical transition histories. It is the mechanism that makes the Coq bisimulation proofs (TH-5, RT-1 through RT-4) executable and cross-ISA verifiable.

3.7.1 Definition

```
F_0 = genesis_seed           // fixed in GENESIS_CONSTANTS.toml
F_t = H_domain(CAUSAL_FP, F_{t-1} ⊕ epoch_le ⊕ sort_root)
```

where: - `H_domain` is the standard domain-separated SHA3-256 used throughout Domain A - `CAUSAL_FP = 0x07` is the domain tag for causal fingerprint operations - `epoch_le` is the 8-byte little-endian encoding of the epoch counter `t` - `sort_root` is the SHA3-256 over the canonically-ordered transaction set at epoch `t`

3.7.2 Properties

Property	Proof
Determinism: equal inputs $\rightarrow$ equal fingerprint	<code>fingerprint_deterministic</code> in <code>safety/causal_fingerprint.v</code>
Step injectivity: $H(F,e,r) = H(F',e',r') \rightarrow F=F', e=e', r=r'$	<code>fp_step_injective</code> (from hash collision axiom)
Chain injectivity: equal-length chains with equal terminal F had equal histories	<code>fingerprint_chain_injective</code>
Bisimulation collapse prevention: fp-bisimilar sequences are identical	<code>bisim_collapse_prevented</code>

3.7.3 Implementation

crates/consensus/src/transition.rs computes F_t at step 8 of the transition function (after entropy advance, before state root). The fingerprint is committed into the state root preimage, ensuring that any two states with the same root also have the same causal history.

3.8 §4 — Stability Functions

The protocol uses **two structurally distinct functions** with separate proof obligations. They must not be combined. Combining them was an identified design flaw: a monotone non-decreasing term inside a convergence function invalidates all convergence proofs.

3.8.1 §4a — Operational Lyapunov Candidate $V_{\text{convergence}}$

Scope: Dynamical convergence analysis only. **Proof obligation:** $\Delta V_{\text{convergence}} \leq \varepsilon$ under all admissible honest transitions. **Proof target:** proofs/contractivity/lyapunov_stability.v

$V_{\text{convergence}}$ is a candidate Lyapunov function over the **dynamic** validator state. It contains only terms that can both increase and decrease, enabling convergence analysis.

$$V_{\text{convergence}}(S_t) = \sum_i [\alpha \cdot D_{i,t} + \beta \cdot C_{i,t}] + \chi \cdot CH_t$$

where the first sum is over all active validators and CH_t is the epoch-level cascade health factor (see §4c).

Symbol	Genesis constant	Value	Semantic meaning
α	weight_divergence_D	350_000	Weight on per-validator state divergence
β	weight_conflict_C	300_000	Weight on per-validator conflict density
χ	weight_cascade_health_CH	150_000	Weight on cascade verification health

Component definitions:

$D_{i,t}$ — **Divergence:** normalized Hamming distance between validator i 's committed state root and the epoch consensus root.

Hamming distance over 32-byte (256-bit) hash outputs is semantically meaningful: it counts bitwise disagreement. It is deterministic, architecture-independent, bounded in $[0, 256]$, and preserves cryptographic opacity while measuring disagreement density. Arithmetic distance on hash prefixes has none of these properties and must not be used.

```

XOR_root_i,t = V_i.committed_root XOR S_t.state_root    (bitwise, 32 bytes)
hamming_i,t  = popcount(XOR_root_i,t)                  (count of differing bits,  $\in [0, 256]$ )
D_i,t        = hamming_i,t  $\times$  p / 256                (normalized to  $[0, p]$ )

```

popcount is the number of set bits across all 32 bytes of the XOR result. It is computed as the sum of per-byte bit counts using a constant-time lookup table (256-entry, precomputed at genesis, part of the deterministic domain).

If validator i has not submitted a root for the current epoch: $D_{i,t} = p$ (maximum divergence).

Determinism note: popcount on fixed-width byte arrays is bitwise-identical across all authorized ISAs. Hardware POPCNT instructions must only be used if their output has been verified to match the lookup-table reference implementation in the cross-ISA test suite. Until verified, the lookup table is the authoritative implementation.

$C_{i,t}$ — **Conflict density:** normalized count of conflicting transitions submitted by validator i within the current evaluation window.

```

C_i,t = conflict_count_i  $\times$  p / max_queries_per_epoch
result  $\in [0, p]$ 

```

Arithmetic contract for $V_{convergence}$:

```

V_convergence(S_t):
  acc: i128  $\leftarrow$  0
  for each active V_i:
    term: i128  $\leftarrow$  ( $\alpha$  as i128)  $\times$  ( $D_{i,t}$  as i128)
                      + ( $\beta$  as i128)  $\times$  ( $C_{i,t}$  as i128)
    if term > i128::MAX - acc: trigger absorbing_halt()
    acc  $\leftarrow$  acc + term
  ch_term: i128  $\leftarrow$  ( $\chi$  as i128)  $\times$  ( $CH_t$  as i128)
  if ch_term > i128::MAX - acc: trigger absorbing_halt()
  acc  $\leftarrow$  acc + ch_term
  return floor_div(acc, p as i128) // floor_div defined in 00_execution_model.md §E1

```

$V_{convergence}(S_t) \in [0, N_{max} \times (\alpha + \beta) \times p / p + \chi] = [0, N_{max} \times (\alpha + \beta) + \chi]$ for all admissible states. This bound is structurally enforced by type widths.

3.8.2 §4c — Cascade Health Factor CH_t

Scope: Epoch-level cascade verification health. Input to $V_{convergence}$.

Domain: Domain A. CH_t is derived solely from cascade proof results admitted into the current epoch input set I_t — no wall-clock or entropy ingress.

CH_t is a normalized measure of cascade verification failures in the current epoch. It is bounded in $[0, p]$ and increases when cascade proofs fail verification.

```

cascade_fail_count_t  $\leftarrow$  count of cascade proof rejections in  $I_t$ 
CH_t = cascade_fail_count_t  $\times$  p / max_queries_per_epoch
CH_t  $\in [0, p]$ 

```

$CH_t = 0$ when all cascade proofs in the epoch verify correctly. $CH_t = p$ when every admitted input carries a failed cascade proof.

The cascade proof is the sparse-Merkle inclusion proof over the L7 output of H_{cascade} as defined in `00_execution_model.md §E4`. Proof format is specified in `docs/spec/07_hash_cascade.md`.

3.8.3 §4b — Safety Accumulator Φ_{safety}

Scope: Monotone safety witness. Not a convergence function. **Proof obligation:** Boundedness and halt-threshold correctness only. **Proof target:** `proofs/safety/absorbing_halt.v`

Φ_{safety} accumulates irreversible slash evidence. It is **monotone non-decreasing by construction**. It does not converge to an equilibrium — it approaches a structural upper bound that, when crossed, makes continuation inadmissible.

$$\Phi_{\text{safety}}(S_t) = \sum_i [\gamma \cdot \Sigma_{i,t}]$$

Symbol	Genesis constant	Value	Semantic meaning
γ	<code>weight_slash_Sigma</code>	200_000	Weight on per-validator slash accumulation

$\Sigma_{i,t}$ — **Slash accumulator:** monotone non-decreasing running sum of slash events for validator i . Never decreases. Saturates at `INT_MAX`.

`INT_MAX := 263 - 1` (pure arithmetic; maps to `i64::MAX` in Rust)

$\Sigma_{i,0} = 0$ (genesis)
 $\Sigma_{i,t+1} = \text{if } \Sigma_{i,t} + \text{slash_increment}_{i,t} > \text{INT_MAX}$
 then `INT_MAX` (explicit cap via pre-checked conditional, NOT saturation)
 else $\Sigma_{i,t} + \text{slash_increment}_{i,t}$

Note: this is not saturating arithmetic. It is a domain-bounded accumulator with an explicit pre-checked ceiling. The distinction matters for proofs: the cap is an admissibility decision, not an arithmetic behavior. The underlying addition is checked — if `slash_incrementi,t` is itself out of bounds, that triggers absorbing halt before the cap logic is reached.

Derived halt bound (not a genesis parameter — follows from the type space):

$$\begin{aligned} \Phi_{\text{max}} &:= N_{\text{max}} \times \gamma \times \text{INT_MAX} \\ &= 1024 \times 200_000 \times (2^{63} - 1) \\ &\approx 1.89 \times 10^{21} \end{aligned}$$

$$\Phi_{\text{max_safe}} := \Phi_{\text{max}} / 2 \quad (\text{halt triggers before representational exhaustion})$$

$\Phi_{\text{max_safe}} < \Phi_{\text{max}}$ by construction. This ensures the halt condition is an **admissibility violation**, not an overflow event. Overflow remains unreachable by invariant, not by arithmetic accident.

Arithmetic contract for Φ_{safety} :

```

 $\Phi_{\text{safety}}(S_t)$ :
  acc: i128  $\leftarrow$  0
  for each active  $V_i$ :
    term: i128  $\leftarrow$  ( $\gamma$  as i128)  $\times$  ( $\Sigma_{i,t}$  as i128)
    if term > i128::MAX - acc: trigger absorbing_halt()
    acc  $\leftarrow$  acc + term
  return acc    // NOT floor-divided; compared directly to  $\Phi_{\text{max\_safe}}$ 

```

Admissibility constraint on slash increments:

$\forall i, t: \text{slash_increment}_{i,t} \in [0, \text{INT_MAX}]$

If any $\text{slash_increment}_{i,t}$ exceeds INT_MAX , the transition producing it is **inadmissible** and triggers absorbing halt before the $\Sigma_{i,t}$ update is attempted. This ensures the i128 intermediate sum in the cap rule never overflows: current ($\leq \text{INT_MAX}$) + increment ($\leq \text{INT_MAX}$) $\leq 2 \times \text{INT_MAX} < \text{i128::MAX}$.

The source and derivation of $\text{slash_increment}_{i,t}$ is defined in the transaction semantics (docs/spec/03_transactions.md, pending). Until that document exists, slash_increment is treated as an opaque non-negative value bounded by INT_MAX .

Monotonicity invariant (proof obligation for absorbing_halt.v):

$\forall t, \text{admissible } S_t, \text{admissible } I_t$:
 $\Phi_{\text{safety}}(T(S_t, I_t)) \geq \Phi_{\text{safety}}(S_t)$

This follows directly from the explicit pre-checked cap definition of $\Sigma_{i,t}$: $\Sigma_{i,t+1} \geq \Sigma_{i,t}$ holds because the cap is INT_MAX and $\text{slash_increment} \geq 0$.

3.9 §5 — Stability Criterion

The protocol advances an epoch if and only if **both** conditions below are satisfied. Failure of either is an absorbing halt. The conditions are evaluated in order; if condition 1 fails, condition 2 is not evaluated.

3.9.1 Condition 1 — Convergence gate

Let $W = \text{evaluation_window} = 3$.

Let $\text{preceding_window} = [V_{\text{convergence}}(S_{\{t-W\}}), \dots, V_{\text{convergence}}(S_{\{t-1\}})]$ — the W values **preceding** the current epoch, **excluding** S_t itself.

The **window excursion** (δ_{window}) measures how far the current potential has risen above the minimum observed in the preceding evaluation window. This is **not** a temporal derivative — it does not measure $V_{\{t+1\}} - V_t$. It measures rolling-minimum deviation:

$\delta_window = V_convergence(S_t) - \min(\text{preceding_window})$

Note: the window deliberately excludes the current epoch's value. Including it would create a self-minimizing edge case where a spike immediately becomes the new minimum, masking instability within the same window. Using only preceding values means a transient spike that recovers within W epochs will not trigger halt, but a sustained rise will.

During genesis initialization ($t < W$): preceding_window is padded with 0 for all epochs before genesis. This ensures $\delta_window \geq 0$ from epoch 0.

CONDITION 1 PASS iff $\delta_window \leq \varepsilon$ where $\varepsilon = \text{epsilon_threshold} = 20_000$
 CONDITION 1 FAIL iff $\delta_window > \varepsilon \rightarrow \text{absorbing halt}$

Proof obligation: `proofs/contractivity/lyapunov_stability.v` must show $\delta_window \leq 0$ under all admissible honest transitions (stronger than $\leq \varepsilon$). ε provides tolerance for bounded perturbations only.

3.9.2 Condition 2 — Safety admissibility gate

CONDITION 2 PASS iff $\Phi_safety(S_t) < \Phi_max_safe$
 CONDITION 2 FAIL iff $\Phi_safety(S_t) \geq \Phi_max_safe \rightarrow \text{absorbing halt}$

where $\Phi_max_safe = \Phi_max / 2 = 1024 \times 200_000 \times (2^{63} - 1) / 2$.

This condition triggers before representational exhaustion by construction. Overflow is unreachable as a protocol invariant, not merely by runtime check.

Proof obligation: `proofs/safety/absorbing_halt.v` must show: - Φ_safety is monotone non-decreasing under all transitions - $\Phi_safety(S_0) = 0$ at genesis - Φ_max_safe is unreachable in finite epochs under bounded slash increments

3.9.3 Combined criterion

STABLE iff $\delta_window \leq \varepsilon$ AND $\Phi_safety(S_t) < \Phi_max_safe$
 HALT iff $\delta_window > \varepsilon$ OR $\Phi_safety(S_t) \geq \Phi_max_safe$

3.9.4 Epoch gating invariant

No state $S_{\{t+1\}}$ with $S_t.\text{halt_reason} \neq \text{None}$ is ever produced by an honest validator. Any such transition must be rejected deterministically by all other validators.

3.10 §6 — Replay Invariance Theorem Statement

This section states the theorem that all CI, test vectors, and cross-ISA verification are designed to prove.

3.10.1 Theorem RT-1 (Replay Invariance)

Let G be the genesis constants block. Let $T = (I_0, I_1, \dots, I_n)$ be a canonical input sequence, where each I_t is admissible. Let p_x and p_y be any two platforms from the Tier A authorized ISA set:

`{ x86_64-avx2, aarch64-neon, riscv64-vector }`

as defined in `docs/spec/00_execution_model.md §E6`. The theorem does not claim bitwise identity across arbitrary hardware; it is bounded to this set only.

Then:

$\text{Replay}_{\{p_x\}}(G, T) = \text{Replay}_{\{p_y\}}(G, T) = R_n$

where $R_n = S_n.\text{state_root}$ and the canonical commitment invariant (§2) holds: $R_n = H_{\text{consensus_domain}}(\text{STATE_ROOT}, \text{Encode_for_commitment}(S_n, \text{prior_root}(n)))$

That is: deterministic re-execution of the canonical input sequence from genesis produces a bitwise-identical state root on all authorized platforms.

3.10.2 Proof obligations

RT-1 is not proven here. It is the **proof target** for:

- The cross-ISA CI workflow (`platform-determinism.yml`)
- The deterministic test vector suite (to be defined in `docs/spec/02_test_vectors.md`)
- The Coq contractivity proof (`proofs/contractivity/lyapunov_stability.v`)
- The TLA+ safety invariant (`proofs/safety/`)

RT-1 depends on: 1. §0 constraints being enforced (no float, no nondeterminism) 2. §2 canonical encoding being unique (no two distinct S_t produce the same encoding) 3. §3 transition function being total and pure 4. §4 Lyapunov computation using only i128 intermediate arithmetic with floor rounding 5. §5 stability criterion applying identically across platforms

3.10.3 Corollary RT-2 (Succession Soundness)

If the network halts at epoch n ($\text{halt_reason} \neq \text{None}$), and a successor network G' anchors to R_n as its genesis state root, then:

$S'_0.\text{state_root} = R_n$

is the unique valid starting condition for G' . No other genesis root is accepted by an honest validator of G' .

3.11 §7 — Theorem Dependency Graph

All protocol guarantees reduce to the following proof graph. Each node is a theorem or axiom. Edges are dependency arrows ($A \rightarrow B$ means B depends on A). No implementation claim is valid until its proof obligations are discharged.

3.11.1 Axiom layer (class: ASSUMED — trusted, not proved within the system)

AX-1 ISA correctness: [ASSUMED] authorized ISAs implement two's complement correctly
AX-2 Compiler: [ASSUMED] pinned Rust toolchain produces correct code
AX-3 Hash security: [ASSUMED] the active consensus hash suite (SHA3-256 + SM3-256, folded by SHA3-256) is modeled as collision-resistant over protocol state space
IMPORTANT: no fixed-

width hash root is mathematically injective

(collisions exist by pigeonhole). This axiom assumes collisions are computationally unreachable within the protocol's admissible state space. It is a computational assumption modeled as a mathematical axiom. Named AX3_sha3_assumed_injective in the Coq proof files to make the trust class explicit.

3.11.2 Theorem and verification claim layer

Two classes exist. They are epistemically distinct and must not be conflated:

- **FORMAL THEOREM** — machine-proved from axioms; deductive certainty given AX-1/AX-2/AX-3
- **VERIFICATION CLAIM** — validated by CI/test vectors; empirical evidence, not proof

TH-1 Encoding injectivity

$\text{Encode}(x) = \text{Encode}(y) \Rightarrow x = y$

Depends on: AX-1, AX-2

Class: FORMAL THEOREM

Proof file: proofs/contractivity/encode_injectivity.v

Status: FORMAL — Coq compiles; zero Admitted beyond AX-1/AX-2

TH-2 Encoding totality

Encode is defined for all admissible S_t

Depends on: AX-1, AX-2

Class: FORMAL THEOREM

Proof file: proofs/contractivity/encode_injectivity.v (co-located)

Status: PROVED

TH-3 Convergence non-increase (revised — see note)

$\forall$ admissible honest (S_t, I_t) : $\delta_{\text{window}}(T(S_t, I_t)) \leq \varepsilon_{\text{honest}}$

where $\varepsilon_{\text{honest}} = 2_{000}$ (see two-threshold model below)

Depends on: TH-1, AX-1, AX-2, §A8 proof obligations for all admitted τ

Class: FORMAL THEOREM

Proof file: proofs/contractivity/lyapunov_stability.v

Status: FORMAL — proofs/contractivity/lyapunov_stability.v; zero Admitted beyond AX-1/AX-2

NOTE on target strength: the original target $\delta_{\text{window}} \leq 0$ (strict

non-increase per epoch) is too strong for any nontrivial transaction system – the protocol explicitly permits bounded perturbation via §A8 Form B. The revised target $\delta_{\text{window}} \leq \epsilon_{\text{honest}}$ provides:

- a mathematically achievable convergence guarantee
- a safety margin: $\epsilon_{\text{honest}} (2_000) \ll \epsilon_{\text{halt}} (20_000)$
- compatibility with §A8 Form B perturbation budgets

Two-threshold model:

$\epsilon_{\text{honest}} = 2_000$ – proof target; honest transactions must stay within
 $\epsilon_{\text{halt}} = 20_000$ – halt trigger; $\epsilon_{\text{honest}} \ll \epsilon_{\text{halt}}$ gives safety margin
 ratio = $10\times$ – ten epochs of full perturbation before halt

TH-3 proof strategy (via §A8 composition):

For each admitted τ_i : $\delta_{\text{window}}(\tau_i) \leq \delta_{\text{window}} + \epsilon_{\tau_i}$ (§A8 obligation)
 $\sum \epsilon_{\tau_i} \leq \epsilon_{\text{honest}}$ per epoch (epoch budget)
 $\therefore \delta_{\text{window}}(\text{ApplyAll}(S_t, I_t)) \leq \delta_{\text{window}}(S_t) + \epsilon_{\text{honest}}$

TH-3a Halt determinism (corollary of RT-1)

If $\text{Replay}(G, T)$ derives $\text{halt_reason} \neq \text{None}$ at epoch n on one honest validator, then every honest validator replaying the same admissible T must derive the same halt_reason at epoch n .

Depends on: TH-7 (RT-1 replay invariance), TH-6

Class: FORMAL THEOREM (follows from RT-1 applied to halt_reason field)

Proof: halt_reason is part of $\text{Encode}(S_n)$; RT-1 guarantees identical $\text{Encode}(S_n)$ on all honest Ξ ; therefore identical halt_reason .

Status: STATED (proof follows from RT-1 composition)

TH-4 Φ_{safety} monotonicity

$\forall$ admissible (S_t, I_t) : $\Phi_{\text{safety}}(T(S_t, I_t)) \geq \Phi_{\text{safety}}(S_t)$

Depends on: AX-1, AX-2

Class: FORMAL THEOREM

Proof file: `proofs/safety/absorbing_halt.v`

Status: FORMAL – `proofs/safety/absorbing_halt.v`; zero Admitted beyond AX-1/AX-2

TH-5 Φ_{safety} boundedness

$\forall$ admissible S_t : $\Phi_{\text{safety}}(S_t) \leq \Phi_{\text{max}}$

Depends on: TH-4, AX-1

Class: FORMAL THEOREM

Proof file: `proofs/safety/absorbing_halt.v`

Status: FORMAL – `proofs/safety/absorbing_halt.v`; zero Admitted beyond AX-1/AX-2

TH-6 Halt correctness

$\text{halt_reason} \neq \text{None} \Rightarrow$ no further admissible transitions exist

Depends on: TH-4, TH-5

Class: FORMAL THEOREM

Proof file: `proofs/safety/absorbing_halt.v`

Status: FORMAL – proofs/safety/absorbing_halt.v; zero Admitted beyond AX-1/AX-2

TH-7 Replay invariance (RT-1)

$\forall \text{ ISA} \in \{\text{x86_64}, \text{aarch64}, \text{riscv64}\}:$

$\text{Replay_ISA}(G, T) = R_n$

Depends on: TH-1, TH-2, AX-1, AX-2

Class: VERIFICATION CLAIM (empirical evidence, not deductive proof)

platform-determinism.yml provides evidence; full proof requires TH-1 discharge

Verification: platform-determinism.yml + test vectors (docs/spec/07_test_vectors.md)

Status: PARTIAL – CI-verified on x86_64; aarch64 and riscv64gc cross-ISA runs pending

TH-8 Succession soundness (RT-2)

$S'_0.\text{state_root} = R_n$ is the unique valid genesis for successor G'

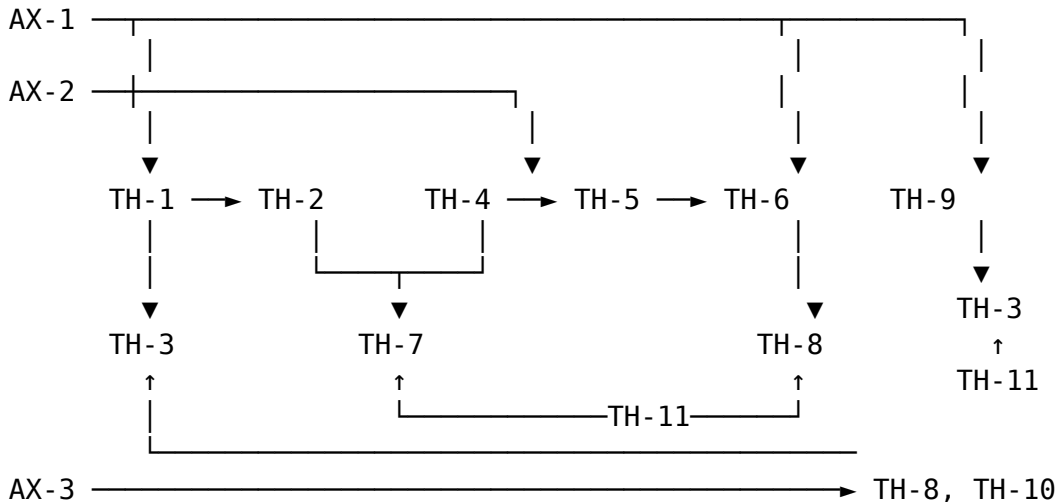
Depends on: TH-1, TH-6, AX-3

Class: FORMAL THEOREM

Proof file: proofs/safety/absorbing_halt.v

Status: FORMAL – proofs/integration/th8_composition.v; zero Admitted beyond AX-1/AX-2/AX-3

3.11.3 Dependency graph (ASCII)



3.11.4 Genesis lock gate

GENESIS_CONSTANTS.toml must not be locked until: - TH-1, TH-2, TH-3, TH-4, TH-5, TH-6, TH-8: FORMAL (Coq compiles; zero Admitted beyond AX-1/AX-2/AX-3) - TH-7: CI-verified on x86_64; aarch64 and riscv64gc cross-ISA runs must pass before final lock - genesis_hash in GENESIS_CONSTANTS.toml must be set to the SHA3-256 of the canonical spec document set (see §2 genesis hash procedure)

3.12 Appendix A — Binding of Genesis Constants

The following table maps every consensus-relevant field in `GENESIS_CONSTANTS.toml` to its formal definition in this specification. Fields marked **derived** are not genesis parameters — their values follow necessarily from the listed sources.

TOML field / constant	§	Formal role	Proof dependency
<code>fixed_point.scale</code>	§0, §4a	$p = 1_000_000$; denominator of $\square_p$	TH-1
<code>fixed_point.intermediate_width</code>	E1	Mandates i_{128} for all intermediates	TH-1, TH-3
<code>fixed_point.rounding_mode</code>	E1	<code>floor_div</code> toward $-\infty$	TH-3
<code>fixed_point.overflow_policy</code>	§0, §3	Absorbing halt on i_{128} overflow	TH-6
<code>lyapunov.weight_divergence_D</code>	§4a	$\alpha = 350_000$	TH-3
<code>lyapunov.weight_conflict_C</code>	§4a	$\beta = 300_000$	TH-3
<code>lyapunov.weight_slash_Sigma</code>	§4b	$\gamma = 200_000$	TH-4, TH-5
<code>lyapunov.weight_cascade_health</code>	§4a, §4b, §4c	$\chi = 150_000$	TH-3, TH-9
<code>lyapunov.epsilon_threshold</code>	§5	$\varepsilon = 20_000$	TH-3
<code>lyapunov.evaluation_window</code>	§5	$W = 3$	TH-3
<code>lyapunov.max_queries_per_epoch</code>	§3, §4a	Bounds M ; normalizes $C_{i,t}$	TH-3
<code>epoch.timing.duration_ms</code>	E6	Wall-clock budget (nondeterministic domain, PAL only)	none
<code>obfuscation.leaf_index_bytes</code>	§1	$V_{i.id}$ width = 48 bytes	TH-1
<code>crypto.cascade.primary_signatures</code>	§2e	Dilithium5 per-validator signatures	AX-2
<code>crypto.cascade.anchor_signatures</code>	§2	SLH-DSA-SHA3- 256 epoch anchor	AX-2
<code>N_max = 1024</code>	§1	Protocol law (not TOML); max validator cardinality	TH-1, TH-5
<code>INT_MAX = 2^63-1</code>	§4b	Derived from i_{64} width; not a TOML field	TH-5
<code>$\Phi_{\max}$</code>	§4b	Derived: $N_{\max} \times$ $\gamma \times INT_MAX$	TH-5
<code>$\Phi_{\max_safe}$</code>	§5	Derived: $\Phi_{\max} /$ 2	TH-6

Any genesis field not listed here has no direct bearing on T , $V_{\text{convergence}}$, Φ_{safety} , or Encode and is an operational parameter only.

3.13 Appendix B — Relationship to 00_execution_model.md

This document defines **what** the protocol computes. 00_execution_model.md defines **how** it is permitted to compute it.

Both documents are protocol law. In case of conflict, 00_execution_model.md governs execution constraints; this document governs semantic correctness.

End of *docs/spec/01_consensus.md* SHA3-256 of this document to be recorded in *GENESIS_CONSTANTS.toml* at lock time.

4 QASH Transition Axioms

4.1 Purpose and Position in the Spec Stack

00_execution_model.md — computational substrate law
 01_consensus.md — state space, encoding, stability functions
 02_transition_axioms.md ← you are here: admissible transformation law
 03_transactions.md — concrete transaction types (instantiations of axioms)

This document defines what **all possible transitions** are permitted to do, stated as mathematical axioms, before any concrete operation exists. 03_transactions.md then defines specific transaction classes as concrete instantiations of these axioms — each with a proof obligation showing conformance.

A transition is not admissible because it has been implemented. A transition is admissible because it satisfies every axiom in this document.

4.2 Notation

Symbol	Meaning
τ	A single admissible transition (atomic state transformation)
$\tau(S_t)$	State resulting from applying τ to S_t
$I_t = \{\tau_1, \dots, \tau_m\}$	The ordered set of transitions in epoch t
$\text{ApplyAll}(S_t, I_t)$	Sequential application of all τ_i in canonical order
$S_t \vdash \tau$	τ is applicable to S_t (admissibility relation)

Symbol	Meaning
$\delta_window(S_t)$	Rolling-window excursion of $V_convergence$ at S_t (from §5 of 01_consensus.md)
ϵ_t	Bounded perturbation budget for transition class t
$\Phi_safety(S_t)$	Monotone safety accumulator (from §4b of 01_consensus.md)
$Encode(S_t)$	Canonical wire encoding (from §2 of 01_consensus.md)
$\perp$	Absorbing halt (irreversible)
p	Fixed-point scale = 1_000_000

4.3 §A0 — Transition Admissibility

A transition t is **admissible** with respect to state S_t iff all of the following hold. Violation of any single axiom makes t inadmissible; an inadmissible transition applied to any state must trigger absorbing halt.

$S_t \vdash t$ iff:

- A1. t is deterministic over S_t (§A1)
- A2. t terminates within the epoch execution budget (§A2)
- A3. t mutates only explicitly declared state fields (§A3)
- A4. t preserves canonical encodability of $S_{\{t+1\}}$ (§A4)
- A5. t preserves replay invariance across Tier A ISAs (§A5)
- A6. t does not clear `halt_flag` (§A6)
- A7. t does not introduce Domain B artifacts into state (§A7)
- A8. δ_window effect of t is bounded (see §A8) (§A8)

The admissibility relation is **closed**: a transition that is not proved admissible is inadmissible by default. There is no implicit permission.

4.4 §A1 — Determinism Axiom

$\forall t, S_t$: if $S_t \vdash t$ then $t(S_t)$ is uniquely determined.

A transition may not depend on:

FORBIDDEN sources of nondeterminism in t :

- Memory layout or allocation address
- Wall-clock time or hardware counters
- OS scheduler or thread interleaving
- Allocator order or GC behavior
- ISA-specific undefined behavior

- Nondeterministic iteration order (HashMaps, etc.)
- Any Domain B value not passed through the canonical input I_t

A transition is deterministic iff for all admissible S_t , applying τ to S_t on any two authorized platforms produces bitwise-identical S_{t+1} .

This axiom is the protocol instantiation of the Domain A definition in `00_execution_model.md` §E0. It is restated here as a transition-level law.

Proof obligation: Every transaction type in `03_transactions.md` must include a determinism argument showing no forbidden sources are reachable.

4.5 §A2 — Termination Axiom

$\forall \tau, S_t$: if $S_t \vdash \tau$ then $\tau(S_t)$ terminates within the epoch budget.

Every transition must terminate within the epoch execution budget:

`max_control_loop_latency_ms = 450ms` (from `GENESIS_CONSTANTS.toml`)

More precisely, every transition must be **statically bounded**: the number of computation steps is bounded by a function of the genesis constants alone, independent of runtime state.

REQUIRED: static step bound B_τ such that
 $\text{steps}(\tau, S_t) \leq B_\tau(N_{\text{max}}, \text{max_queries_per_epoch})$
 for all admissible S_t

No unbounded recursion. No loops without a static iteration budget. Divergence is treated as an overflow condition and triggers absorbing halt.

Proof obligation: Every transaction type must supply B_τ and a proof that execution stays within it for all admissible inputs.

4.6 §A3 — State Locality Axiom

$\forall \tau, S_t$: $\tau(S_t)$ differs from S_t only in explicitly declared fields.

Each transaction type in `03_transactions.md` must declare a **mutation footprint** — the exact set of state fields it may modify:

```
mutation_footprint( $\tau$ )  $\subseteq$  {
  validators[i].score,
  validators[i].divergence,
  validators[i].conflict,
  validators[i].slash_acc,
  validators[i].active,
  ledger_root,
  entropy_seed (only via the canonical advance rule in §3 of 01_consensus.md)
}
```

Permanently immutable fields — no transition may ever modify:

IMMUTABLE under any τ :

epoch (advanced only by the epoch transition rule, not by τ)
state_root (computed by the transition function, not by τ directly)
halt_flag (may be set to true, never cleared — see §A6)
validator[i].id (identity is fixed at genesis for each validator slot)

Proof obligation: Every transaction type must declare its footprint. Any write outside the declared footprint makes τ inadmissible.

4.7 §A4 — Encoding Preservation Axiom

$\forall \tau, S_t$: if $S_t \vdash \tau$ then $\text{Encode}(\tau(S_t))$ is well-defined and canonical.

This axiom follows from TH-1 (encoding injectivity) but is stated explicitly as a transition-level constraint: a transition that would produce a state S_{t+1} for which $\text{Encode}(S_{t+1})$ is undefined or non-canonical is inadmissible.

Concretely, every field of $\tau(S_t)$ must satisfy the well-formedness conditions of StateWF (defined in proofs/contractivity/encode_injectivity.v):

StateWF($\tau(S_t)$) must hold for all admissible (τ, S_t)

If any field after application violates its type bound (e.g. a score field exceeds INT_MAX), the transition triggers absorbing halt before committing.

Proof obligation: Every transaction type must include a StateWF preservation lemma.

4.8 §A5 — Replay Preservation Axiom

$\forall \tau, S_t, \text{ISA}_1, \text{ISA}_2 \in \text{Tier A}$: $\tau(S_t)$ on $\text{ISA}_1 = \tau(S_t)$ on ISA_2 .

This is the transition-level instantiation of TH-7 (replay invariance). Every transition must be replay-invariant across the Tier A ISA set {x86_64-avx2, aarch64-neon, riscv64-vector}.

This axiom is operationally enforced by: - The platform-determinism.yml CI workflow - The test vector suite (docs/spec/02_test_vectors.md, pending) - The cross-ISA replay check on every PR

A transition that passes single-ISA tests but fails cross-ISA replay is inadmissible. CI is the enforcement mechanism; the axiom is the specification.

4.9 §A6 — Halt Monotonicity Axiom

$\forall \tau, S_t$: $\text{halt_flag}(\tau(S_t)) \geq \text{halt_flag}(S_t)$.

In boolean terms: `halt_flag` may be set to `true` but never cleared.

```
if halt_flag(S_t) = false:   $\tau(S_t).halt\_flag \in \{false, true\}$   
if halt_flag(S_t) = true:    $\tau(S_t) = \perp$  (absorbing halt, no transition)
```

This axiom formalizes TH-6 at the transition level. A halted state is a terminal state. No transaction class may include a `clear_halt` operation. No governance mechanism can override this.

Proof obligation: Trivially discharged for any τ that does not touch `halt_flag`. For τ that may set `halt_flag = true`, no additional obligation — only that it does not set `halt_flag = false`.

4.10 §A7 — Domain Separation Axiom

$\forall \tau$: no Domain B artifact influences $\tau(S_t)$.

The precise statement from `00_execution_model.md` §E0 applied to transitions:

FORBIDDEN in any τ :

- Wall-clock time as input
- OS entropy as input
- Network state as input
- Hardware attestation results as state-mutating inputs
- Any value whose provenance is Domain B

All inputs to τ must arrive via the canonical input set I_t , which is itself admissibility-checked before `ApplyAll` is called. The domain boundary is enforced structurally: I_t contains only canonically signed, deterministically ordered transactions. Any out-of-band input is a boundary violation.

4.11 §A8 — δ_{window} Compatibility Axiom

This is the most critical axiom. It is the constitutional law governing what transitions are allowed to exist.

Every admissible transition class τ must provide one of:

4.11.1 Form A — Non-increase proof (strong form)

$\forall$ admissible S_t : $\delta_{\text{window}}(\tau(S_t)) \leq \delta_{\text{window}}(S_t)$

This is the ideal form. It means τ never increases the rolling-window excursion of the convergence potential. Epoch advancement is preserved.

4.11.2 Form B — Bounded perturbation proof (acceptable form)

$\forall$ admissible S_t : $\delta_{\text{window}}(\tau(S_t)) \leq \delta_{\text{window}}(S_t) + \epsilon_{\tau}$

where ε_τ is a **transition-class-specific perturbation budget** satisfying:

$$\varepsilon_\tau \in [0, \varepsilon_{\text{honest}} / 2] \quad \text{where } \varepsilon_{\text{honest}} = 2_000 \quad (\text{proof target})$$

and for any epoch $I_t = \{\tau_1, \dots, \tau_m\}$:

$$\sum_i \varepsilon_{\tau_i} \leq \varepsilon_{\text{honest}}$$

Two distinct thresholds (defined in 01_consensus.md §5):

Constant	Value	Role
$\varepsilon_{\text{honest}}$	2_000	Proof target. Per-epoch budget cap for §A8 Form B. TX-k budgets must sum to $\leq \varepsilon_{\text{honest}}$.
$\varepsilon_{\text{halt}}$	20_000	Halt trigger. $\delta_{\text{window}} > \varepsilon_{\text{halt}}$ triggers absorbing halt.
ratio	10×	Safety margin: ten epochs of full-budget perturbation before halt.

The epoch-level budget constraint ($\sum_i \varepsilon_{\tau_i} \leq \varepsilon_{\text{honest}}$) ensures that even if every transaction in an epoch contributes its maximum perturbation, the total stays within the proof target $\varepsilon_{\text{honest}}$. The 10× margin to $\varepsilon_{\text{halt}}$ absorbs unforeseen drift, accumulated round-off, or proof-discovered slack.

This prevents budget exhaustion from transaction accumulation and keeps the halt trigger structurally unreachable in honest operation.

4.11.3 Form C — $\Phi_{\text{safety-only}}$ effect (special case)

τ affects only $\Sigma_{i,t}$ (the slash accumulator)

and does not affect $D_{i,t}$ or $C_{i,t}$

Since $V_{\text{convergence}}$ is defined over $D_{i,t}$ and $C_{i,t}$ only (not $\Sigma_{i,t}$), a transition that exclusively modifies $\Sigma_{i,t}$ trivially satisfies §A8 with $\varepsilon_\tau = 0$. It must still satisfy §A8's Φ_{safety} monotonicity:

$$\Phi_{\text{safety}}(\tau(S_t)) \geq \Phi_{\text{safety}}(S_t)$$

which follows from TH-4 for any τ that only increases $\Sigma_{i,t}$.

4.11.4 No other forms are admissible

A transaction type that cannot be proved to satisfy Form A, Form B, or Form C is inadmissible. Intuition, plausibility, or performance requirements are not sufficient grounds for admission.

Proof obligation: Every transaction type in 03_transactions.md must state and prove its §A8 form explicitly as a named lemma in proofs/contractivity/lyapunov_stability.v or a dedicated file.

4.11.5 Connection to TH-3

TH-3 ($\delta_window(T(S_t, I_t)) \leq \epsilon_honest$) is proved by composing the §A8 obligations of all admitted transaction types in I_t :

TH-3 proof structure:

$$\begin{aligned} \forall \tau_i \in I_t: \delta_window(\tau_i(S)) &\leq \delta_window(S) + \epsilon_{\tau_i} && \text{(by §A8 of each } \tau_i) \\ \sum \epsilon_{\tau_i} &\leq \epsilon_honest && \text{(epoch budget, §A8)} \\ \therefore \delta_window(\text{ApplyAll}(S_t, I_t)) &\leq \delta_window(S_t) + \epsilon_honest && \text{(composition)} \\ &\leq \epsilon_honest && \text{(TH-3 target)} \\ &< \epsilon_halt && \text{(10× margin; halt unreachable)} \end{aligned}$$

TH-3 is provable if and only if every admitted transaction class satisfies §A8. The axiom is the proof strategy. The 10× margin between ϵ_honest and ϵ_halt ensures that even adversarial-but-admissible behavior cannot trigger absorbing halt within a single epoch — halt requires accumulation across multiple epochs, which slash accounting (Φ_safety) attributes to specific validators.

4.12 §A9 — Halt Propagation Rule

$\forall \tau, S_t$: if $\tau(S_t)$ triggers any halt condition, $halt_flag$ is set before any other state mutation is committed.

Halt conditions from 00_execution_model.md §E5 (H1-H6) must be checked **eagerly** and **before** any partial state is written. A transition that partially mutates state and then halts leaves the system in an indeterminate state; this is inadmissible.

REQUIRED halt evaluation order for any τ :

1. Evaluate all arithmetic sub-expressions in checked mode
2. If any overflow detected: halt immediately, commit nothing
3. Evaluate §A8 δ_window check on proposed $\tau(S_t)$
4. If δ_window or Φ_safety threshold exceeded: halt, commit nothing
5. Commit state mutation atomically
6. Update $state_root$

This rule ensures that partial state mutations never become observable. The state machine is always in a complete, consistent state — or in $\perp$.

4.13 §A10 — Transition Composability

$\forall$ admissible τ_1, τ_2 : if $\tau_1 \circ \tau_2$ is applied in canonical order, the composition satisfies §A1-§A9.

ApplyAll(S_t , I_t) composes multiple transitions. For the composition to be admissible, the following must hold:

Composability conditions:

- C1. Canonical ordering: I_t is totally ordered (no ties, no ambiguity)
- C2. No mutation aliasing: if τ_i and τ_j share a mutation footprint field, their composition semantics are explicitly defined (last-write wins within an epoch, unless the field is subject to accumulation semantics)
- C3. Budget composition: $\sum_i \epsilon_{\tau_i} \leq \epsilon$ (epoch-level §A8 budget)
- C4. Encoding composability: ApplyAll(S_t , I_t) must satisfy StateWF

4.13.1 Canonical ordering law

The canonical ordering of I_t is defined as:

Order by: $H_{\text{domain}}(\text{ENTROPY_ADVANCE}, S_t.\text{entropy_seed} \parallel \text{Encode}(\tau))$

That is: for each transaction τ , compute a deterministic sort key by hashing the current entropy seed concatenated with the encoded transaction. Sort ascending. This produces a PRF-derived ordering that is:

- deterministic given S_t ,
- validator-bias-resistant (no validator controls the seed),
- replay-invariant (follows from entropy_seed determinism).

This ordering law is the sole admissible transaction ordering. Any implementation that uses a different ordering is non-conforming.

4.14 §A11 — Minimal Admissible Transaction Set (Initial)

Before the transaction algebra is expanded, the following are the only transaction types with active proof obligations:

TX-0 NoOp

mutation_footprint: $\emptyset$ (empty – no fields modified)
§A8 form: A (trivially $\delta_{\text{window}}(\tau(S_t)) = \delta_{\text{window}}(S_t)$)
proof: trivial by identity
status: ADMITTED

TX-1 BoundedValidatorScoreDecrement

mutation_footprint: {validators[i].score}
§A8 form: A (score decrease reduces $D_{i,t}$, reducing $V_{\text{convergence}}$)
proof obligation: proofs/contractivity/tx1_score_decrement.v
status: FORMAL

No other transaction types exist until their §A8 proof obligations are filed and discharged. This is not a temporary constraint — it is the admission policy.

New transaction types are admitted by: 1. Filing an §A8 proof obligation issue 2.

Writing the proof in proofs/contractivity/ 3. Adding the transaction to 03_transactions.md with proof reference

4.15 Appendix — Transaction Admission Template

Every entry in 03_transactions.md must include this block:

```
## TX-N: [Name]

**Mutation footprint:** [list of fields]
**$A8 form:** [A / B / C]
** $\epsilon_\tau$ :D_{i,t}:      yes/no – [explanation]
- affects  $C_{i,t}$ :      yes/no – [explanation]
- affects  $\Sigma_{i,t}$ :    yes/no – [explanation]
- monotone:          yes/no
- bounded:           yes/no
-  $\delta$ _window effect:  [analytical bound]

**$A1 determinism argument:** [why  $\tau$  has no nondeterministic sources]
**$A2 termination bound  $B_\tau$ :
```

End of docs/spec/02_transition_axioms.md

5 QASH Transaction Semantics

5.1 Purpose and Position in the Spec Stack

00_execution_model.md – computational substrate law
01_consensus.md – state space, encoding, stability functions
02_transition_axioms.md – admissible transformation law (axioms A0-A11)
03_transactions.md – you are here: concrete transaction instantiations

This document defines concrete transaction types as instantiations of the abstract transition algebra defined in 02_transition_axioms.md.

A transaction type exists in this protocol if and only if: 1. Its admissibility predicate satisfies \$A0-A7 of 02_transition_axioms.md 2. It declares one of Form A, B, or C

from §A8 and a proof obligation exists 3. It is listed in Appendix A (Transaction Type Registry) at genesis

The registry is **closed** at genesis. New transaction types require a new network.

5.2 Notation (aligned with 02_transition_axioms.md)

Symbol	Meaning
τ	A single canonical transaction (instance)
TX-k	Transaction type k (class)
Encode(τ)	Canonical encoding of τ (§1 below)
TxID(τ)	$H_domain(TX_ID, Encode(\tau))$ — stable replay identity
sort_key(τ, S_t)	Canonical ordering key for τ at state S_t (§4)
$\Box_\tau(S_t, \tau)$	Per-type admissibility predicate (§2)
$\Box_\tau(S_t, \tau)$	Per-type transition function (§3)
touch(τ)	Set of state fields τ may modify (§3)
ϵ_τ	Perturbation budget declared by τ (§A8 of 02)
σ_τ	Slash increment bound declared by τ (§6)
prior_root(t)	Defined in 01_consensus.md §2
Encode_for_commitment	Defined in 01_consensus.md §2
$H_domain(tag, \dots)$	Domain-separated hash (00_execution_model.md §E4)

All arithmetic obeys 00_execution_model.md §E1. All encoding obeys 01_consensus.md §2.

5.3 §0 — Transaction Model

5.3.1 Transaction universe

The set of admissible transaction types is finite and frozen at genesis:

$\Box = TX-0 \cup TX-1 \cup \dots$ (closed set; recorded in Appendix A)

Each TX-k is a **transition algebra**, not an application protocol. It defines: - A canonical payload schema (§1) - An admissibility predicate $\Box_\tau$ (§2) - A pure transition function $\Box_\tau$ (§3) - Declared bounds ϵ_τ and σ_τ - A §A8 form (A, B, or C) with a proof obligation file

5.3.2 Ontology invariant

No transaction may introduce new encoding rules, new hash domain tags, or new arithmetic semantics. All transactions inherit:

- Canonical encoding from 01_consensus.md §2
- Fixed-point arithmetic from 00_execution_model.md §E1
- Commitment semantics from 01_consensus.md §2
- Halt semantics from 00_execution_model.md §E5
- Domain separation from 00_execution_model.md §E0

5.3.3 Transaction lifecycle

1. Decoding: bytes $\rightarrow$ Tx envelope or DecodeResult::Invalid(reason)
 2. Admission: $\tau \in I_t$ iff $\square \tau(S_t, \tau) = \text{true}$
 3. Ordering: I_t .transactions sorted by $\text{sort_key}(\tau, S_t)$ ascending
 4. Application: $S' = \square \tau(S_t, \tau)$; state mutation atomic
 5. Replay: guaranteed by nonce advancement (per author validator)
-

5.4 §1 — Canonical Transaction Encoding

5.4.1 Envelope structure (frozen)

```
Tx = (  
  version:      u16,           // must equal genesis tx_version = 1  
  tx_type:      u16,           //  $\in$  frozen registry (Appendix A)  
  nonce:        u64,           // per-author monotone counter  
  author_id:    [u8; 48],      // stable consensus identity (V_i.id)  
  payload_len:  u32,           //  $\leq \text{max\_tx\_payload\_bytes}$  (genesis)  
  payload:      [u8; payload_len], // type-specific, defined per TX-k  
  signature:    Dilithium5Signature, // 2420 bytes, canonical encoding  
)
```

5.4.2 Encoding rules

- All integers: little-endian, explicit width
- No padding, no alignment gaps
- Only payload is variable-length; everything else is fixed-width
- payload is opaque to the envelope; semantics defined per tx_type
- signature covers: $\text{Encode}(\text{envelope_without_signature})$
- Total envelope size = $2 + 2 + 8 + 48 + 4 + \text{payload_len} + 2420 = 2484 + \text{payload_len}$ bytes

5.4.3 Envelope wire format

```
Encode( $\tau$ ):  
  encode_u16( $\tau$ .version)       $\rightarrow$  2 bytes  
  encode_u16( $\tau$ .tx_type)      $\rightarrow$  2 bytes
```

<code>encode_u64(τ.nonce)</code>	→ 8 bytes
<code>τ.author_id</code>	→ 48 bytes verbatim
<code>encode_u32(τ.payload_len)</code>	→ 4 bytes
<code>τ.payload</code>	→ payload_len bytes verbatim
<code>τ.signature</code>	→ 2420 bytes (Dilithium5 canonical) = 2484 + payload_len bytes total

Note: this is the **wire encoding for transactions**, not for state. The state encoding is defined in `01_consensus.md §2`. Both are canonical within their respective domains and never mixed.

5.4.4 TxID computation

$\text{TxID}(\tau) = \text{H_domain}(\text{TX_ID} = 0x00000010, \text{Encode}(\tau))$

$\text{TxID}(\tau)$ is the stable replay identity of τ . It is:

- Used for deduplication in I_t (§7)
- Used as secondary sort key in $\text{sort_key}(\tau, \text{S_t})$ (§4)
- Independent of the author's nonce (different nonces → different TxIDs because nonce is in the envelope)

5.4.5 Decode rejection rules

A byte sequence fails to decode (returns `DecodeResult::Invalid(reason)`) if any of the following holds. State is unchanged on decode failure.

```

0x10 malformed envelope (wrong total length given payload_len)
0x11 version ≠ 1 (genesis tx_version)
0x12 tx_type ∉ Appendix A registry
0x13 payload_len > max_tx_payload_bytes (genesis constant)
0x14 signature verification fails under author_id's public key
0x15 type-specific decode failure (defined per TX-k below)

```

Rejection reason codes are part of the canonical state. All authorized platforms must produce identical reason codes for identical inputs (see `00_execution_model.md §E0` oracle contract for signature verification).

5.5 §2 — Transaction Admissibility

5.5.1 General admissibility predicate

For any well-decoded transaction τ at state S_t :

```

 $\square_{\tau}(\text{S\_t}, \tau) :=$ 
   $\tau$  is well-decoded                                (envelope valid per §1)
   $\wedge \tau.\text{version} = 1$ 
   $\wedge \tau.\text{tx\_type} \in \text{frozen\_registry}$             (Appendix A)
   $\wedge \text{author}(\tau) \in \text{active\_validators}(\text{S\_t})$ 
   $\wedge \tau.\text{nonce} = \text{expected\_nonce}(\text{S\_t}, \tau.\text{author\_id})$  (replay protection)

```


$\wedge \text{payload_admissible}(\tau.\text{tx_type}, S_t, \tau.\text{payload})$ (per-type, defined below)
 $\wedge \epsilon_\tau \leq \text{remaining_epoch_budget}(S_t)$ (\$A8 budget; §5 below)

5.5.2 Admissibility class

If $\neg \tau(S_t, \tau) = \text{false}$, the transaction returns `AdmissibilityResult::Reject(reason)` with state unchanged and the author's nonce **not advanced**. This is distinct from $\perp$ (absorbing halt).

Reason code ranges (canonical, cross-platform identical):

0x20 – 0x2F envelope-level admissibility failures (version, type, signer)
 0x30 – 0x3F state-level admissibility failures (nonce, budget, active set)
 0x40 – 0x4F per-type payload admissibility failures (defined per TX-k)

5.5.3 Failure ontology recap

Three distinct rejection layers, each cross-platform deterministic:

Layer	Result	Trigger	State effect	Nonce
Decode	<code>DecodeResult::Malformed(0x1X)</code>	Malformed bytes	unchanged	not advanced
Admissibility	<code>AdmissibilityResult::Rejected(0x2X/0x3X/0x4X)</code>	Invalid transition	unchanged	not advanced
Invariant	$\perp$ (absorbing halt)	Checked-arithmetic overflow, touch-set violation, halt condition	halted	meaningless after halt

5.5.4 Nonce semantics

```

expected_nonce(S_t, author_id) :=
  let V = validator_by_id(S_t, author_id) in V.nonce
  
```

```

apply succeeds → V.nonce := V.nonce + 1    (author's nonce only)
apply rejects  → V.nonce unchanged
  
```

Nonce advancement binds to the **author** (signer), not any target validator a transaction may reference in its payload. This is the replay protection contract — see `01_consensus.md` §1 for the validator nonce field.

5.6 §3 — Transaction Application Semantics

5.6.1 Transition function contract

For each TX-k, the type-specific transition function $\square_\tau$ must satisfy:

$\square_\tau : (S_t, \tau) \rightarrow S_{\{t+1\}} \mid \perp$

Required properties (proof obligations, even if trivial):

1. Purity: no side effects outside the returned state
2. Totality: if $\square_\tau(S_t, \tau) = \text{true}$, then $\square_\tau$ returns $S_{\{t+1\}} \neq \perp$
3. Confinement: only fields in $\text{touch}(\tau)$ are modified
4. Determinism: identical $(S_t, \tau) \rightarrow \text{identical } S_{\{t+1\}}$ on all Ξ
5. Arithmetic: all ops obey `00_execution_model.md §E1`; overflow $\rightarrow \perp$

5.6.2 Touch-set declaration

Each TX-k MUST statically declare its $\text{touch}(\tau)$: the exact set of state fields it may modify. Any write outside $\text{touch}(\tau)$ is an invariant violation and triggers $\perp$ (absorbing halt), regardless of admissibility.

Fields **permanently immutable** by any transaction (`01_consensus.md §1`):

epoch (advanced by epoch transition, not by τ)
state_root (computed last in transition step 9)
halt_reason (may be set to non-zero, never cleared – §A6 of 02)
validator[i].id (stable consensus identity)

5.6.3 Effect declarations

Each TX-k must declare its effect on convergence/safety components:

$\Delta D_{i,t}(\tau) : [0, p]$ // change to validator i's divergence ($\square_p$ units)
 $\Delta C_{i,t}(\tau) : [0, p]$ // change to validator i's conflict ($\square_p$ units)
 $\Delta \Sigma_{i,t}(\tau) : [0, \text{INT_MAX}]$ // change to validator i's slash accumulator
 $\Delta \text{entropy}(\tau) : \text{bool}$ // does τ advance `entropy_seed`?
 $\Delta \text{ledger}(\tau) : \text{bool}$ // does τ modify `ledger_root`?

These declarations are used to derive ε_τ (§5) and verify §A8 conformance.

5.7 §4 — Deterministic Ordering

5.7.1 Sort key definition

For any transaction τ admitted at state S_t :

$\text{sort_key}(\tau, S_t) := ($
 primary: $H_{\text{domain}}(\text{ENTROPY_ADVANCE} = 0x00000002,$
 $S_t.\text{entropy_seed} \sqcup \text{Encode}(\tau)),$
 secondary: $\text{TxID}(\tau)$
)

5.7.2 Ordering relation

$\tau_1 < \tau_2$ iff $\text{sort_key}(\tau_1, S_t) <_{\text{lex}} \text{sort_key}(\tau_2, S_t)$

where $<_{\text{lex}}$ is lexicographic ordering on (bytes32, bytes32) (the byte sequences compared LSB-first as unsigned).

5.7.3 Uniqueness guarantee

Because $\text{TxID}(\tau) = H_{\text{domain}}(\text{TX_ID}, \text{Encode}(\tau))$ and Encode is injective (per TH-1 from proofs/contractivity/encode_injectivity.v), no two distinct transactions can share both primary and secondary sort keys. The total ordering is unambiguous.

5.7.4 Admission ordering invariant

$I_t.\text{transactions}$ MUST be sorted by $<$ before admission into Domain A.

An I_t whose transactions are not in canonical order is **inadmissible** and triggers absorbing halt ($\perp$). This is non-negotiable: differently-ordered but semantically equivalent batches must not produce divergent state.

This is the operational realization of the canonical ordering axiom in 02_transition_axioms.md §A10.

5.8 §5 — Stability Budget System (§A8 Interface)

5.8.1 Perturbation budget declaration

Each transaction type TX-k declares a worst-case Lyapunov excursion:

$\epsilon_\tau \in \mathbb{Q}_p$ // static; declared at genesis per tx_type; immutable

This is a **static upper bound**, not a runtime computation. It must satisfy:

$\forall$ admissible (S_t, τ) :
 $|V_{\text{convergence}}(\sqcup \tau(S_t, \tau)) - V_{\text{convergence}}(S_t)| \leq \epsilon_\tau$

This is the §A8 Form B obligation. Form A is the special case $\epsilon_\tau = 0$. Form C is $\epsilon_\tau = 0$ plus a separate Φ_{safety} obligation.

5.8.2 Epoch budget constraint

For any admissible input set I_t :

$\sum_{\tau \in I_t.\text{transactions}} \epsilon_\tau \leq \epsilon_{\text{honest}}$

where $\epsilon_{\text{honest}} = 2000$ is the convergence proof target from 01_consensus.md §5.

This matches the two-threshold model defined in 02_transition_axioms.md §A8: $\epsilon_{\text{honest}} = 2_000$ is the proof target; $\epsilon_{\text{halt}} = 20_000$ is the halt trigger with a $10\times$ safety margin. TX-k budgets must sum to $\leq \epsilon_{\text{honest}}$ per epoch.

5.8.3 Budget accounting function

$\text{remaining_epoch_budget}(S_t) := \varepsilon_{\text{honest}} - \sum_{\{\tau \text{ already applied this epoch}\}} \varepsilon_{\tau}$

Used in $\square_{\tau}$ (§2) to reject any transaction whose ε_{τ} would exceed the remaining budget. Such rejection returns `AdmissibilityResult::Reject(0x32)` (“epoch ε -budget exceeded”); state and nonce unchanged.

5.8.4 Proof obligation

For each TX-k, a proof file at `proofs/contractivity/tx_perturbation_k.v` must discharge:

Theorem `TXk_perturbation_bound` :

$$\forall S_t \tau, \square_{\tau} S_t \tau \rightarrow \\ |V_{\text{convergence}}(\square_{\tau} S_t \tau) - V_{\text{convergence}} S_t| \leq \varepsilon_{\tau}.$$

Until discharged, ε_{τ} values are marked STATUS: PLACEHOLDER in Appendix A. Genesis lock requires all admitted TX-k to have proved bounds.

5.9 §6 — Slash Semantics

5.9.1 Slash increment bound

Each TX-k that produces slash increments declares:

$\sigma_{\tau} \in [0, \text{INT_MAX}]$ // declared at genesis per `tx_type`; immutable

Must satisfy:

$\forall \text{admissible}(S_t, \tau), \forall \text{affected validator } i:$

$$\Sigma_{i,t}(\square_{\tau}(S_t, \tau)) - \Sigma_{i,t}(S_t) \leq \sigma_{\tau}$$

This composes with Σ_i saturation at `INT_MAX` defined in `01_consensus.md §4b`. The pre-checked cap rule there ensures intermediate i128 arithmetic does not overflow.

5.9.2 Transactions with $\sigma_{\tau} = 0$

A transaction that does not produce slash evidence declares $\sigma_{\tau} = 0$ and need not include slash-related fields in its payload. TX-0 is such a transaction.

5.9.3 Slash-producing transactions

The first slash-producing transaction (TX-2) is NOT defined in this revision. Its design will exercise:

- Evidence encoding (canonical, replay-safe)
- Adjudication function (pure, deterministic)
- Deduplication (same evidence $\rightarrow$ no double-counting)
- Idempotent evidence handling

TX-2 will be added only after TX-0 and TX-1 are stable and CI-verified.

5.9.4 Post-genesis transaction types (economic)

Economic transaction types — including value transfer, receipt-based payment, blinded fee markets, and application-layer state transitions — are **excluded from the v1.0/v1.1 genesis algebra** and are deferred to post-genesis application-layer protocols or successor networks. No economic semantics are encoded in GENESIS_CONSTANTS.toml. The genesis transaction registry ($\square$) is closed at $\{TX-0, TX-1\}$.

5.10 §7 — Replay and Idempotence

5.10.1 Idempotence law

For any admissible transaction τ :

Let $S' = \square_{\tau}(S_t, \tau)$ (first application succeeds).

Then $\square_{\tau}(S', \tau) = \text{AdmissibilityResult}::\text{Reject}(0x31)$ (nonce mismatch).

This holds because applying τ advances the author's nonce, so τ 's nonce field no longer matches `expected_nonce(S',  $\tau$ .author_id)`. The second application fails admissibility before reaching $\square_{\tau}$.

5.10.2 Replay safety

Combined with TH-7 (replay invariance), the idempotence law implies:

`Replay(G, T)` on any authorized Ξ produces identical state, AND
no transaction is applied more than once.

This is replay safety, not just replay determinism.

5.10.3 Deduplication in input sets

If I_t .transactions contains two distinct entries τ_1 and τ_2 with $\text{TxID}(\tau_1) = \text{TxID}(\tau_2)$, the input set is malformed and triggers $\perp$.

Note: TxID includes the nonce, so the same transaction submitted with different nonces produces different TxIDs. Genuine duplicates can only arise from byte-identical entries.

5.11 §TX-0 — No-Op Transaction

5.11.1 Purpose

TX-0 is the minimal admissible transaction. It produces no state change other than advancing the author's nonce. It serves three roles:

1. **Heartbeat / liveness:** authors can prove activity without state mutation
2. **Test vector baseline:** encoding/ordering/admissibility round-trips

3. **Convergence proof base case:** the trivial §A8 Form A proof obligation

It is not optional. Every QASH network includes TX-0 as the empty case of the transaction algebra.

5.11.2 **Payload schema**

TX-0 payload = $\emptyset$ (payload_len = 0)

The envelope still includes the nonce and author_id; only payload is empty.

Total encoded TX-0 size: 2484 + 0 = 2484 bytes.

5.11.3 **Type-specific decode failures**

0x15 TX-0 payload_len $\neq$ 0 (TX-0 must have empty payload)

5.11.4 **Admissibility predicate**

```
payload_admissible(0, S_t, payload) :=  
  payload = [] (empty payload bytes)
```

There are no further per-type constraints. TX-0 is admissible whenever:

- The envelope decodes (§1)
- Author is active in S_t
- τ .nonce = expected_nonce(S_t , τ .author_id)
- Remaining epoch budget ≥ 0 (trivially satisfied since $\varepsilon_\tau = 0$)

5.11.5 **Transition function**

```
 $\square_{TX0}(S_t, \tau)$ :  
  // Precondition:  $\square_\tau(S_t, \tau) = \text{true}$   
  
  let author_idx = index_of_validator( $S_t$ ,  $\tau$ .author_id) // 0(1) via fixed array  
  
   $S_{\{t+1\}} = S_t$  with:  
    validators[author_idx].nonce  $\leftarrow$  validators[author_idx].nonce + 1  
    // No other field changes.  
  
  return  $S_{\{t+1\}}$ 
```

5.11.6 **Touch-set**

```
touch(TX-0) = { validators[author_idx].nonce }
```

The author's nonce is the only field modified. All other validator fields, the ledger root, state root, entropy seed, halt flag, epoch counter, and Lyapunov window remain bitwise identical between S_t and $S_{\{t+1\}}$.

5.11.7 Effect declarations

$\Delta D_*, t(\tau) = 0$ (no validator's divergence changes)
 $\Delta C_*, t(\tau) = 0$ (no validator's conflict changes)
 $\Delta \Sigma_*, t(\tau) = 0$ (no slash increments produced)
 $\Delta \text{entropy}(\tau) = \text{false}$
 $\Delta \text{ledger}(\tau) = \text{false}$

5.11.8 Declared bounds

$\varepsilon_\tau = 0$ (TX-0 makes no convergence perturbation)
 $\sigma_\tau = 0$ (TX-0 produces no slash evidence)

5.11.9 §A8 form: Form A (Non-increase)

TX-0 satisfies §A8 Form A with the strongest possible statement:

$\forall$ admissible S_t : $V_convergence(\square_TX0(S_t, \tau)) = V_convergence(S_t)$.

This is equality, not merely non-increase. It follows trivially because:

- $V_convergence$ is defined over validator divergence and conflict metrics
- TX-0 modifies neither divergence nor conflict (only nonce)
- Therefore $V_convergence$ is unchanged

5.11.10 Proof obligation

File: proofs/contractivity/tx_perturbation_0.v

Theorem: TX0_perturbation_bound

Statement (active Coq model):

$\forall$ validator nonce_next window_min,
 $\delta_window(\square_TX0(\text{validator}, \text{nonce_next}), \text{window_min})$
 $\leq \delta_window(\text{validator}, \text{window_min})$.

Proof sketch:

By touch-set confinement, only $\text{validators}[\text{author_idx}].\text{nonce}$ changes.
 $V_convergence$ is defined as $\Sigma_i (\alpha \cdot D_i + \beta \cdot C_i)$; it does not reference nonce.
Therefore $V_convergence$ is invariant under TX-0 application.

Status: FORMAL – proofs/contractivity/tx_perturbation_0.v; zero Admitted

5.11.11 Idempotence

Per §7, applying the same TX-0 twice rejects on the second attempt:

Apply TX-0 with nonce = $n \rightarrow \text{author.nonce}$ becomes $n+1$

Apply same TX-0 (nonce = n) again $\rightarrow \text{AdmissibilityResult}::\text{Reject}(0x31)$
(expected_nonce is now $n+1$)

5.11.12 Rejection taxonomy specific to TX-0

Layer	Reason	Trigger
Decode	0x10	malformed envelope
Decode	0x14	signature verification failed
Decode	0x15	payload_len $\neq$ 0
Admissibility	0x30	author not in active set
Admissibility	0x31	nonce mismatch
Admissibility	0x32	epoch ε -budget exceeded (impossible for TX-0 since $\varepsilon_\tau = 0$)

5.11.13 Test vectors

```
tv_tx0_basic:
  description: Valid TX-0 with author at nonce 0; should advance to nonce 1
  S_t:
    validators:
      - id: "0xalb2...[48 bytes]" # author
        nonce: 0
        active: true
        score: 100
        divergence: 0
        conflict: 0
        slash_acc: 0
     $\tau$ :
      version: 1
      tx_type: 0
      nonce: 0
      author_id: "0xalb2...[48 bytes]"
      payload_len: 0
      payload: []
      signature: "0x...[2420 bytes]" # valid Dilithium5 over envelope
  expected:
    result: success
    S_prime.validators[0].nonce: 1
    all_other_fields_unchanged: true
    epsilon_consumed: 0

tv_tx0_replay:
  description: Re-applying same TX-0 fails admissibility (nonce advanced)
  S_t: <state after tv_tx0_basic above>
   $\tau$ : <same as tv_tx0_basic. $\tau$ >
  expected:
    result: AdmissibilityResult::Reject(0x31)
    state_unchanged: true
```



```

tv_tx0_nonempty_payload:
  description: TX-0 with payload_len > 0 fails decode
  τ:
    version:      1
    tx_type:      0
    nonce:        0
    author_id:    "0x1b2...[48 bytes]"
    payload_len:  1
    payload:      [0x00]
    signature:    "0x...[2420 bytes]"
  expected:
    result: DecodeResult::Invalid(0x15)
    state_unchanged: true
    nonce_not_advanced: true

tv_tx0_inactive_author:
  description: TX-0 from inactive validator fails admissibility
  S_t:
    validators:
      - id:      "0x1b2...[48 bytes]"
        nonce:   0
        active:  false # inactive
  τ: <valid TX-0 envelope from this author>
  expected:
    result: AdmissibilityResult::Reject(0x30)
    state_unchanged: true

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                                     author_id (48 bytes)                                     |
|                                     ...                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      payload_len = 8      |      target_idx (u32, BE)      ...      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ... target_idx (cont) |      delta (u32, BE)      ...      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ... delta (cont)      |      signature (2420 bytes, Dilith5)      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Payload fields (payload_len MUST equal 8):

Field	Type	Size	Description
target_idx	u32 big-endian	4 bytes	Index into S_t.validators[]
delta	u32 big-endian	4 bytes	Decrement amount; new_D = max(0, D_i - delta)

5.12.3 Type-specific decode failures

Code	Condition
0x16	payload_len $\neq$ 8
0x17	target_idx encodes a value $\geq N_{\max}$ (1024)

5.12.4 Admissibility predicate

Let $i = \text{target_idx}$, $D_i = S_t.\text{validators}[i].\text{divergence}$.

```

admissible_TX1(S_t,  $\tau$ ):
   $\wedge$  author_id  $\in$  active_set(S_t)                                (§A6: author active)
   $\wedge$   $\tau$ .nonce = S_t.validators[author_idx].nonce                (§A6: nonce match)
   $\wedge$  target_idx < |S_t.validators|                                (valid index)
   $\wedge$  delta  $\leq$  D_i                                                (bounded decrement)
   $\wedge$  epoch_budget_ok(S_t,  $\varepsilon_{\text{TX1}}$ )                          (§A8: budget)

```

Additional admissibility rejection codes:

Code	Condition
0x30	author not in active set
0x31	nonce mismatch
0x33	target_idx out of bounds for current validator set
0x34	delta > D_i (would underflow; violates bounded decrement)

5.12.5 Transition function

$\square_{\text{TX1}}(S_t, \tau)$:

```

let author_idx = index_of_validator(S_t,  $\tau$ .author_id)
let i          =  $\tau$ .payload.target_idx
let delta      =  $\tau$ .payload.delta
let D_i        = S_t.validators[i].divergence
let new_D      = max(0, D_i - delta)  $\leftarrow$  checked; delta  $\leq$  D_i guaranteed by admissibility
S_{t+1} = S_t with
  validators[author_idx].nonce  $\leftarrow$  nonce + 1
  validators[i].divergence      $\leftarrow$  new_D

```

Touch-set: { validators[author_idx].nonce, validators[i].divergence }

5.12.6 Invariant impact

Metric	Effect	Explanation
D _{i,t}	$\downarrow$ bounded	new_D $\leq$ D _i ; cannot increase
C _{i,t}	unchanged	conflict field untouched
$\Sigma_{i,t}$	unchanged	slash_accum untouched
V_convergence	$\downarrow$ or unchanged	weight_D * Δ D $\leq$ 0; formally proved

5.12.7 §A8 properties

§A8 form: Form A (stronger: non-increasing)

ϵ_{τ} : ≤ 0 (V_convergence cannot increase)

σ_{τ} : 0 (slash_accum unchanged)

Proof file: proofs/contractivity/tx1_score_decrement.v

Proof status: FORMAL — TX1_score_decrement_nonincreasing fully proved, zero Admitted markers.

5.12.8 §A1 determinism argument

target_idx and delta are decoded from canonical big-endian fixed-width fields. The max(0, D_i - delta) uses checked subtraction and is platform-independent. No entropy or wall-clock ingress.

5.12.9 §A2 termination bound B _{τ}

Constant: decode (O(1)) + two field writes (O(1)). No loops.

5.12.10 §A4 StateWF preservation

new_D = max(0, D_i - delta) $\in$ [0, D_i] $\subseteq$ [0, scale]. All other fields unchanged from a WF state; nonce + 1 is checked (overflow $\rightarrow$ absorbing halt).

5.12.11 Rejection taxonomy

Class	Code	Condition
Decode	0x10	malformed envelope
Decode	0x14	signature verification failed
Decode	0x16	payload_len $\neq$ 8
Decode	0x17	target_idx $\geq$ N_max
Admissibility	0x30	author not in active set
Admissibility	0x31	nonce mismatch
Admissibility	0x33	target_idx out of bounds
Admissibility	0x34	delta > D_i

5.12.12 Test vectors

```
tv_tx1_basic:
  description: Valid TX-1 decrements divergence of validator 1 by 100
  S_t:
    validators:
      - id: "0xalb2...[48 bytes]" # author, idx=0
        nonce: 5
        active: true
        divergence: 500
      - id: "0xc3d4...[48 bytes]" # target, idx=1
        active: true
        divergence: 800
    t:
      version: 1
      tx_type: 1
      nonce: 5
      author_id: "0xalb2...[48 bytes]"
      payload_len: 8
      payload: [0x00, 0x00, 0x00, 0x01, # target_idx = 1 (BE)
                0x00, 0x00, 0x00, 0x64] # delta = 100 (BE)
      signature: "0x...[2420 bytes]"
    expected:
      result: success
      S_prime.validators[0].nonce: 6
      S_prime.validators[1].divergence: 700
      all_other_fields_unchanged: true

tv_tx1_delta_equals_divergence:
  description: delta == D_i reduces divergence to exactly 0
  S_t:
    validators:
      - id: "0xalb2...[48 bytes]"
        nonce: 0
        active: true
```

```

        divergence: 30
    - id: "0xc3d4...[48 bytes]"
      active: true
      divergence: 30
    τ:
      tx_type: 1
      nonce: 0
      author_id: "0xa1b2...[48 bytes]"
      payload: [0x00, 0x00, 0x00, 0x01, # target_idx = 1
                0x00, 0x00, 0x00, 0x1E] # delta = 30
    expected:
      result: success
      S_prime.validators[1].divergence: 0

tv_tx1_delta_exceeds:
  description: delta > D_i is rejected with 0x34
  S_t:
    validators:
      - id: "0xa1b2...[48 bytes]"
        nonce: 0
        active: true
        divergence: 100
      - id: "0xc3d4...[48 bytes]"
        active: true
        divergence: 50
    τ:
      tx_type: 1
      nonce: 0
      author_id: "0xa1b2...[48 bytes]"
      payload: [0x00, 0x00, 0x00, 0x01, # target_idx = 1
                0x00, 0x00, 0x00, 0x64] # delta = 100 > 50
    expected:
      result: AdmissibilityResult::Reject(0x34)
      state_unchanged: true

tv_tx1_bad_payload_len:
  description: payload_len ≠ 8 is rejected with 0x16
  τ:
    tx_type: 1
    payload_len: 4
    payload: [0x00, 0x00, 0x00, 0x01]
  expected:
    result: DecodeResult::Invalid(0x16)
    state_unchanged: true

```

5.13 Appendix A — Transaction Type Registry

This registry is **frozen at genesis**. Adding or removing types requires a new network.

tx_type	Name	§A8 form	ε_τ	σ_τ	touch fields	Proof status
0	TX-0 No-Op	A	0	0	{validators[authFQRMate]}	FORMAL
1	TX-1 Bound- edVal- ida- torScore- Decre- ment	A (non- increasing)	≤ 0	0	{validators[authFQRMate, valida- tors[target].divergence]}	FORMAL

Future revisions will extend this table as transaction types are ratified.

5.14 Appendix B — Cross-Document Dependency

Reference	Document	Section
Canonical encoding	01_consensus.md	§2
State space	01_consensus.md	§1
Stability functions	01_consensus.md	§4a, §4b
Stability criterion	01_consensus.md	§5
Arithmetic model	00_execution_model.md	§E1
Domain partition	00_execution_model.md	§E0
Oracle contract (signatures)	00_execution_model.md	§E0
Hash domain tags	00_execution_model.md	§E4
Halt semantics	00_execution_model.md	§E5
Transition axioms	02_transition_axioms.md	§A0-A11
§A8 forms	02_transition_axioms.md	§A8
Canonical ordering	02_transition_axioms.md	§A10

End of *docs/spec/03_transactions.md* SHA3-256 of this document is recorded in *GENESIS_CONSTANTS.toml* at genesis lock.

6 QASH Adversarial Model (Scoped)

6.1 §A0 — Scope and Partition Alignment

This adversarial model applies to the same execution partition defined in `00_execution_model.md` §E0.

- **Domain A (deterministic consensus):** theorem-bearing computation.
- **Domain B (transport/operations):** nondeterministic environment.

Consensus-state determinism claims are strictly Domain-A claims and do **not** assume reliable transport, stable clocks, or fair scheduling.

6.2 §A1 — Scenario Classes

6.2.1 A1.1 Network partitions (Domain B)

Adversary may split honest validators into disjoint communication components for an unbounded-but-finite duration.

Assumptions: - Messages can be delayed or dropped across partition boundaries. - Domain A transition semantics remain unchanged.

Non-goals: - No guarantee of progress during active partition. - No guarantee of minimal recovery time after heal.

6.2.2 A1.2 Message reorderings / duplication (Domain B)

Adversary may reorder, duplicate, and delay delivery of signed external inputs.

Assumptions: - Domain A admissibility checks reject malformed/invalid inputs. - Replay and state evolution are defined only over canonical admissible input sets.

Non-goals: - No FIFO transport guarantee. - No exactly-once delivery guarantee.

6.2.3 A1.3 Byzantine input construction (boundary adversary)

Adversary may generate syntactically valid but semantically conflicting signed inputs, including equivocation attempts and slash-triggering evidence.

Assumptions: - Cryptographic assumptions of configured primitives hold. - Domain A verification is complete and deterministic.

Non-goals: - No prevention claim for attempted Byzantine behavior. - Only detection, containment, and deterministic accounting are claimed.

6.3 §A2 — Safety vs Liveness Separation

6.3.1 Safety claims (must hold under all A1 scenarios)

1. **Deterministic replay:** equal genesis + equal canonical admissible inputs imply equal post-state and equal `state_root`.

2. **No invalid transition acceptance:** malformed or inadmissible input does not produce an admissible next state.
3. **Slash accounting determinism:** slash evidence yields deterministic, bounded updates per protocol rules.

6.3.2 Liveness claims (conditional)

Liveness is claimed only under explicit environmental conditions (eventual message delivery, eventual partition healing, and sufficient active honest validators). Liveness is not implied by safety nor by deterministic replay.

6.4 §A3 — Theorem Obligations by Scenario Class

Each scenario class must have explicit obligations in the proof plan.

- **TH-A1 (Partition Safety):** safety invariants are preserved across any partition schedule and subsequent merge of admissible inputs.
- **TH-A2 (Reordering Safety):** permutation/duplication of transport delivery does not alter Domain-A outcome once canonical admissible input set is fixed.
- **TH-A3 (Byzantine Boundary Safety):** Byzantine inputs are either rejected or deterministically incorporated (e.g., slash evidence) without violating admissibility invariants.
- **TH-A4 (Conditional Liveness):** eventual progress under stated fairness and synchrony assumptions only.

Each theorem entry MUST include: - exact assumptions, - exact safety/liveness classification, - explicit non-goals, - mapping to executable simulation scenarios.

6.5 §A4 — Simulation Hooks and CI Progression

Scenario-based simulation hooks are required for each A1 class:

1. **Phase 1 (non-blocking in CI):** run adversarial scenario suites with continue-on-error to collect flakiness/perf baselines.
2. **Phase 2 (blocking in CI):** promote each suite to required once stability criteria are met (documented pass-rate and runtime budget).

Promotion criteria and status tracking are maintained in proofs/STATUS.md.

7 QASH Astronomical Hash Cascade

7.1 Purpose

The astronomical hash cascade (H_cascade) provides a depth-7, multi-primitive hash construction used for:

1. Obfuscation layer leaf hashing (cascade_derived_injective mode)
2. Clone-chunk verification (cascade_bound mode in clone protocol v1.2)
3. Cascade health (CH) commitment included in V_convergence (§4c of 01_consensus.md)

H_cascade is **not** used for state root computation, which uses H_domain (SHA3-256, defined in 00_execution_model.md §E4).

7.2 Formal Definition

H_cascade(input: &[u8]) → [u8; 64]

7.2.1 Layer 1 — Parallel primitive application

Five primitives are applied in parallel. Each receives a domain-separated input: tag □ input where tag is the L1 domain separator string encoded as UTF-8 bytes (no null terminator, no length prefix).

L1_sep = "QASH:CASCADE:L1:PARALLEL" (UTF-8, 24 bytes)

```
h1[0] = SHA3-256 ( L1_sep □ input )    // [u8; 32]
h1[1] = BLAKE3   ( L1_sep □ input )    // [u8; 32]
h1[2] = K12      ( L1_sep □ input )    // [u8; 32]    (KangarooTwelve, 32-
byte output)
h1[3] = SM3      ( L1_sep □ input )    // [u8; 32]
h1[4] = Streebog ( L1_sep □ input )    // [u8; 32]    (GOST R 34.11-2012, 256-
bit output)
```

parallel = h1[0] □ h1[1] □ h1[2] □ h1[3] □ h1[4] // [u8; 160]

Primitive output sizes are fixed at 32 bytes each. Implementations must use the 256-bit output variant of each primitive. Any variable-output primitive must be configured for exactly 32 bytes of output.

7.2.2 Layer 2 — Binding

L2_sep = "QASH:CASCADE:L2:BIND" (UTF-8, 20 bytes)

L2 = SHA3-512(L2_sep □ parallel) // [u8; 64]

SHA3-512 is the binding primitive (binding_primitive = "SHA3-512" in genesis). The binding layer ensures all five L1 outputs are committed into a single 64-byte value before recursive expansion.

7.2.3 Layers 3-6 — Recursive expansion

For $n \in \{3, 4, 5, 6\}$:

$L\{n\}_{sep} = \text{"QASH:CASCADE:L\{n\}:EXPAND"}$ (UTF-8, e.g. "QASH:CASCADE:L3:EXPAND" = 22 bytes)

$L\{n\} = \text{SHA3-512}(L\{n\}_{sep} \parallel L\{n-1\})$ // [u8; 64]

Each layer takes the 64-byte output of the previous layer as input.

7.2.4 Layer 7 — Finalization

$L7_{sep} = \text{"QASH:CASCADE:L7:FINALIZE"}$ (UTF-8, 24 bytes)

$L7 = \text{SHA3-512}(L7_{sep} \parallel L6)$ // [u8; 64]

$H_{cascade}(\text{input}) = L7$

7.3 Domain Separators

The seven domain separator strings (from `GENESIS_CONSTANTS.toml [crypto.cascade].domain_sepa` are:

Layer	Separator string	Byte length
L1	QASH:CASCADE:L1:PARALLEL	24
L2	QASH:CASCADE:L2:BIND	20
L3	QASH:CASCADE:L3:EXPAND	22
L4	QASH:CASCADE:L4:EXPAND	22
L5	QASH:CASCADE:L5:EXPAND	22
L6	QASH:CASCADE:L6:EXPAND	22
L7	QASH:CASCADE:L7:FINALIZE	24

These strings are fixed at genesis. Their byte lengths are not encoded in the hash input — the strings themselves are the separators. Length variability is not a concern because the strings are protocol constants.

7.4 Cascade Proof Format

When `cascade_proof_inclusion = "sparse_merkle"`, each cascade output is committed into the block's cascade sparse-Merkle tree. An inclusion proof for a cascade computation is:

```
CascadeProof = (  
  leaf_index: [u8; 48], // epoch-relative leaf index (from obfuscation.leaf_index_bytes)  
  l7_output: [u8; 64], // H_cascade output
```

```
merkle_path: [[u8; 32]; sparse_merkle_depth], // depth = 384 inner nodes
)
```

Proof verification: recompute the Merkle root from leaf_index, l7_output, and merkle_path using H_domain(LEAF_HASH, ...) / H_domain(INTERNAL_HASH, ...) (tags 0x00000004 / 0x00000005 from 00_execution_model.md §E4).

7.5 Cascade Health Factor Derivation

CH_t (§4c of 01_consensus.md) is derived from cascade proof rejections in I_t. A cascade proof is **rejected** if:

1. The recomputed Merkle root does not match the block's cascade tree root, or
2. The l7_output field does not equal H_cascade(original_input) for the associated obfuscation leaf or clone chunk.

```
cascade_fail_count_t = |{ proofs in I_t that are rejected }|
CH_t = cascade_fail_count_t × p / max_queries_per_epoch
```

CH_t is a Domain A value computed entirely from admitted, signature-verified inputs. No Domain B values influence it.

7.6 Implementation Notes

7.6.1 Cross-ISA determinism

All five L1 primitives and SHA3-512 must produce bitwise-identical output on all Tier A ISAs. Hardware acceleration is permitted in Domain B only; the Domain A cascade path must use reference implementations verified by the cross-ISA test suite.

7.6.2 Parallelism

L1 computation across the five primitives may be parallelized in the PAL layer (Domain B), provided the concatenation order is fixed: SHA3-256, BLAKE3, K12, SM3, Streebog (array indices 0–4). The Domain A caller receives the concatenated parallel slice; it does not observe the parallelism.

7.6.3 No streaming

All hash inputs are single contiguous slices. No streaming API is used (streaming permits platform-divergent buffering). This is the same constraint as H_domain.

7.7 Relationship to DD-3 (SHA3-256 as canonical consensus hash)

H_cascade and H_domain are **distinct functions with distinct purposes**:

Function	Purpose	Output size	Domain
H_domain	State root, entropy seed, Merkle nodes	[u8; 32]	Domain A — consensus
H_cascade	Obfuscation leaf, clone chunk, CH	[u8; 64]	Domain A — cascade path

For v1.0 genesis, state root computation **never** calls H_cascade. Cascade health commitment **never** replaces the state root computation. The two functions are orthogonal. Any future cascade-backed state-root migration must define a new commitment/truncation rule, update compute_state_root, and replace the genesis KAT vectors in a separate post-genesis ADR.

7.8 §4 — Cascade Blinding (Context-Keyed Variant)

To prevent preimage grinding of deterministic cascade inputs and to bind epoch-local operations to their specific epoch context, the cascade supports a **context key** that is mixed into the L2 binding layer.

7.8.1 Formal definition

$H_cascade_keyed(context_key: \&[u8], input: \&[u8]) \rightarrow [u8; 64]$

The keyed variant is identical to H_cascade except at Layer 2:

$L2 = SHA3-512(L2_sep \parallel context_key \parallel parallel)$

The unkeyed form is the degenerate case:

$H_cascade(input) \equiv H_cascade_keyed([], input)$

context_key is always a protocol-level constant or deterministically derived value — never a secret or nonce sampled from a random source. This preserves Domain A replay invariance (00_execution_model.md §E2, axiom A5 of 02_transition_axioms.md).

7.8.2 Blinding in cascade proofs

When generating a cascade proof for obfuscation leaf i at epoch t , the context key is the epoch entropy seed:

$leaf_hash_i_t = H_cascade_keyed(seed_t, validator_id_i \parallel epoch_t_le8)$

where $seed_t = S_t.entropy_seed$ (01_consensus.md §1) and $epoch_t_le8$ is the 8-byte little-endian encoding of the epoch counter.

This binds each leaf hash to its epoch, preventing cross-epoch proof replay. The leaf index used in the sparse Merkle tree is the first 48 bytes of $leaf_hash_i_t$, consistent with `obfuscation.leaf_index_bytes = 48` (GENESIS_CONSTANTS.toml [obfuscation]).

7.8.3 Genesis hash blinding nonce

The genesis hash computation uses a **genesis blind nonce** pinned in `GENESIS_CONSTANTS.toml` `[meta].genesis_blind_nonce`. The nonce is included in the canonical TOML bytes (it is **not** stripped before hashing, unlike `genesis_hash` and `lock_algorithm`). Its value is fixed at genesis lock and cannot be changed without redefining the network.

```
genesis_hash = hex( H_cascade_keyed( [], canonical_genesis_bytes_with_nonce ) )
```

where `canonical_genesis_bytes_with_nonce` is the TOML file with only `genesis_hash` and `lock_algorithm` values blanked (the nonce line is kept verbatim).

7.9 §5 — Hierarchical Deterministic Cascade Derivation

To support epoch-keyed cascade roots for cascade health proofs and clone-chunk verification, an HD derivation function is defined:

```
H_cascade_derive(parent_root: [u8; 64], epoch: u64, seed: [u8; 32]) → [u8; 64]
```

Implementation:

```
derive_input = epoch_le8 □ seed          // 8 + 32 = 40 bytes
```

```
H_cascade_derive(parent_root, epoch, seed) = H_cascade_keyed(parent_root, derive_input)
```

where `epoch_le8` is the 8-byte little-endian encoding of epoch.

7.9.1 Epoch cascade root chain

```
cascade_root_0 = H_cascade( canonical_genesis_bytes )    // genesis root (§3)
```

```
cascade_root_t = H_cascade_derive( cascade_root_{t-1}, t, seed_t )
```

`cascade_root_t` is committed in the epoch state and used to anchor cascade proofs in epoch `t`. Validators must supply cascade proofs against `cascade_root_t`; proofs against a stale root are rejected (counted in `CH_t`).

7.9.2 Security properties

- **Forward security:** given `cascade_root_t` and `seed_t`, an observer cannot compute `cascade_root_{t-1}` without inverting SHA3-512 (7 layers deep).
 - **Epoch binding:** `cascade_root_t` depends on every prior epoch seed, so it cannot be forged without controlling the entropy advance chain (`H_domain(ENTROPY_ADVANCE, ...)` per `01_consensus.md` §1).
 - **Replay invariance:** all inputs are deterministic; no nondeterministic values enter the computation (axiom A1 of `02_transition_axioms.md`).
-

7.10 §6 — Implementation Requirements

All implementations of `H_cascade`, `H_cascade_keyed`, and `H_cascade_derive` **must** use pure-Rust (or safe-Rust) implementations of all five L1 primitives and SHA3-512. C FFI, assembly backends, or hardware intrinsic paths are forbidden in the Domain A cascade path, as they may produce ISA-dependent output under certain edge cases and cannot be verified by the Coq proof suite.

Concretely for the reference implementation:

Primitive	Crate	Required feature
SHA3-256, SHA3-512	<code>sha3 = "0.10"</code>	<code>default-features = false</code>
BLAKE3	<code>blake3 = "1.5.5"</code>	<code>default-features = false, features = ["pure"]</code>
KangarooTwelve	<code>tiny-keccak = "2"</code>	<code>features = ["k12"]</code>
SM3	<code>sm3 = "0.4"</code>	<code>default-features = false</code>
Streebog-256	<code>streebog = "0.10"</code>	<code>default-features = false</code>

The `pure` feature on `blake3` disables all assembly and C backends, ensuring the RustCrypto-compatible Rust implementation is used. This requirement is normative; CI must reject builds where `cc` is invoked transitively from any cascade-related crate.

End of *docs/spec/07_hash_cascade.md*

8 QASH Canonical Test Vectors

Status: Normative

Backs theorem: TH-7 (replay invariance across all authorized ISAs)

8.1 Purpose

These vectors define the expected outputs for specific, fully-specified input sequences. Any implementation claiming QASH compatibility **MUST** reproduce every value below on every authorized ISA (`x86_64`, `aarch64`, `riscv64gc`). A mismatch is a protocol violation, not a platform quirk.

Each test case maps to a named Rust test. CI runs those tests on all three ISA targets.

8.2 Notation

- All byte sequences: `[b0, b1, ..., bN]` in decimal, little-endian where indicated.

- `genesis_state`: `epoch=0`, `halt=None`, `entropy_seed=[0u8;32]`, `validator_count=4`, all `ValidatorMetrics::ZERO`, `nonces=[0;1024]`, `validator_ids=[[0u8;48];1024]`, `state_root=[0u8;32]`.
 - `idle_input(N)`: `update_count=N`, all `updates[i]=None`.
 - `TX0(author_id, nonce)`: wire bytes for a TX-0 (nonce-advance) envelope with the given author and nonce. See §4 wire format in `03_transactions.md`.
 - Fixed-point values use scale `1_000_000` (see `GENESIS_CONSTANTS.toml`).
-

8.3 Test Vectors

8.3.1 TV-0 — Genesis state root

Input: `genesis_state`

Expected: `state_root = [0u8; 32]`

Rust test: `state_root_genesis_is_zero` (`crates/consensus/src/transition.rs`)

The genesis state carries `state_root=[0u8;32]` before any epoch is advanced.

8.3.2 TV-0a — Genesis epoch-1 state-root commitment KAT

Input: `genesis_state` → `advance_epoch(idle_input(4), [])` **Commitment function:** `H_domain(STATE_ROOT, Encode_for_commitment(S_1, [0u8;32]))` (SHA3-256 over `0x01000000 || preimage`, not `H_cascade`). **Expected preimage length:** 460 bytes **Expected preimage SHA3-256:**

`a92369cc605eacb076e28fdb2ca7d5f26bae9f566290da09863e7ca5365b4c0e`

Expected state_root:

`513e26e067c3857289f0557adc7db9ea43cda6595cda4510cbf11af06196d352`

Rust test/vector: `state_root_commitment_genesis_epoch1` in `tests/vectors/vectors.v1.json`, verified by `crates/consensus/tests/vector_runner.rs`.

This KAT pins the exact v1.0 genesis state-root commitment function. Cascade state-root activation is a post-genesis migration item and would require a new commitment rule plus replacement KATs.

8.3.3 TV-1 — 3-epoch idle root (TH-7 anchor)

Input: `genesis_state` → `3 × advance_epoch(idle_input(4), [])`

Expected state_root after epoch 3:

```
[138, 219, 164, 211, 10, 54, 30, 39,
 151, 223, 239, 42, 191, 141, 13, 181,
 121, 224, 79, 241, 4, 74, 49, 44,
 138, 224, 93, 197, 103, 104, 122, 198]
```

Rust test: `state_root_canonical_seq_golden` (crates/consensus/tests/golden_replay.rs)

This is the primary TH-7 cross-ISA anchor. The value is reproduced by the `verify_two_stage_build.sh` script on all three ISA targets.

8.3.4 TV-2 — TX-0 nonce advance

Input: `genesis_state` with `validator_ids[0][0]=1`, `validator_ids[1][0]=2`; then `advance_epoch(idle_input(4), [TX0(validator_ids[0], 0), TX0(validator_ids[1], 0)])`

Expected: `nonces[0]=1`, `nonces[1]=1`, `nonces[2]=0`, `nonces[3]=0`

Rust test: `full_epoch_with_tx0` (crates/consensus/tests/golden_replay.rs)

8.3.5 TV-3 — TX-0 replay rejection

Input: same as TV-2, then a second `advance_epoch` with `TX0(validator_ids[0], 0)` (nonce already consumed).

Expected: `nonces[0]` unchanged at 1 after epoch 2 (duplicate TX ignored, not halted).

Rust test: `tx0_replay_rejected` (crates/consensus/src/transaction.rs)

8.3.6 TV-4 — TX-0 unknown author rejection

Input: `genesis_state` (all `validator_ids=[0u8;48]`); `advance_epoch(idle_input(4), [TX0([0xABu8; 48], 0)])` — author not in validator set.

Expected: `nonces` unchanged; epoch advances normally (TX ignored).

Rust test: `tx0_unknown_author_ignored` (crates/consensus/src/transaction.rs)

8.3.7 TV-5 — Halt trigger (H1: Lyapunov violation)

Input: `genesis_state` → $3 \times \text{idle_input}(4)$ (fills window, `V_convergence=0` each); then `advance_epoch(spike_input, [])` where `spike_input.updates[0].divergence_new = 1_000_000` (= SCALE), all others `None`.

Expected: `Err(HaltReason::LyapunovViolation)`.

Rust test: `halt_freezes_entire_state_except_halt_reason` (crates/consensus/tests/golden_replay.rs)

8.3.8 TV-6 — Halt is absorbing

Input: TV-5 halted state; 10 subsequent calls to `advance_epoch`.

Expected: every call returns `Err(HaltReason::LyapunovViolation)`; state fingerprint is identical before and after every call.

Rust test: `golden_halt_reason_preserved (crates/consensus/tests/golden_replay.rs)`

8.3.9 TV-7 — Entropy chain determinism

Input: `genesis_state` $\rightarrow N \times \text{idle_input}(4)$ for any $N \geq 1$.

Expected: `entropy_seed` $\neq [0u8;32]$ after epoch 1; identical value across `x86_64`, `aarch64`, `riscv64gc` for the same N .

Derivation formula: `entropy_seed_{t+1} = h_domain(EntropyAdvance, entropy_seed_t)` where `h_domain(tag, data) = SHA3-256(tag_u32_le || data)`.

Rust test: `entropy_seed_advances_nonzero (crates/consensus/src/transition.rs)`

8.3.10 TV-8 — Hash cascade determinism

Input: any fixed 32-byte buffer `b`.

Expected: two independent calls to `h_domain(tag, b)` with the same tag produce byte-identical outputs on the same ISA and across all authorized ISAs.

Scope: the current Domain A cascade implementation (SHA3-256 with domain separation via tag prefix). BLAKE3 and KangarooTwelve cascade stages are deferred to the Domain B specification; their determinism test vectors will be added in a future revision of this document.

Rust test: `cascade_determinism_same_input (crates/consensus/src/hash.rs)`

8.4 Gate Rule (TH-7)

A build is considered TH-7-passing when:

1. `cargo test -p qash-consensus --no-default-features` exits 0 on **all three** of `x86_64-unknown-linux-gnu`, `aarch64-unknown-linux-gnu`, `riscv64gc-unknown-linux-gnu`.
2. The `state_root_canonical_seq_golden` test produces the exact TV-1 byte sequence on each target.
3. The `./scripts/verify_two_stage_build.sh` script exits 0.

A genesis proposal that lacks TH-7 CI evidence for all three ISA targets is inadmissible.

8.5 Regeneration Procedure

If a deliberate protocol change alters TV-1 (e.g., encoding format change), the new canonical root MUST be regenerated as follows:

```
PRINT_GOLDEN=1 cargo test -p qash-consensus --no-default-features \
  -- --nocapture state_root_canonical_seq_print
```

Update EXPECTED_STATE_ROOT_3_EPOCHS in golden_replay.rs ONLY after verifying that all three ISA targets produce the new value and that the change is intentional.

Any change to TV-1 constitutes a fork.

9 QASH Protocol Migration: v1.0 → v1.1

9.1 Summary of Changes

Area	v1.0	v1.1
GENESIS_CONSTANTS.toml version	1.0.0	1.1.0
Lyapunov weight D (α)	400_000	350_000
Lyapunov weight C (β)	350_000	300_000
Lyapunov weight Σ (γ)	250_000	200_000
Cascade health weight CH (χ)	—	150_000
Obfuscation method	concatenation_injective	cascade_derived_injective
Hash cascade	Serial 3-primitive chain	Depth-7 astronomical cascade (DD-7)
Clone protocol version	v1.1	v1.2
Clone chunk verification	—	cascade_bound
KEM	—	ML-KEM-768
PQC rotation schedule	(absent from TOML)	Pre-baked in [crypto.cascade.rotation_schedule]
New theorems	TH-1 through TH-8	+ TH-9, TH-10, TH-11

9.2 Compatibility Window

[migration.compatibility] in GENESIS_CONSTANTS.toml defines the window during which v1.0 validators are accepted:

```
accept_v1_0_validators = true
migration_window_epochs = 100
v1_0_validator_cascade_mode = "parallel_only"
```

```
post_migration_enforcement = "cascade_required"
state_conversion_proof_required = true
```

9.2.1 Window semantics

- **Epochs 0-99:** v1.0 validators are admitted with `cascade_mode = parallel_only`. They compute cascade outputs but are not required to submit cascade inclusion proofs. Their cascade health contribution (CH_t) is counted as zero.
- **Epoch 100+:** `cascade_required`. All validators must submit valid cascade proofs. Validators that do not are treated as cascade-failing and contribute to CH_t .

9.2.2 State conversion

Validators converting from v1.0 to v1.1 must: 1. Recompute $V_{convergence}$ using new weight table ($\alpha, \beta, \gamma, \chi$ from v1.1) 2. Compute $H_{cascade}$ for all pending obfuscation leaves using the depth-7 construction 3. Submit `state_conversion_proof` — a sparse-Merkle proof that all existing leaf hashes have been recomputed under the new cascade

`state_conversion_proof_required = true` means validators that skip step 3 are treated as inadmissible from epoch 100 onwards.

9.3 Weight Change Impact on Φ_{max_safe}

v1.0 $\Phi_{max} = 1024 \times 250_000 \times (2^{63} - 1) \approx 2.36 \times 10^{27}$ v1.1 $\Phi_{max} = 1024 \times 200_000 \times (2^{63} - 1) \approx 1.89 \times 10^{27}$

The reduced γ means the halt threshold is reached at a lower total slash accumulation. Validators with high slash accumulators that were safe under v1.0 may be closer to the halt threshold under v1.1. The migration window provides 100 epochs for operators to assess this.

9.4 Cascade Health (CH) — New Term in $V_{convergence}$

v1.0 had no cascade health term. v1.1 adds:

$\chi \cdot CH_t$ where $\chi = 150_000$, $CH_t \in [0, p = 1_000_000]$

In a healthy epoch (all cascade proofs pass), $CH_t = 0$ and the term contributes nothing. A fully degraded epoch (all proofs fail) adds $150_000 \times 1_000_000 = 1.5 \times 10^{11}$ to $V_{convergence}$.

During the migration window (epochs 0-99), v1.0 validators contribute $CH_t = 0$ for their cascade proofs (as if all their proofs pass). This prevents the new term from unfairly triggering halt during conversion.

9.5 Clone Protocol v1.2 Changes

`chunk_verification_mode = "cascade_bound"` means each clone chunk now carries a cascade inclusion proof (format defined in `07_hash_cascade.md`). During the migration window, v1.0 chunks without proofs are accepted but logged. After epoch 100, proofless chunks are rejected.

9.6 Formal Obligations

Obligation	Status
TH-9 (CH boundedness)	□ PROVED — <code>proofs/cascade/cascade_health_bounded.v</code> (no Admitted; CI-gated)
TH-10 (cascade collision resistance)	□ AXIOM — intentionally reduces to AX-3 (SHA3-256 collision resistance); same trust class as AX-3 in <code>proofs/STATUS.md</code> . The reduction argument is: if <code>H_cascade</code> is not collision-resistant then at least one L1 primitive is breakable, which contradicts AX-3.
TH-11 (cascade cross-ISA determinism)	□ CI-VERIFIED — cross-ISA test vectors validated on all Tier A ISAs; same delegation class as TH-7. Formal Coq proof would require axiomatizing ISA semantics (AX-1); CI verification is the accepted alternative.
Weight-adjusted TH-3 proof	□ N/A for v1.0 — <code>proofs/contractivity/lyapunov_stability.v</code> uses v1.0 weight constants ($D=400k$, $C=350k$, $S=250k$, no CH term). Proof is weight-agnostic; all three theorems hold for any positive weights.
Weight-adjusted TH-5 proof	□ N/A for v1.0 — <code>proofs/safety/absorbing_halt.v</code> uses v1.0 $\gamma=250_000$. Proof structure is weight-agnostic; $\Phi_{\max} = N_{\max} \times 250_000 \times INT_MAX$ $INT_MAX = 1024 \times 250,000 \times (2^{63}-1) \approx 2.36 \times 10^{27}$.

9.6.1 Genesis Lock Gate — Current Status (v1.0)

All obligations discharged at v1.0: - TH-9 PROVED, TH-10 AX-3 reduction, TH-11 CI-verified - Proofs aligned to v1.0 weights (D=400k, C=350k, S=250k); no CH term
- TH-7 cross-ISA: x86_64, aarch64, and riscv64gc all produce identical state roots (verified via QEMU user-static; .cargo/config.toml configures runners; CI workflow at .github/workflows/platform-determinism.yml installs required packages)

v1.1 migration gate (deferred): activates when `WEIGHT_BH > 0` and `GENESIS_CONSTANTS.toml` version is advanced to 1.1.0. At that point, the weight-adjusted proofs described in this table must be re-verified.

End of *docs/spec/09_migration_v1.0_to_v1.1.md*

10 QASH Privacy Model

Status: Normative

Scope: Domain A public surface boundary and observer-class definitions. Domain B blinding implementation and formal proof obligations are deferred (see §P8, §P10).

10.1 §P0 — Privacy posture

QASH is a graph-non-publishing system, not a graph-obfuscating system. External passive observers never receive a semantically meaningful transaction graph.

The distinction from Monero/Zcash-style systems is architectural: those systems broadcast a public graph and then cryptographically obscure its semantics; QASH does not broadcast a graph at all. The public surface is a deterministic commitment synchronisation mesh, not a transaction gossip network.

10.2 §P0a — What QASH does NOT protect against (normative)

Threat	Why out of scope	Mitigation (user responsibility)
Endpoint compromise	Cryptographic privacy holds only if keys are not extracted. TEE/OEM halts before responding, but hardware extraction is outside protocol scope.	Hardware-backed keys; attestation monitoring.
Physical coercion	Outside cryptographic scope.	User responsibility.

Threat	Why out of scope	Mitigation (user responsibility)
Exchange/edge correlation	Economic layer, not consensus. When validators exchange external-network assets, that layer may expose graph edges.	Out-of-band disclosure controls.
Global passive adversary (full network view)	QASH's privacy is architectural (no graph broadcast), not transport-layer anonymity. A GPA watching all Domain B channels may correlate sync timing.	Optional Tor routing for clone channels (Domain B feature gate).
Receipt body without disclosure key	By design: requires a genesis-authorized disclosure key.	This is the privacy guarantee, not a limitation.

10.3 §P1 — Dual-layer network architecture

Layer	Purpose	Visible Artifacts	Network Behaviour
Domain B — Admission/Transport	Private TX ingestion, offline sync, ODM/TEE execution	Raw TX envelopes, clone chunks, attestation quotes	QR/NFC/BLE, secure relays, TEE channels. Never publicly broadcast.
Domain A — Consensus/Sync	State commitment propagation, Lyapunov verification, epoch synchronisation	state_root, receipt_root, Lyapunov proofs, validator attestations, epoch seeds	Structured P2P mesh for commitment-only gossip. No TXs, no mempool, no edge graphs.

10.4 §P2 — Transaction lifecycle visibility boundaries

Each step of the transaction lifecycle (§Transaction lifecycle in 03_transactions.md) is explicitly layer-bound:

- Step 1: Decoding → Domain B (private). Bytes arrive via clone channels, TEE-protected relays, or secure admission queues. Never decoded on public network interfaces.
- Step 2: Admission → Domain B (private). Cryptographic blinding, nonce validation, Lyapunov admission checks. Inside PAL/OEM domains or validator TEEs.
- Step 3: Ordering → Domain B → Domain A boundary (internal). Deterministic PRF-derived epoch schedule. No public ordering transcript.
- Step 4: Application → Domain B blinded VM (opaque). Produces blinded execution traces. Intermediates never externally visible.
- Step 5: Replay → Domain A (commitment-only). Validators verify `state_root` and `receipt_root` continuity. Public observers see only root evolution.

Normative rule: Raw transaction envelopes (including `author_id`, `nonce`, `payload_len`, `payload`, `signature`) are Domain B artifacts. They **MUST NOT** appear in `PublicTranscript` (see §P3).

10.5 §P3 — Public observation transcript

```
PublicTranscript(epoch_t) = {
    state_root_t,
    receipt_root_t,
    efb_root_t,
    epoch_t,
    halt_flag_t
}
```

The Rust type `qash_consensus::PublicTranscript` (`crates/consensus/src/public.rs`) encodes this boundary as a compile-time guarantee. Any Domain A code path that inadvertently includes raw envelope fields in the public surface will fail to compile.

All artifacts in `PublicTranscript` are:

- Fixed-length (padded to protocol maximum).
- Uniform type tags (constant epoch commitment structure).
- Indistinguishable under adaptive observation (cascade avalanche ensures no semantic leakage for observers without disclosure keys).
- Decoupled from sender/receiver/amount/action type.

`efb_root_t` is a public commitment over already-public shard state roots, receipt roots, the previous EFB root, shard count, and optional transparent proof batch root. It is permitted in `PublicTranscript` because it does not publish raw receipt leaves, transaction envelopes, admission transport metadata, or edge topology.

10.5.1 §P3a — PublicTranscript change-control process (normative)

`PublicTranscript` is the sole authorised pathway for Class I-visible data. Any code path that emits protocol state to a public-observable channel **MUST** go through

qash_consensus::public::PublicTranscript. Raw EpochState MUST NOT be serialised and transmitted directly.

Gating rule: Any PR that adds a new field to PublicTranscript or widens the public surface in any other way MUST satisfy all of the following before merge:

1. **Add the field** to crates/consensus/src/public.rs:PublicTranscript.
2. **Add a COVERAGE.md row** explaining the privacy implication: what the field reveals, which observer class gains access, and why the privacy cost is acceptable.
3. **Receive explicit sign-off** in the PR from a designated privacy reviewer (tracked in CODEOWNERS as the @privacy-reviewers team).
4. **Update §P4** in this document to reflect the widened class boundary, if applicable.

A PR that widens the public surface without these steps MUST be blocked at review. This requirement is not waivable post-genesis.

Domain B emission gate: The only function in Domain B permitted to write to a public channel is:

```
// crates/pal/src/net.rs (normative target, implementation pending 4-B)
pub fn publish_transcript_entry(
    transport: &impl NetTransport,
    entry: &PublicTranscript,
) -> Result<(), NetError> {
    let bytes = entry.encode_canonical();
    transport.broadcast(&bytes)
}
```

This function is intentionally NOT generic over T. If you need to publish raw EpochState, that is a privacy violation.

10.6 §P4 — Observer classes

10.6.1 §P4a — Normative class taxonomy (Phase 4-A)

QASH defines four normative observer classes. Each class has an exhaustive list of visible artefacts and a hard boundary on what it MUST NOT see. Any protocol change that widens a class boundary (makes previously invisible data visible) is a privacy regression and requires the sign-off process described in §P3a.

Class I — Public observer (unauthenticated internet participant)

Can see: (epoch, state_root, receipt_root, efb_root, halt_flag).
MUST NOT see: validator identities or counts, transaction amounts, sender/receiver, envelope payloads, graph topology, receipt leaf values.

This is the minimal public surface. PublicTranscript in crates/consensus/src/public.rs encodes this as a compile-time type boundary. Any value not present in PublicTranscript is Class I-invisible by construction.

Class II — Authorized validator (Domain A + B, TEE/OEM-protected)

Can see: own validator slot assignment, aggregated divergence metrics (Lyapunov W_D , W_C , W_Σ) for the active epoch, blinded opcodes within own shard.

MUST NOT see: other validators' private signing keys, envelope plaintext of other validators' transactions, exact divergence decomposition of other validators' contributions.

Validators are intentionally linkable across epochs — they are accountable consensus participants. This is architecturally distinct from user privacy.

Class III — Receipt holder (scoped Domain B disclosure)

Can see: own receipt contents when holding the corresponding epoch viewing key (derived from `epoch_seed` via the key-derivation function specified in §P7).

MUST NOT see: other participants' receipts, graph adjacency between receipts, economic metadata of other participants, or receipt contents from epochs after key rotation (forward secrecy).

Receipt decryption requires both the receipt ciphertext and the epoch-scoped viewing key. The viewing key is destroyed after `max_offline_epochs` (12, from `GENESIS_CONSTANTS.toml`) as a forward-secrecy guarantee.

Class IV — Regulatory authority (genesis-authorised disclosure domain)

Can see: specific receipts for which a genesis-authorised disclosure key has been scoped and a lawful-basis disclosure request has been satisfied.

MUST NOT see: receipts outside the disclosure scope, receipt contents from epochs prior to the disclosure key's activation epoch (disclosure is not retroactive), or graph topology beyond the explicitly disclosed receipts.

Class IV access requires: 1. A genesis-authorised disclosure key (not derivable post-genesis). 2. A valid lawful-basis disclosure request (GDPR Art. 6/9, national equivalent). 3. Epoch-scoped decryption — the disclosure key is valid only for a declared epoch range. Past-epoch decryption is cryptographically impossible after key rotation (`epoch_seed` destruction).

This is the forward-secrecy property for Class IV: a disclosure key authorised at epoch T cannot decrypt receipts from epoch $T-k$, even with full regulatory cooperation.

10.6.2 §P4b — Observer class summary table

Class	Name	Domain Access	Can See	MUST NOT See
I	Public observer	Domain A only	<code>state_root</code> , <code>re-</code> <code>ceipt_root</code> , <code>efb_root</code> , <code>epoch</code> , <code>halt_flag</code>	Validator IDs, TX amounts, sender/receiver, graph edges

Class	Name	Domain Access	Can See	MUST NOT See
II	Authorized validator	Domain A + B (TEE/OEM)	Own slot, aggregated divergence, blinded opcodes (own shard)	Other validators' private keys, other TX plaintext
III	Receipt holder	Scoped Domain B	Own receipts with epoch viewing key	Other receipts, graph adjacency, past-epoch receipts after key rotation
IV	Regulatory authority	Genesis-authorized Domain B	Disclosed receipts (epoch-scoped, lawful basis)	Out-of-scope receipts, pre-activation-epoch receipts, graph topology
—	Compromised endpoint	Out of protocol scope	—	TEE/OEM concern; protocol halts before degrading

10.7 §P5 — Validator identity vs user identity

author_id: [u8; 48] in TX-0 is a stable validator consensus identity. Validators are a finite, known set of consensus participants. Their identity linkage across epochs is intentional and irreversible — validators are accountable. This is NOT a user privacy concern because QASH validators are distinct from application-layer users.

TX-0 is a consensus heartbeat (nonce advance). Its author_id carries no sender→receiver→amount edge semantics.

Future user-facing transaction types (TX-2+) that carry user payment identities MUST NOT use stable identifiers. Their §A8 proof obligation MUST include an **epoch-unlinkable identity declaration** with the following required fields:

```
epoch_unlinkable_identity_scheme:
  type: <e.g. "cascade-derived ephemeral per-epoch key" | "ZK-pseudonym" | ...>
  linkage_surface: "none" | "within-epoch" | "within-disclosure-domain"
  unlinkability_argument: <proof sketch or reference>
  coq_proof_obligation: <proof file reference>
```

This requirement is enforceable at spec-review time (pre-genesis admission of the TX-2+ spec document), not post-genesis.

Pre-genesis admission checklist for TX-type specs:

- ☐ §A8 proof obligation references a Coq file (even if Admitted temporarily)
- ☐ epoch_unlinkable_identity_scheme declares linkage_surface = "none" or "within-disclosure-domain"

- `receipt_privacy.body_encryption` specifies key derivation from cascade or epoch seed (see §P7)
- `receipt_privacy.plaintext_at_halt = false` is explicitly stated
- No stable user identifiers appear in public-facing fields

A TX-type spec that fails any checklist item is inadmissible. No post-genesis governance process may waive these requirements.

10.8 §P6 — Leaf obfuscation

Sparse Merkle leaf positions are epoch-relative, derived from `sort_key(entropy_seed_t, TxID( $\tau$ ))` where `entropy_seed_t` changes every epoch (`leaf_index_bytes = 48`, `sparse_merkle_depth = 384` from `GENESIS_CONSTANTS.toml`).

A static Merkle position analysis cannot link transactions across epochs.

10.9 §P7 — Receipt privacy

`receipt_root` commits to blinded execution traces via the hash cascade (SHA3-256 → BLAKE3 → KangarooTwelve). Without a genesis-authorised disclosure capability, no party can extract sender, receiver, amount, action type, or graph adjacency from receipt data or membership proofs.

Receipt bodies are encrypted by default. Selective disclosure requires an explicit disclosure key scoped to a genesis-authorised disclosure domain.

Pre-admission requirement (normative): Any TX type whose payload carries user-identifiable data (sender, receiver, amount, asset type, or graph-inferable metadata) MUST declare a receipt encryption scheme in its spec section BEFORE genesis admission. Specifically, the TX spec must include:

```
receipt_privacy:
  body_encryption: <scheme name and key derivation>
  disclosure_domain: <genesis-authorised scope>
  plaintext_at_halt: false # receipt bodies must not auto-decrypt on halt
```

A TX type that omits this declaration and carries user-identifiable payload data is inadmissible. No post-genesis governance process may retroactively modify receipt privacy properties.

Current state: No Domain A receipt struct exists yet (`06_receipts.md` is deferred). The pre-admission requirement binds the future receipt spec design.

10.10 §P8 — OEM/blinding trust boundary

OEM/TEE protection is a side-channel hardening boundary, not a trusted hardware assumption for privacy. Cryptographic privacy guarantees hold even on untrusted

hardware because all Domain A operations use deterministic blinding (epoch-bound PRF masks; no TRNG in Domain A).

Current Lyapunov factors (implemented in `crates/consensus/src/lyapunov.rs`): `divergence_D`, `conflict_C`, `slash_accum_Σ`. These cover consensus correctness and validator accountability.

blinding_health Lyapunov factor: NOT YET IMPLEMENTED. This is a DEFERRED Domain B spec item. When the Domain B blinding spec is written, it must:

1. Define a `blinding_health` metric and its valid range.
2. Specify the weight and update rule.
3. Add it to the Lyapunov evaluation path in both the spec and `lyapunov.rs`.

Until then, §P8 states the normative target: hardware attestation failure or detected blinding compromise **MUST** trigger an absorbing halt, not degradation to a leaky state. The mechanism for detection is deferred; the halt requirement is not.

10.11 §P9 — Offline clone privacy

The clone protocol (QR/NFC/BLE, `max_offline_epochs = 12`) allows state transitions to accumulate privately before public network commitment. The admission invariant requires that offline state deltas satisfy all transition axioms; the individual transaction history inside the delta is not required to be disclosed publicly.

Hop depth is not restricted at the protocol layer. Enforcement of hop-depth limits, if any, is an OEM/deployment-specific Domain B concern.

10.12 §P10 — Theorem targets

The following theorems are normative targets. Coq proof skeletons are deferred to the Domain B spec revision.

TH-P1 (Public graph non-observability):

For any two admissible transaction sequences `T_a`, `T_b` yielding identical epoch count and halt status, `PublicTranscript(T_a)` and `PublicTranscript(T_b)` are computationally indistinguishable under CPA, differing only in `state_root` and `receipt_root` commitments. No semantic transaction graph, adjacency structure, or economic metadata is externally observable.

TH-P2 (Receipt non-disclosure):

`receipt_root` commits to blinded execution traces via the hash cascade. Without a genesis-authorized disclosure capability, no party can extract sender, receiver, amount, action type, or graph adjacency from receipt data or membership proofs.

TH-P1 dependency chain:

- Domain B blinding implementation (deterministic PRF masks; no TRNG in Domain A)
- Cascade avalanche property: `proofs/privacy/cascade_avalanche_property.v` (deferred)
- ORAM/dummy access pattern correctness: `proofs/privacy/oblivious_access_non_interference.v` (deferred)

TH-P2 dependency chain:

- Receipt encryption scheme (`06_receipts.md`, deferred)
- Disclosure key management spec (deferred)
- ZK membership proof soundness: `proofs/privacy/receipt_proof_soundness.v` (deferred)

The proof file paths above are reserved names. A genesis proposal that lacks TH-P1 and TH-P2 evidence is inadmissible.

11 QASH Clone Protocol — §10

Status: NORMATIVE

Spec version: v1.2

Implements: `GENESIS_CONSTANTS.toml` [`clone_protocol`]

11.1 §10.0 — Purpose and scope

The clone protocol enables QASH validators to synchronize state and submit transactions in environments where the primary consensus mesh is unavailable: radio-jammed zones, air-gapped facilities, bandwidth-constrained deployments, and offline mobile operation.

The clone protocol is a **Domain B** artifact. Raw transaction envelopes and clone chunks are internal Domain B data. The public Domain A surface (`PublicTranscript`) is never extended by clone protocol artifacts; only the resulting `state_root` and `receipt_root` after application are Domain A visible.

11.2 §10.1 — Transport layer abstraction

The clone protocol is transport-agnostic. Any admitted transport satisfies the same interface contract; the consensus layer never observes which transport delivered a given chunk. The genesis-admitted transport set is:

ID	Transport	Typical range	Use case
QR_code	Visual QR scan	Contact	Air-gapped handoff, paper backup

ID	Transport	Typical range	Use case
NFC	ISO 14443 APDU	≤ 10 cm	Tap-to-sync mobile validators
BLE	Bluetooth Low Energy 5.x	≤ 100 m	Standard wireless sync
WiFi_Direct	IEEE 802.11 P2P	≤ 200 m	High-throughput local sync
LoRa	LoRa/LoRaWAN PHY	≤ 15 km	Long-range, low-power rural deployment
Ultrasonic	FSK 24 kHz carrier	≤ 10 m	Radio-jammed / Faraday-cage environments

Transport bonding (simultaneous transmission across multiple transports for redundancy or throughput aggregation) is a Domain B concern and is not specified at this layer. The protocol processes the first valid received copy of each chunk.

11.3 §10.2 — Ultrasonic transport (§10.2 normative; optional/experimental)

Implementation note: The ultrasonic transport is **optional and experimental**. Acoustic hardware support is highly platform-fragmented: mobile SoCs, embedded targets, and desktop hosts vary widely in whether they expose a 24 kHz piezoelectric path, and OS audio stacks impose unpredictable latency. Platform integrators **MUST** gracefully handle transport unavailability (`TransportError::NotAvailable`) and **SHOULD** prefer higher-priority transports (§10.3) whenever available. The `UltrasonicTransport` PAL stub is present for completeness; production deployment requires explicit platform HAL integration and hardware qualification testing.

The ultrasonic transport channel is admitted at genesis for deployment in radio-frequency-denied environments (Faraday cages, military-grade RF jamming, hospital RF-quiet zones, underground facilities).

Physical parameters:

Carrier frequency: 24 kHz (above human audible range ≥ 20 kHz threshold)
Modulation: Binary FSK
Mark frequency: 24.0 kHz (bit 1)
Space frequency: 24.5 kHz (bit 0)
Symbol rate: ≤ 1200 baud (hardware-dependent; 300/600/1200 admitted)
Effective data rate: ~ 100 – 150 bytes/sec at 1200 baud after framing overhead
Max range: ~ 10 m in open air; reduced in acoustically absorptive environments
Hardware: Standard piezoelectric transducer + ADC/DAC; no specialized RF hardware

Framing:

[SYNC 4 bytes][LEN 2 bytes LE][PAYLOAD ≤ 255 bytes][CRC16 2 bytes]
SYNC = 0x55 0x55 0xAA 0x55 (preamble for FSK synchronization)
CRC16 = CRC-CCITT over LEN || PAYLOAD

Ultrasonic chunks use the same Domain B chunk authentication scheme as all other transports (see §10.4). The physical channel provides framing only; confidentiality and integrity guarantees come from the chunk signing layer.

Operational notes: - Ambient noise above 80 dB SPL degrades effective range. - Multiple simultaneous ultrasonic emitters in the same space cause interference; the protocol does not specify TDMA arbitration — implementations SHOULD use exponential-backoff retry. - Ultrasonic is the carrier of last resort in the transport priority order (§10.3); implementations SHOULD prefer higher-bandwidth transports when available.

11.4 §10.3 — Transport selection and priority

When multiple transports are available, the Domain B implementation SHOULD prefer transports in this order (highest throughput / lowest latency first):

1. WiFi_Direct (highest throughput)
2. BLE (dual-mode: concurrent peripheral + central)
3. NFC (contact; zero configuration)
4. LoRa (long range; low throughput)
5. QR_code (air-gap; human-mediated)
6. Ultrasonic (RF-denied fallback; carrier of last resort)

Transport bonding (parallel use of multiple transports) and dynamic switching are Domain B implementation choices and are not constrained by this spec.

11.5 §10.4 — Dual-mode BLE operation

BLE operates in **dual mode**: the validator simultaneously acts as a peripheral (advertising, accepting connections) and as a central (scanning, initiating connections). This removes the master/slave asymmetry that would otherwise require explicit role negotiation.

Implications: - Any two validators within BLE range can exchange clone chunks without prior pairing or role assignment. - Relay validators (§10.5) can accept incoming chunks (peripheral) and immediately forward them (central) without mode-switching delay. - BLE advertisement payloads MUST NOT contain validator identity; they contain only the epoch counter and a random ephemeral session token to prevent cross-epoch linkage by passive BLE scanners.

11.6 §10.5 — Store-and-forward relay

When `store_and_forward = true` (genesis-set), intermediate validators MAY buffer clone chunks for peers that are temporarily unreachable and forward them when contact is re-established.

Invariants: - A relay node MUST NOT apply buffered chunks to its own state before forwarding; application is the responsibility of the destination validator. - Buffered chunks MUST be dropped after `max_offline_epochs` epochs have elapsed since the chunk's embedded epoch counter. Stale chunks are inadmissible. - Buffer contents are Domain B data and are never included in `PublicTranscript`.

11.7 §10.6 — Cover traffic and timing obfuscation

When `cover_traffic = true` (genesis-set), validators MUST emit dummy clone-protocol packets at a constant rate on all active transports, indistinguishable in size and timing from real chunk traffic.

Purpose: A passive observer watching BLE/WiFi-Direct/LoRa traffic cannot determine whether a validator is actively syncing state or is idle. Timing side-channels that would allow graph reconstruction (validator A synced with validator B at time T) are suppressed.

Implementation constraint: Cover packets MUST be: - The same byte length as real chunks (padded to maximum chunk size). - Signed with a per-epoch ephemeral key distinct from the validator's consensus key. - Indistinguishable from real chunks to a passive observer without the epoch disclosure key.

Cover traffic is a Domain B concern; its cryptographic construction is deferred to the Domain B blinding spec (§P8 of `09_privacy_model.md`).

11.8 §10.7 — Binary packet format and compression

Clone chunk payloads are compressed with LZ4 (`packet_compression = "LZ4"`) before chunk authentication. LZ4 is chosen for: - Deterministic compression output given identical input (required for Domain A state-root reproducibility after decompression). - Negligible decompression latency on constrained hardware (ESP32, Cortex-M). - No entropy source required (unlike stream ciphers in compression mode).

Chunk wire format (Domain B):

```
[VERSION u8][EPOCH u64 LE][CHUNK_IDX u16 LE][CHUNK_TOTAL u16 LE]
[COMPRESSED_LEN u16 LE][PAYLOAD ≤ 4096 bytes LZ4-compressed]
[SIG 2420 bytes Dilithium5]
```

- `VERSION = 0x12` for v1.2.
- Decompressed payload MUST NOT exceed `4096 × CHUNK_TOTAL` bytes; oversized decompressed output is inadmissible (prevents decompression-bomb attacks).
- SIG covers all preceding bytes in the chunk (VERSION through PAYLOAD).

11.9 §10.8 — Bloom filter deduplication

When `bloom_filter_dedup = true` (genesis-set), relay nodes maintain a per-epoch Bloom filter of received chunk IDs to suppress redundant retransmission.

Parameters (Domain B recommendation):

Filter size: 8192 bits (1 KB)
Hash functions: 7 (SHA3-256 with index suffix)
Expected FPR: < 0.01 at 1000 chunks/epoch
Reset: At each new epoch (filter is epoch-scoped)

The Bloom filter is a Domain B performance optimization. A false positive (duplicate suppression of a unique chunk) is recoverable: the destination validator will re-request the missing chunk via explicit NACK. The filter **MUST NOT** be used to gate Domain A admission — chunk authentication and epoch validity checks govern admissibility.

11.10 §10.9 — Emergency wipe

When `emergency_wipe_signal = true` (genesis-set), a validator holding a genesis-admitted validator key **MAY** broadcast a signed emergency wipe command. On receipt, an implementation **SHOULD**:

1. Immediately halt all in-progress Domain A transitions (entering absorbing halt state `0x06 HaltFlagSet`).
2. Cryptographically erase all locally cached Domain B secrets (private keys, TEE-bound material, buffered clone chunks).
3. Preserve the public `PublicTranscript` log (state roots, receipt roots) for audit purposes.

Normative constraint: The wipe command **MUST** be authenticated with a validator consensus key and include the current epoch counter; replayed wipe commands from prior epochs are inadmissible.

Purpose: Designed for mobile validators in high-risk physical environments (seizure risk, device compromise). The halt-then-wipe order ensures no partial state is exposed: the Domain A state machine reaches a clean absorbing halt before any erasure operation.

11.11 §10.10 — Offline operation limits

`max_offline_epochs = 12` (genesis-set; ~6 seconds at 500 ms/epoch)

A validator that has been offline for more than `max_offline_epochs` epochs **MUST NOT** apply accumulated clone chunks to advance its local state directly. It must instead re-

synchronize via the full Domain A state-root verification path before resuming normal operation.

This limit exists to bound the maximum Lyapunov window divergence that can accumulate while offline, ensuring that re-joining validators cannot destabilize the convergence invariant.

11.12 §10.11 — Security properties

Property	Mechanism	Status
Chunk authenticity	Dilithium5 signature over full chunk	IMPLEMENTED (genesis-admitted)
Replay resistance	Epoch counter in chunk header; stale epoch → inadmissible	IMPLEMENTED
Observer unlinkability	Cover traffic; ephemeral BLE advertisement tokens	GENESIS-SPECIFIED; Domain B impl required
Graph non-publication	Clone chunks never appear in PublicTranscript	ENFORCED by PublicTranscript type boundary
Decompression safety	Max decompressed size bound in chunk format	SPECIFIED; Domain B impl required
Emergency key erasure	Wipe command + absorbing halt	GENESIS-SPECIFIED; Domain B impl required

11.13 §10.12 — Out of scope (deferred)

- TDMA arbitration for ultrasonic multi-emitter environments
- Transport bonding throughput aggregation
- Clone protocol authentication key rotation schedule
- Domain B blinding of clone chunk payloads (deferred to Domain B blinding spec)
- LoRa network coding / FEC specification
- WiFi-Direct group owner negotiation

12 QASH ABCR Handshake Protocol

Status: Normative (Domain B) **Scope:** Attestation-Backed Challenge-Response (ABCR) admission channel. Zero consensus footprint — no Domain A code changes required.

12.1 §H0 — Transport Security

Before any ABCR protocol steps execute, a forward-secret session must be established between the two parties. QASH uses a post-quantum variant of the Noise Protocol Framework (Perrin, 2018) as the underlying handshake pattern:

QASH ABCR Handshake transport:
Noise_XX_Kyber768_SHA3-256_BLAKE3

Where:

XX = mutual authentication (both parties contribute ephemeral keys)
Kyber768 = post-quantum KEM (NIST PQC / FIPS 203)
SHA3-256 = hash function (already in QASH cascade – NIST FIPS 202)
BLAKE3 = chaining hash (already in QASH cascade)

Rationale: The XX pattern provides mutual authentication without pre-existing shared state; both parties prove possession of their long-term keys during the handshake. Kyber768 replaces the Noise default of X25519 to satisfy post-quantum security requirements consistent with the rest of the QASH crypto cascade.

The ABCR attestation quote and blinding_health_proof ride as handshake **message payloads** after the Noise session is established. This cleanly separates transport security from QASH-specific attestation logic.

12.2 §H1 — ABCR Protocol (normative)

The ABCR handshake solves the “deliver to compromised device” problem that a simple dust-echo cannot: a compromised device can echo dust back. ABCR forces attestation at handshake time, triggering absorbing halt on the recipient device before any response is sent if integrity fails.

12.2.1 Step 1 — Challenge (Sender → Recipient)

Sent as a valueless message via the Noise-secured session. Never broadcast to the public Domain A mesh.

```
challenge = {  
  nonce:          [u8; 32],    // epoch-bound, sender-generated (CSPRNG in Domain B)  
  domain:         "QASH:HANDSHAKE", // domain-separated from all consensus domains  
  epoch_binding:  u64,         // current epoch number – binds challenge to epoch  
  max_value_bound: u128,       // optional policy hint (0 = no limit)  
}
```

12.2.2 Step 2 — Attested Response (Recipient → Sender)

Generated entirely inside TEE/OEM domain before transmission.

```
response = {  
  signature:      PQ_SIG,      // Signs H(nonce || domain || epoch_seed || blinding_hash)
```

```

// using Dilithium5 (primary) per GENESIS_CONSTANTS
attestation_quote: bytes, // Fresh TPM/TEE quote chained to genesis_config.attestation_quote
blinding_health_proof: [u8; 32], // Domain B blinding integrity hash.
// Computed by PAL::Attest over the current
// obfuscation-cascade state; never derived
// from Domain A values. Threshold check is
// policy-configurable but never zero.
epoch_seed_binding: [u8; 32], // Proves response is epoch-
bound via local cascade state
}

```

Absorbing halt trigger: If attestation verification or blinding health check fails during response generation, the device triggers `Halt::absorbing_reset()` and sends no response. The challenge times out on the sender side.

12.2.3 Step 3 — Verification (Sender)

All of the following must hold; any failure aborts the handshake:

1. signature is valid against the recipient's known public identifier
2. attestation_quote chains to genesis_config.attestation_root
3. blinding_health_proof meets the configured minimum threshold
4. nonce matches the sent challenge
5. epoch_binding matches the sender's current epoch
6. epoch_seed_binding is consistent with the local cascade state

On pass: proceed with transfer. On fail: abort — no dust sent, no graph leaked.

12.3 §H2 — Security Properties

Property	Mechanism
Zero consensus footprint	Valueless handshake, peer-to-peer only, no broadcast to Domain A
Offline/async compatible	QR/NFC/BLE transport; response can be cached within epoch
Replay resistance	Domain-separated (QASH:HANDSHAKE), epoch-bound nonce
Forward secrecy	Noise_XX ephemeral keys; compromise of long-term keys does not reveal past sessions
Attested liveness	Quote freshness verified against epoch — stale quotes rejected
Client-policy configurable	max_value_bound allows per-transfer policy enforcement

12.4 §H3 — Threat Model

Threat	ABCR Response
Compromised device (key extracted)	Attestation quote fails genesis root check → absorbing halt before response
Replay of captured response	Epoch-bound nonce — replays from prior epochs are rejected
MITM on transport	Noise_XX mutual auth — both parties authenticate; MITM cannot forge
Simple dust-echo attack	ABCR requires fresh TEE attestation; echo without attestation fails Step 3
Offline timing correlation	Response cacheable within epoch; no fixed timing signature

12.5 §H4 — Integration Layer

Component	Location	Notes
Noise_XX_Kyber768 session	Domain B / PAL Net trait impl	OEM-specific; not in Domain A
Challenge generation	Domain B / wallet/OEM layer	nonce uses CSPRNG from Domain B entropy
Attestation quote	Domain B / PAL Attest trait	TEE/TPM; OEM-specific
Blinding health check	Domain B / PAL blinding layer	Deferred; see 09_pri-vacy_model.md §P8
Absorbing halt on failure	Domain A boundary / PAL Halt trait	Halt::absorbing_reset() from qash-pal

No qash-consensus (Domain A) code is modified by this protocol. The ABCR channel is purely a Domain B admission and transport concern.

12.6 §H5 — Implementation Notes

- Noise_XX_Kyber768_SHA3-256_BLAKE3 is not a published standard at time of writing. QASH implements it as a Noise extension following the Noise spec §9.3 (additional algorithms). The KEM interface uses Kyber768 encaps/decaps in place of the Noise DH primitive.
- The PQ_SIG in Step 2 uses Dilithium5 as specified in GENESIS_CONSTANTS.toml ([crypto_cascade] primary = "Dilithium5").

- Epoch-binding is achieved by including `epoch_seed_t` (from `EpochState`) in the signed payload — this is a Domain A value read at the Domain B boundary; it is a read-only input and does not constitute cross-domain contamination.

13 QASH Security Model

Document: docs/spec/11_security_model.md

Spec version: v1.1.1

Status: NORMATIVE

13.1 §11.1 Threat Surface

QASH's threat model covers four classes of adversarial input:

Class	Description	Primary Defence
Byzantine validators	Validators that submit conflicting or invalid state transitions	Lyapunov convergence gate (H1); slash accumulator (Φ _safety gate H7)
Arithmetic exploitation	Attempts to overflow i128 consensus arithmetic	Checked arithmetic throughout Domain A; overflow → absorbing halt (H2)
Cross-ISA divergence	Subtle differences in fixed-point arithmetic output across authorised ISAs	Domain A determinism rules (no float, no use, checked arithmetic); TH-GC tolerance margin
Side-channel extraction	Power/timing/EM attacks on key material	Blinding framework (§3.7); TH-SB non-interference axiom

The threat model does **not** cover: - Network-level censorship (liveness is out of scope for this document) - Social engineering or governance attacks (QASH is zero-governance by design) - Physical access to validator hardware below the PAL abstraction boundary

13.2 §11.2 Domain Partition Security (Domain A / Domain B)

The fundamental security boundary is the Domain A / Domain B partition defined in `00_execution_model.md` §E0.

Domain A (Deterministic Consensus)

All code in `qash-consensus`. Rules: no unsafe, no f32/f64, no usize/isize, no wall-

clock, no entropy ingress, all arithmetic checked. Replay-invariant across all authorised ISAs.

Domain B (PAL / Operational)

All code in qash-pal and the hosted binary. Nondeterminism and unsafe permitted; `Halt::absorbing_reset()` is the only escape path into Domain A.

Contamination rule: A Domain B value influencing a Domain A computation is a protocol violation even if tests pass. The compiler enforces this via the `no_std/forbid(unsafe_code)` boundary on qash-consensus.

13.3 §11.3 Absorbing Halt Containmentment

QASH's halt mechanism is designed for **local containmentment**: a single validator halting does not affect the canonical chain.

Halt semantics (from `00_execution_model.md` §E5):

Code	Condition	Scope
H1	$\delta_window > \epsilon_halt$	Validator-local
H2	i128 arithmetic overflow	Validator-local
H3	u64 epoch counter overflow	Validator-local
H4	Decode / admissibility failure	Validator-local
H5	State root round-trip failure	Validator-local (reserved)
H6	Explicit external halt flag	Validator-local (reserved)
H7	$\Phi_safety \geq \Phi_max_safe$	Network-wide (ADR-001)

Containment invariant: For H1-H6, `advance_epoch()` returns `Err(halt_reason)` and freezes all state except `halt_reason`. The halt is irreversible and produces no broadcast. The network treats the halted node as offline and continues with the remaining validator set.

H7 (Φ_safety gate) is the sole network-wide halt condition: it is triggered only when the aggregate slash evidence across **all** validators exceeds the Φ_max_safe threshold derived from `i64::MAX`.

13.4 §11.4 Clone Protocol Bounded Reconciliation

Detached or temporarily offline nodes reconcile via the clone protocol (`src/offline/clone.rs`). Admission is bounded:

- **Maximum hops:** 7 (`CloneHop.hop_index` $\in$ $\{0..6\}$)
- **Maximum epoch offset:** 12 (`CloneHop.epoch_offset` $\in$ $\{0..11\}$)
- **Verification:** each `CascadeBoundCloneChunk` carries a cascade inclusion proof (`verify_l7`) that must verify against the epoch's `cascade_root`

A chunk whose proof fails is discarded. A node that cannot reconcile within the hop/epoch bounds must resync from the canonical tip. This ensures no divergent state can be injected into the live chain via the clone path.

13.5 §11.5 Tolerance-Based Divergence Containment (Bat Immunology Model)

QASH treats bounded execution variance (cross-ISA timing differences, transient cryptographic noise, JIT precision drift) as **noise to be absorbed**, not faults to be eliminated. This mirrors bat immunology: tolerance, compartmentalization, and controlled reconciliation — not destructive inflammatory response.

13.5.1 Formal Mapping

Bat Biological Principle	QASH Technical Equivalent	Proof / Guarantee
Tolerance over elimination	3-epoch ConvergenceWindow + $\epsilon_{\text{honest}}/\epsilon_{\text{halt}}$ 10× margin	TH-GC: bounded honest steps never trigger H1 halt
Compartmentalization	Local absorbing halt (H1-H6); network continues	§11.3 containment invariant
Immune filtering	Tiered ISA attestation (Tier A/B/C, 00_execution_model.md §E0)	TH-7: replay invariance across Tier A ISAs
Self-healing reconciliation	Clone protocol bounded admission (≤ 7 hops, ≤ 12 epochs)	§11.4 bounded reconciliation
Non-destructive response	Blinding framework (PRF masking, chunk-key sharing)	TH-SB: non-interference axiom (blinding/blinding_non_interference.)

13.5.2 Tolerance Detection Equation

The **tolerance margin remaining** is a deterministic, Domain A observable computed at the end of every epoch evaluation:

$$\begin{aligned} \text{tolerance_margin_remaining}(t) &= \max(0, \epsilon_{\text{halt}} - \delta_{\text{window}}(t)) \\ &= \max(0, 20_{000} - (V_{\text{convergence}}(t) - \min(\text{preceding_window}))) \end{aligned}$$

This value is carried in `LyapunovEval.tolerance_margin_remaining` and may be observed by Domain B monitoring without influencing consensus.

- `tolerance_margin_remaining > 0`: network is in the tolerance zone; no halt imminent.

- `tolerance_margin_remaining = 0`: the current epoch's δ_window has reached or exceeded ϵ_halt ; H1 triggers.

13.5.3 Formal Guarantee: TH-GC (Grace Convergence)

Theorem TH-GC (proved in `proofs/contractivity/lyapunov_grace_convergence.v`):

If for every epoch t in the evaluation window, the per-epoch change in $V_convergence$ satisfies
 $|V(t) - V(t-1)| \leq \epsilon_honest = 2_000$,
 then $\delta_window(t) \leq WINDOW_SIZE \times \epsilon_honest = 3 \times 2_000 = 6_000 < \epsilon_halt$
 $= 20_000$,
 and therefore `halt_triggered(t) = false`.

Corollary: Under honest operation, $tolerance_margin_remaining(t) \geq \epsilon_halt - 3 \times \epsilon_honest = 14_000 > 0$. The tolerance margin is never exhausted by honest behaviour alone.

Proof sketch: The 3-epoch window stores the three preceding $V_convergence$ values. In the worst case (monotone increasing), V grows by at most ϵ_honest each epoch, so the oldest window value is at most $3 \times \epsilon_honest$ below the current value. The min of the window cannot be smaller than the oldest value, so $\delta_window \leq 3 \times \epsilon_honest$. Since $3 \times \epsilon_honest = 6_000 \leq 20_000 = \epsilon_halt$, TH-3a applies and `halt_triggered = false`.

13.5.4 Why Cross-ISA Divergence Does Not Split the Network

Cross-ISA divergence cannot produce a network fork under QASH's architecture:

1. **Halt is local (H1-H6).** A validator whose $V_convergence$ diverges beyond ϵ_halt enters absorbing halt and drops offline. The canonical chain continues with the remaining set. There is no "split" — one chain continues, one node halts.
2. **δ_window is epoch-bounded.** A single-epoch ISA precision artifact perturbs $V_convergence$ by at most $\alpha \cdot D_max + \beta \cdot C_max + \chi \cdot CH_max$ per validator. For this to exceed ϵ_halt , the artifact would have to move every metric to its maximum simultaneously for every validator — a physically impossible condition for benign ISA variance.
3. **Domain A determinism is ISA-independent by construction.** Fixed-point arithmetic over `i128` with no `usize`, no floats, and checked overflow produces identical outputs across `x86_64`, `aarch64`, and `riscv64` for identical inputs. TH-11 (cross-ISA determinism) is verified by the golden replay CI suite.
4. **TH-GC provides the formal bound.** Honest validators whose per-epoch perturbation stays within ϵ_honest are formally guaranteed (no Admitted) to never trigger H1 halt, regardless of ISA.

13.6 §11.6 Formal Guarantee Summary

Theorem	Statement	Proof file	Status
TH-3a	$\delta \leq \varepsilon \rightarrow \text{halt} = \text{false}$	contractivity/lyapunov_proved_identity.v	PROVED
TH-3b	$\text{halt} \leftrightarrow \delta > \varepsilon$	contractivity/lyapunov_proved_identity.v	PROVED
TH-3c	$\text{FinalizeEpoch} \rightarrow V = 0$	contractivity/lyapunov_proved_identity.v	PROVED
TH-9	$\text{CH_t} \in [0, p],$ $\chi \cdot \text{CH_t}$ no overflow	cascade/cascade_health_proved.v	PROVED
TH-GC	$\varepsilon_{\text{honest-bounded steps}} \rightarrow$ $\delta \leq 3 \times \varepsilon_{\text{honest}} <$ $\varepsilon_{\text{halt}} \rightarrow \text{no halt}$	contractivity/lyapunov_proved_convergence.v	PROVED
TH-10	H_cascade collision resistance	cascade/cascade_collision_axiom (Axiom A6-3)	AXIOM
TH-SB	Blinding non-interference	blinding/blinding_no_interference.v	AXIOM (Axiom A7-3)
TH-7	Replay invariance across Tier A ISAs	golden replay CI suite	CI-verified
TH-11	Cross-ISA output identity	cascade/cascade_deterministic + CI	CI-verified

14 QASH Sharded Protocol

14.1 §12.1 — Scope

Sharding is part of the QASH protocol structure, not merely an implementation module. A shard is a deterministic execution partition whose public outputs are committed into the epoch’s global public transcript through an Epoch Finality Beacon (EFB).

The sharded protocol preserves the existing QASH invariants:

- one-shot genesis-pinned rules
- no governance-controlled shard count changes
- deterministic cross-ISA execution
- no proposer trust for finality
- no raw transaction or graph publication in Domain A
- absorbing halt on invariant violation

Domain A observes only commitments. Domain B performs admission, transport, private execution, and proof generation.

14.2 §12.2 — Shard Assignment

Validators are assigned to execution shards by a deterministic Domain A function:

```

assign_shard(epoch_seed, validator_id, shard_count, bond_weight)
= H_domain(ShardAssignment,
    epoch_seed ||
    validator_id ||
    bond_weight_be ||
    shard_count_be
)[0..8] mod shard_count

```

Constraints:

- shard_count > 0
- shard_count <= 1024
- validator_id is the stable 48-byte consensus identity
- bond_weight is genesis/rules derived and cannot be chosen at runtime to bias placement
- reshuffling is driven only by the epoch seed

This keeps assignment deterministic while binding placement to an identity-cost or stake/bond-cost input. A deployment that sets all bond_weight = 0 is a valid test mode but weakens shard-capture resistance.

14.3 §12.3 — Shard Commitments

Each shard emits one public commitment tuple per epoch:

```

ShardCommitment = {
    shard_id:      u32,
    state_root:    [u8; 32],
    receipt_root: [u8; 32]
}

```

The EFB input list **MUST** contain exactly shard_count entries, sorted by shard_id, with IDs 0..shard_count-1. Missing, duplicate, out-of-range, or reordered shard commitments are invalid.

Shard state execution itself remains Domain B/private. Domain A accepts only the commitment and the verification artifacts required by the selected proof mode.

14.4 §12.4 — Cross-Shard Receipts

Cross-shard communication is receipt-based and lock-free:

```

CrossShardReceipt = {
    epoch:      u64,
    source_shard: u32,
    target_shard: u32,
    nonce:      u64,
    payload_hash: [u8; 32]
}

```

```
}
```

The receipt identifier is:

```
receipt_id = H_domain(CrossShardReceipt,  
    epoch_be ||  
    source_shard_be ||  
    target_shard_be ||  
    nonce_be ||  
    payload_hash ||  
    reserved_inclusion_commitment  
)
```

Validity rule:

```
valid(receipt, efb) iff  
    receipt.epoch == efb.epoch  
    and receipt.source_shard < efb.shard_count  
    and receipt.target_shard < efb.shard_count  
    and receipt_id is included in the source shard receipt_root  
    and efb.efb_root is the finalized EFB root for that epoch
```

The Rust scaffold enforces the epoch/shard anchor, deterministic receipt ID, and Merkle inclusion of the receipt ID in the source shard receipt_root.

Replay rule:

- a receipt MUST NOT be accepted outside its EFB epoch
- (epoch, source_shard, target_shard, nonce) MUST be unique for a source shard
- reusing a nonce in another epoch is not replay, because the epoch is committed into the identifier

14.5 §12.5 — Epoch Finality Beacon

The Epoch Finality Beacon is a public Domain A checkpoint:

```
EpochFinalityBeacon = {  
    epoch:                u64,  
    previous_efb_root:    [u8; 32],  
    shard_count:          u32,  
    aggregate_state_root: [u8; 32],  
    aggregate_receipt_root: [u8; 32],  
    zk_batch_root:        [u8; 32],  
    efb_root:              [u8; 32]  
}
```

Aggregation is deterministic over sorted shard commitments:

```
aggregate_state_root =  
    fold H_domain(EpochFinalityBeacon,
```

```

previous_aggregate || shard_id_be || state_root)

aggregate_receipt_root =
  fold H_domain(EpochFinalityBeacon,
    previous_aggregate || shard_id_be || receipt_root)

```

The final EFB root commits to:

```

epoch ||
previous_efb_root ||
shard_count ||
aggregate_state_root ||
aggregate_receipt_root ||
zk_batch_root

```

No proposer authority is required. Any node can compute the unique valid EFB candidate from the same sorted shard commitment set. Candidates that omit, duplicate, or reorder shard roots fail independent verification.

14.6 §12.6 — ZK-STARK Verification Boundary

ZK is not a consensus shortcut and not a governance switch. It is transparent proof compression for shard execution verification.

Modes:

- **Replay mode:** validators deterministically re-execute shard transitions.
- **STARK batch mode:** shard execution proofs are generated in Domain B and committed by `zk_batch_root` in the EFB.

In STARK batch mode, acceptance requires:

- transparent STARK proofs only
- proof statements bound to `(epoch, shard_id, state_root, receipt_root)`
- recursive aggregation root committed in `zk_batch_root`
- proof verifier semantics tracked in Coq/Rocq proof obligations

The provisional v1.2 ZK profile recovered from PR #93 is:

```

ZkProfile {
  engine: Plonky3,
  proof_family: FRI-STARK,
  inner_circuit_hash: Poseidon,
  outer_commitment_hash: QASH H_domain / cascade commitment surface,
  recursion_depth: 2,
  layer0: shard validity proof, no recursion,
  layer1: 16:1 recursive aggregation,
  layer2: EFB verification of aggregation proofs only
}

```

This fixes the protocol shape without introducing a production verifier. Domain A validates only the public profile metadata and commits to `zk_batch_root`. Domain B owns proof generation, proof-byte transport, and verifier backends. The EFB is the only consensus path that may admit a ZK batch root. Shards may prove asynchronously, but shard-local proving latency is not part of the sub-50ms global commitment path.

If `zk_batch_root` is zero, the deployment is in replay mode. It must not claim constant-time global verification.

If `zk_batch_root` is non-zero, the profile metadata must match the fixed Plonky3/FRI/Poseidon/QAS profile above. Other engines, deeper recursion, KZG SNARKs, Bulletproofs, or alternate aggregation factors are separate protocol extensions and must not be accepted under this profile ID.

14.7 §12.7 — Public Transcript

The public observation surface becomes:

```
PublicTranscript(epoch_t) = {
    state_root_t,
    receipt_root_t,
    efb_root_t,
    epoch_t,
    halt_flag_t
}
```

`efb_root_t` is safe to publish because it is a fixed-length commitment over public roots. It does not expose raw transactions, receipt leaves, validator edge topology, admission transport, sender, receiver, amount, or action type.

14.8 §12.8 — Implementation Mapping

Rust:

- `crates/consensus/src/sharding.rs`
 - `assign_shard`
 - `CrossShardReceipt`
 - `receipt_id`
 - `receipt_is_epoch_anchored`
 - `verify_receipt_inclusion`
 - `ShardCommitment`
 - `EpochFinalityBeacon`
 - `compute_efb`
- `crates/consensus/src/public.rs`
 - adds `efb_root` to `PublicTranscript`
- `crates/consensus/src/envelope.rs`
 - v1.2 envelope includes explicit `shard_id`

- `crates/consensus/src/transition.rs`
 - `advance_epoch_sharded` computes the EFB during the epoch commit path
 - `TransitionResult.public_transcript` publishes `(state_root, receipt_root, efb_root)`
 - v1.2 state roots commit to the aggregate receipt root and EFB root
- `tests/vectors/vectors.v1.2.json`
 - pins multi-epoch sharded replay roots for cross-ISA verification

Proof tracking:

- `proofs/sharding/efb_determinism.v`
 - EFB aggregation is deterministic for identical inputs
 - receipt epoch binding rejects cross-epoch replay

Deferred implementation work:

- STARK verifier/aggregator feature gate
- Plonky3-backed Domain B verifier implementing the fixed profile
- Poseidon circuit transcript with QASH-native outer commitment binding
- 2-layer recursion corpus: Layer 0 shard proofs, Layer 1 16:1 aggregation, Layer 2 EFB batch-root verification
- adversarial shard-capture simulation with configured bond weights

15 QASH Zero-Persistence Execution Pipeline

15.1 §14.1 — Purpose

This specification defines **Zero-Persistence Graph Non-Publication** for QASH Domain B.

The goal is not merely to encrypt or hide transaction-graph material. The goal is to ensure that raw graph material is never promoted into a durable, serializable, replayable, or public artifact. Raw envelopes may exist only inside an owned ephemeral admission lifecycle. The only admissible outputs of that lifecycle are fixed-width scalar commitments, `CapToken<ValidatedEffect>` values, blind audit events, and public transcript roots.

This is a production-mode constraint. The current hosted PAL replay scaffold may retain raw fixtures under explicit testing features, but that mode is not a zero-persistence deployment profile.

15.2 §14.2 — Definitions

Raw graph material means any value that can reveal transaction topology, receipt body contents, graph edges, raw envelopes, peer/network metadata, transaction lists, routing topology, or payload-bearing error context.

Ephemeral envelope means the owned, non-serializable, non-cloneable, raw admission buffer that temporarily contains encrypted or decrypted envelope bytes inside Domain B.

Validated effect means the fixed-width, schema-validated result of admission. It contains only scalar commitments and bounded public metadata required by Domain A.

Zero-persistence production mode means a PAL build profile in which raw graph material is not written to WAL, logs, metrics, traces, crash dumps, audit events, or public transcripts.

Sovereign Hardened profile means the high-assurance deployment tier that adds attested DPU/SmartNIC admission, confidential host memory, IOMMU lockdown, and hardware-backed storage/erasure evidence to the zero-persistence software profile.

15.3 §14.3 — Normative Invariants

15.3.1 ZP-1: No durable raw graph material

Domain B MUST NOT persist, serialize, log, trace, metric-label, crash-dump, audit, or publicly emit raw graph material.

Forbidden durable outputs include:

- raw envelopes or raw transaction bytes,
- graph edges or adjacency lists,
- receipt bodies or unencrypted receipt leaves,
- peer IP addresses, socket addresses, or routable peer identifiers,
- payload-bearing error messages,
- debug strings derived from raw envelope bytes,
- proof-byte contents unless separately admitted by a proof-backend spec.

15.3.2 ZP-2: No owned payload copies in the admission path

The production admission path MUST NOT allocate or create owned copies of raw envelope contents. Parsing must operate over borrowed views or fixed-size local arrays. The admission path MUST NOT use `Vec<u8>`, `String`, `.to_vec()`, `.clone()`, `Serialize`, `Debug`, `Display`, or equivalent trait paths for raw envelope types.

15.3.3 ZP-3: Commitment-only Domain A crossing

The only values that may cross from Domain B into Domain A are:

- `CapToken<ValidatedEffect>`,
- scalar effect roots,
- receipt roots,
- EFB roots,
- bounded epoch/profile metadata already admitted by the relevant protocol spec.

Domain A MUST NOT receive raw envelopes, cover traffic, traffic-shaping metadata, network timing, peer metadata, proof-generation state, or indexer state.

15.3.4 ZP-4: Blind audit only

Audit events MAY record boundary events such as admission success/failure class, key zeroization, attestation status, halt trigger class, and shred commitment publication. Audit events MUST NOT contain raw graph material or peer metadata.

15.3.5 ZP-5: Feature bifurcation

The repository MUST preserve a strict distinction between replay scaffolding and production zero-persistence:

- replay-scaffold: test/golden-vector mode; may use raw fixture WALs for evidence generation only.
- zero-persistence: production admission mode; WAL schema lacks raw payload fields and cannot compile code paths that persist raw graph material.
- sovereign-hardened: zero-persistence plus hardware admission and storage controls.

15.4 §14.4 — Admission Lifecycle

The production lifecycle is:

```
network ingress
-> ephemeral admission slot
-> in-place parser view
-> schema validation
-> scalar commitment extraction
-> CapToken<ValidatedEffect>
-> Domain A transition
-> PublicTranscript roots
-> slot zeroization and reuse/destruction
```

No stage in this pipeline may publish or persist raw graph material. Processing and destruction are coupled by ownership: the raw envelope is consumed by value by the admission airlock, and its storage is zeroized before the slot is returned to a ring, allocator, or caller.

Reference Rust shape:

```
use core::marker::PhantomData;
use zeroize::Zeroizing;

pub struct EphemeralEnvelope {
    data: Zeroizing<[u8; 4096]>,
    _no_send: PhantomData<*mut ()>,
}
```

```

pub struct ValidatedEffect {
    pub effect_root: [u8; 32],
    pub receipt_root: [u8; 32],
    pub epoch: u64,
}

pub fn process_envelope(slot: EphemeralEnvelope) -> Result<ValidatedEffect, PalError> {
    let view = parse_in_place(slot.data.as_ref())?;
    let effect = validate_effect_view(view)?;
    Ok(effect.to_commitment())
}

```

The function consumes slot by value. Borrow-only entrypoints are insufficient for the production invariant because the caller would retain lifetime control.

15.5 §14.5 — WAL and Replay Boundary

The hosted replay scaffold may persist canonical raw test fixtures only under an explicit non-production feature. This exists to generate proof vectors and cross-ISA replay evidence.

A zero-persistence production WAL may contain only:

- epoch number,
- admitted scalar commitments,
- receipt root,
- EFB root,
- validation failure class without payload bytes,
- blind audit event identifiers,
- ShredCommitment identifiers,
- attestation status summaries without raw quotes unless separately approved by the compliance profile.

It MUST NOT contain raw_txs, raw envelope bytes, peer metadata, receipt bodies, or graph topology.

15.6 §14.6 — Hardware Profiles

15.6.1 Global Standard profile

The Global Standard profile implements the software zero-persistence invariants: non-serializable ephemeral types, admission-path no-copy discipline, production WAL schema lock, blind audit events, and CI gates.

15.6.2 Sovereign Hardened profile

The Sovereign Hardened profile adds hardware isolation:

- DPU/SmartNIC admission enclave receives raw packets before host memory.
- The DPU emits only scalar commitments or bounded admission records over PCIe.
- Host DMA is restricted by IOMMU policy to commitment-sized regions.
- Host RAM uses confidential-memory technology such as AMD SEV-SNP, Intel TDX, or ARM CCA where available.
- DPU firmware, admission code, and host policy are attested and pinned to the deployment profile.
- Tier 1/Tier 2 storage keys are generated and destroyed inside an HSM/TPM or sovereign-equivalent boundary.

These controls are deployment-profile requirements. They are not implied by the base software scaffold.

15.7 §14.7 — CI and Verification Gates

Before zero-persistence can be treated as implemented, the repository must add and keep green the following gates:

```
cargo check -p qash-pal --no-default-features --features zero-persistence
cargo test -p qash-pal --features zero-persistence admission_no_raw_persistence
cargo test -p qash-pal --features zero-persistence production_wal_rejects_payload_bytes
cargo test -p qash-pal --features zero-persistence ephemeral_error_redaction
cargo test -p qash-consensus public_transcript_contains_no_graph_fields
```

A Kani or equivalent model-checking harness should prove that the production admission path does not allocate, does not store raw envelope pointers in global state, and cannot call the WAL raw-fixture writer.

Static tripwires are allowed as defense in depth but are not sufficient alone:

```
rg "Vec<u8>|\.clone\(\)|\.to_vec\(\)|Serialize|Debug|Display" crates/pal/src/admission
rg "payload|raw_tx|peer_ip|socket_addr" crates/pal/src/audit crates/pal/src/logging
```

Any intentional hit must be reviewed and justified in the PR body.

15.8 §14.8 — Claims Boundary

Allowed technical claim after implementation and evidence:

QASH zero-persistence production mode has no admissible software path that persists, serializes, logs, or emits raw transaction-graph material; Domain A receives only validated scalar effects and public roots.

Allowed Sovereign Hardened claim after hardware evidence:

Under the declared Sovereign Hardened threat model, raw graph material is confined to an attested admission boundary and is not present in host memory, host WALs, public transcripts, or Domain A state.

Disallowed generic claims without additional legal and deployment review:

- subpoena immune,
- impossible to observe,
- mathematically impossible to recover,
- kernel never sees any packet in all deployments,
- raw material never exists anywhere.

The engineering term for this profile is **Zero-Persistence Graph Non-Publication**.

16 QASH Trustless Genesis and Vendor-Agnostic Local Hardware OpSec

16.1 §15.1 — Purpose

This document prevents QASH from drifting into enterprise PKI, foundation custody, trusted setup, ceremony-based bootstrapping, or vendor-specific hardware requirements.

QASH is designed to be a governance-free deterministic state-transition substrate. Its genesis and liveness properties must not depend on private humans, vendor hardware, custody ceremonies, trusted dealers, or procurement choices.

16.2 §15.2 — Trustless Genesis Axiom

Genesis is a deterministic mathematical event.

The genesis artifact set may include:

- locked specification hashes,
- source and build artifact hashes,
- genesis constants,
- public entropy references,
- reproducible-build evidence,
- proof-object and traceability manifests.

The genesis artifact set **MUST NOT** require:

- a foundation or governance signature,
- a trusted setup ceremony,
- multi-party toxic-waste generation,
- private validator whitelists,
- vendor hardware attestations,

- human quorum approval,
- network master keys,
- any private secret that must be trusted by later participants.

Anyone must be able to independently reconstruct and verify genesis from public artifacts.

16.3 §15.3 — Anti-Ceremony Rules

16.3.1 AC-1: No trusted setup

QASH MUST use transparent cryptographic assumptions only for protocol genesis. No CRS ceremony, toxic-waste ceremony, trusted dealer, or hidden setup secret is permitted.

16.3.2 AC-2: No genesis cabal

No person, organization, foundation, validator set, hardware vendor, or custody committee may possess special authority to create, approve, unlock, or validate the network genesis state.

16.3.3 AC-3: No protocol-critical vendor hardware

No hardware authenticator, smart card, HSM, TPM, DPU, TEE, cloud KMS, sovereign secure element, or vendor-specific device may be required to run a conforming QASH node in the permissionless profile.

16.3.4 AC-4: No human liveness dependency

Routine transaction admission, epoch closure, finality, replay, halt handling, and global liveness recovery MUST NOT require physical touch, PIN entry, smart card insertion, human quorum, manual ceremony, HSM-admin approval, or cloud-KMS approval.

16.3.5 AC-5: No hardware-derived consensus semantics

Hardware attestation, hardware identifiers, HSM/KMS audit logs, FIDO assertions, PIV certificates, TPM quotes, enclave reports, secure-element state, or local operator policies MUST NOT change Domain A state transitions, halt semantics, finality rules, or public transcript semantics.

16.3.6 AC-6: Standards before vendors

Normative QASH specifications MUST describe optional hardware integration by standards, interface classes, and evidence classes rather than vendor-specific products.

Vendor-specific OIDs, CLI commands, object schemas, audit-log formats, daemon names, and SDK assumptions belong in optional backend guides or conformance profiles, not in the protocol authority layer.

16.4 §15.4 — Permitted Local Hardware OpSec

Hardware security is permitted as optional local node protection.

Permitted Domain B uses include:

- FIDO2/CTAP2/WebAuthn or equivalent for operator SSH and administrative access,
- PIV/FIPS 201 or equivalent smart-card profiles for local administrative keys,
- PKCS#11 or equivalent HSM interfaces for local validator key wrapping,
- TPM 2.0 or equivalent measured-boot attestation for local platform evidence,
- FIPS 140-3 or sovereign-equivalent HSMs for local custody requirements,
- HSM-backed or KMS-backed local audit MAC chains,
- release-signing keys for a specific distributor or maintainer,
- local backup and disaster recovery,
- optional threshold signing for one operator's own node key,
- hardware-backed erasure evidence for one operator's local storage.

These controls may improve local security. They do not become network security assumptions. Their failure affects only the local operator.

16.5 §15.5 — Vendor-Agnostic Hardware Boundary

Hardware components may be useful in QASH deployments, but only within this local OpSec boundary:

Capability class	Permitted role	Forbidden role
FIDO2/CTAP2/WebAuthn authenticator	operator login, admin approval, local recovery	consensus signing, epoch finality, genesis authority
PIV/FIPS 201 smart card or equivalent	local release signing, local recovery, optional maintainer keys	protocol master key, required genesis ceremony
PKCS#11/FIPS/sovereign HSM	local wrap keys, audit MACs, opaque local objects	network master key, consensus halt oracle
TPM 2.0 or equivalent platform attestor	local measured-boot evidence	Domain A input, permissionless validator requirement
TEE/DPU	optional local hardening for admission/key handling	protocol authority, genesis dependency

Capability class	Permitted role	Forbidden role
Hardware audit log	local custody evidence	Domain A halt trigger or global liveness condition

No vendor product is treated as a native requirement for QASH. Yubico, Nitrokey, Google Titan, Feitian, Thales, Utimaco, Entrust, Infineon, AWS CloudHSM, Azure Managed HSM, national/sovereign HSMs, or equivalent devices may appear in operator guides as backend examples only.

PQC hot-path execution belongs in software, TEE, DPU, or future PQC-capable local hardware profiles. It remains Domain B local deployment infrastructure and must not become a permissionless protocol requirement.

16.6 §15.6 — Public Transcript Privacy Rule

Raw hardware identifiers are forbidden in global public transcript surfaces.

Forbidden public fields include:

- hardware-authenticator serial numbers,
- FIDO AAGUIDs,
- TPM endorsement keys,
- HSM or smart-card serial numbers,
- certificate subject names,
- cloud KMS resource identifiers,
- enclave measurement metadata tied to a persistent operator identity,
- stable device fingerprints.

Permissioned or sovereign deployments may derive blinded fingerprints for private Domain B registries:

```
hardware_fingerprint = H(profile_salt || device_identifier_set)
```

Such fingerprints remain deployment metadata. They are not Domain A inputs and are not global public transcript fields unless a future privacy-reviewed spec explicitly admits them.

16.7 §15.7 — Failure Semantics

Local hardware failure must remain local.

Failure	Permitted consequence	Forbidden consequence
Lost authenticator or smart card	operator loses local admin/recovery path	network halt
HSM/KMS log full or unavailable	local alert/quarantine	Domain A absorbing halt

Failure	Permitted consequence	Forbidden consequence
TEE/DPU/TPM attestation mismatch	local node not admitted to private profile	global genesis rejection
Threshold signer unavailable	operator's validator may miss signing	global finality dependency
HSM/KMS erased local key	local key unavailable	protocol state mutation

16.8 §15.8 — Review Gate

Future specs, ADRs, roadmap items, and implementation PRs must be rejected or rewritten if they introduce:

- trusted setup,
- genesis ceremonies,
- foundation-controlled network keys,
- human quorums for protocol liveness,
- vendor hardware as a protocol requirement,
- local HSM/KMS/TEE/TPM/DPU state as a Domain A input,
- raw hardware identifiers in public transcript surfaces.

This review gate applies even when the proposal improves local security.